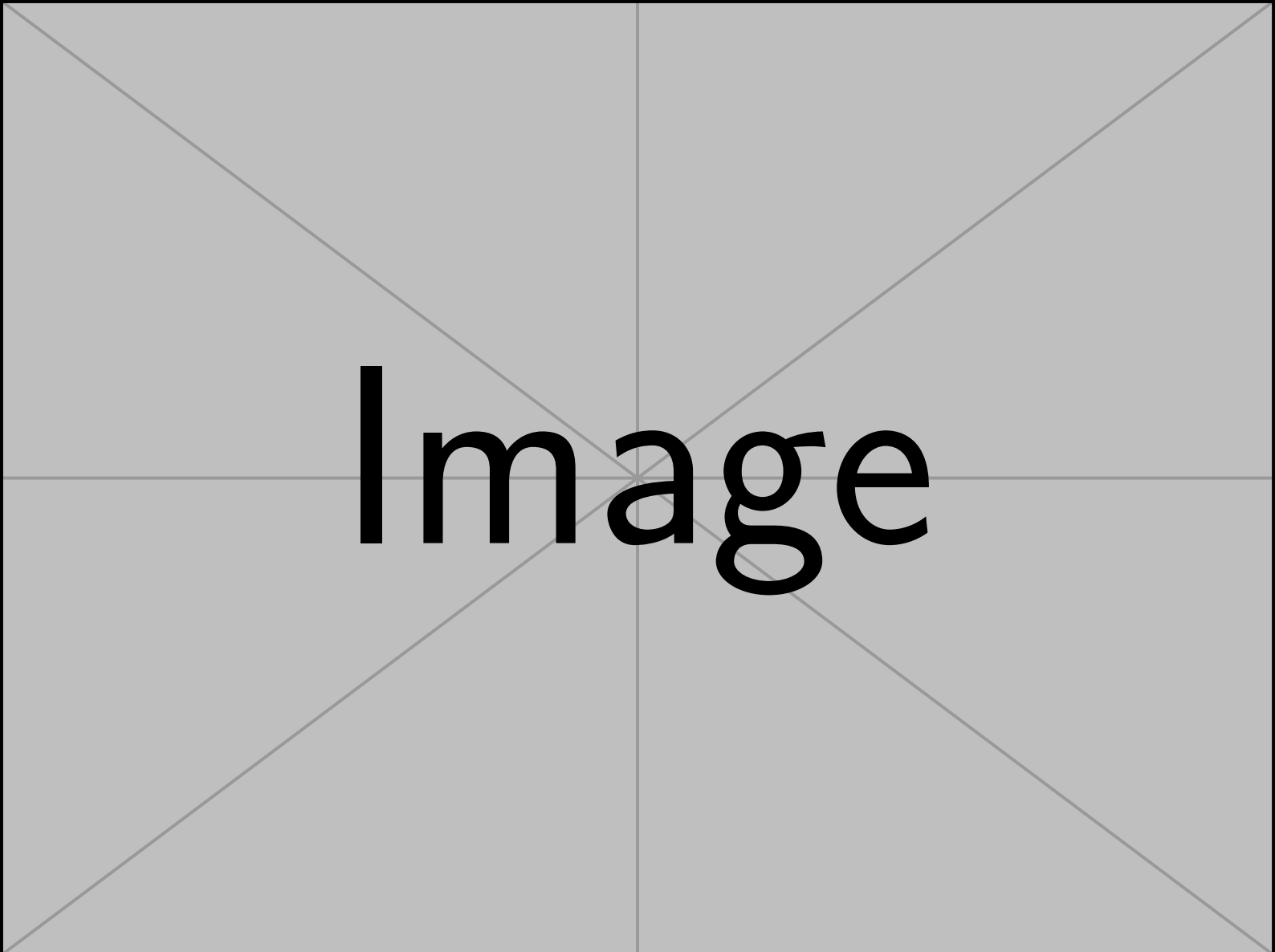




Image

Modul Perkuliahan Desain Web

Panduan Praktik dan Latihan Terstruktur

Author: Tim Pengampu Mata Kuliah Desain Web

Institute: Program Studi / Fakultas

Date: September 5, 2026

Version: 1.0

Penggunaan: Untuk kalangan mahasiswa

Daftar Isi

Bab 1	Pengantar Web & Setup Lingkungan Kerja	1
1.1	Pengenalan Web	1
1.2	Alur Kerja <i>Design-to-Code</i>	1
1.3	Instalasi VS Code & Ekstensi	2
1.4	Akun GitHub & Repository Pertama	3
1.5	Latihan Mandiri	6
1.6	Troubleshooting	6
1.7	Rangkuman	6
Bab 2	Figma Dasar & Pengantar UI/UX	8
2.1	Pengenalan Figma	8
2.2	Membuat Wireframe Halaman Web	10
2.3	Tiga Konsep Dasar UI/UX	11
2.4	Latihan Mandiri	13
2.5	Troubleshooting	14
2.6	Rangkuman	14
Bab 3	HTML Dasar: Struktur & Semantic Tag	15
3.1	Struktur Dokumen HTML	15
3.2	Elemen-Elemen Dasar HTML	16
3.3	Semantic HTML	18
3.4	Elemen Pendukung Penting	21
3.5	Menghubungkan HTML ke CSS (Dasar)	23
3.6	Validasi HTML	24
3.7	Latihan Mandiri	25
3.8	Troubleshooting	26
3.9	Rangkuman	26
Bab 4	HTML Lanjutan: Form, Tabel & Multimedia	27
4.1	Form HTML	27
4.2	Tabel HTML	33
4.3	Multimedia: Gambar, Video, Audio	37
4.4	Praktik: Menggabungkan Semua Elemen	39
4.5	Validasi & Push ke GitHub	44
4.6	Latihan Mandiri	45
4.7	Troubleshooting	46
4.8	Rangkuman	46
Bab 5	CSS Dasar: Selector, Box Model, Typography	47
5.1	Pengantar CSS	47
5.2	Praktikum 1 — Menyiapkan Proyek	48

5.3	Praktikum 2 — Cara Menulis CSS	48
5.4	Praktikum 3 — Selector	50
5.5	Praktikum 4 — Box Model	51
5.6	Praktikum 5 — Typography	53
5.7	Latihan Mandiri	55
5.8	Hasil yang Diharapkan	55
5.9	Troubleshooting	56
5.10	Rangkuman	56
Bab 6	CSS Layout: Flexbox	57
6.1	Pengantar Flexbox	57
6.2	Praktikum 1 — Menyiapkan Proyek	57
6.3	Praktikum 2 — Flex Direction	59
6.4	Praktikum 3 — Justify Content & Align Items	60
6.5	Praktikum 4 — Flex Wrap & Gap	61
6.6	Praktikum 5 — Properti Flex pada Item	62
6.7	Praktikum 6 — Membangun Navbar dengan Flexbox	63
6.8	Praktikum 7 — Galeri Foto dengan Flexbox	65
6.9	Latihan Mandiri	66
6.10	Hasil yang Diharapkan	67
6.11	Troubleshooting	68
6.12	Rangkuman	68
Bab 7	CSS Layout: Grid & Responsive Design	69
7.1	Pengantar CSS Grid	69
7.2	Praktikum 1 — Menyiapkan Proyek	70
7.3	Praktikum 2 — Mengatur Kolom dan Baris	71
7.4	Praktikum 3 — Menempatkan Item di Grid	72
7.5	Praktikum 4 — Pengenalan Media Query	74
7.6	Praktikum 5 — Mobile-First	76
7.7	Praktikum 6 — Membuat Layout Responsif Lengkap	77
7.8	Latihan Mandiri	80
7.9	Hasil yang Diharapkan	81
7.10	Troubleshooting	82
7.11	Rangkuman	82
Bab 8	Git & GitHub Dasar: Init, Commit, Push, Pull	84
8.1	Apa itu Git?	84
8.2	Apa itu GitHub?	84
8.3	Alur Kerja Dasar	84
8.4	Praktikum 1 — Memeriksa Instalasi Git	85
8.5	Praktikum 2 — Konfigurasi Git	85
8.6	Praktikum 3 — Inisialisasi Repository	86
8.7	Praktikum 4 — Membuat File .gitignore	87

8.8	Praktikum 5 — Commit Pertama	87
8.9	Praktikum 6 — Membuat Repository di GitHub	88
8.10	Praktikum 7 — Menghubungkan dan Push	89
8.11	Praktikum 8 — Mengubah File dan Commit Lagi	90
8.12	Praktikum 9 — Pull (Mengambil Perubahan)	90
8.13	Latihan Mandiri	91
8.14	Hasil yang Diharapkan	91
8.15	Troubleshooting	92
8.16	Rangkuman	92
Bab 9	GitHub Kolaborasi: Branch, Pull Request, GitHub Pages	93
9.1	Apa itu Branch?	93
9.2	Apa itu Pull Request (PR)?	93
9.3	Apa itu GitHub Pages?	93
9.4	Praktikum 1 — Membuat Branch	94
9.5	Praktikum 2 — Bekerja di Branch Baru	94
9.6	Praktikum 3 — Menggabungkan Branch (Merge)	95
9.7	Praktikum 4 — Push Branch ke GitHub	96
9.8	Praktikum 5 — Membuat Pull Request (PR)	97
9.9	Praktikum 6 — Merge Pull Request	98
9.10	Praktikum 7 — Menyiapkan GitHub Pages	99
9.11	Praktikum 8 — Memperbarui Website di GitHub Pages	100
9.12	Latihan Mandiri	101
9.13	Hasil yang Diharapkan	101
9.14	Troubleshooting	102
9.15	Rangkuman	102
Bab 10	JavaScript Dasar: Variabel, Fungsi, Kondisi & DOM Selection	103
10.1	Persiapan	103
10.2	Menulis Skrip Pertama	103
10.3	Variabel & Tipe Data	104
10.4	Fungsi	106
10.5	Kondisi (If/Else)	107
10.6	DOM Selection	108
10.7	Latihan	111
10.8	Troubleshooting	113
Bab 11	JavaScript: Event Handling & Manipulasi DOM	114
11.1	Event Handling: onclick & addEventListener	114
11.2	Event pada Form & Input	116
11.3	Manipulasi DOM: Mengubah Konten	118
11.4	Manipulasi DOM: Membuat & Menghapus Elemen	120
11.5	Mini Proyek: Toggle Menu Mobile	123
11.6	Latihan	125

11.7 Troubleshooting	127
Bab 12 Pengenalan Tailwind CSS	128
12.1 Konsep: Apa Itu Tailwind CSS?	128
12.2 Setup Tailwind di Proyek	129
12.3 Eksplorasi Utility Class Dasar	130
12.4 Konversi: CSS Manual ke Tailwind	133
12.5 Layout dengan Tailwind: Flexbox & Grid	136
12.6 Latihan	138
12.7 Troubleshooting	140
Bab 13 Pengenalan React: Komponen, JSX & Props	141
13.1 Mengapa React?	141
13.2 Membaca Kode React (Tanpa Menjalankan)	142
13.3 JSX: HTML yang “Bisa Dikode”	145
13.4 Komponen: Blok Bangunan UI	146
13.5 Props: Mengirim Data ke Komponen	147
13.6 Demo Lengkap: Membaca & Memahami Kode React	149
13.7 Latihan	152
13.8 Troubleshooting & Catatan	154
Bab 14 Workshop Proyek Akhir — Riset UI/UX & Perencanaan Desain	156
14.1 Persiapan Kelompok	156
14.2 Identifikasi Website Target	157
14.3 Heuristic Evaluation	158
14.4 Benchmarking Singkat	159
14.5 Perencanaan Desain di Figma	160
14.6 Commit dan Push	162
14.7 Output yang Diharapkan	162
14.8 Troubleshooting	163
14.9 Tautan Referensi	163
14.10 Persiapan untuk Minggu 15	163
14.11 Latihan Mandiri	163
Bab 15 Workshop Proyek Akhir — Implementasi, Deployment & Progress Clinic	164
15.1 Peralatan yang Dibutuhkan	164
15.2 Synchronisasi Repository	164
15.3 Implementasi Halaman Utama	165
15.4 Sesi Clinic / Mentoring	169
15.5 Deployment ke GitHub Pages	170
15.6 Troubleshooting Deployment	172
15.7 Troubleshooting Umum	172
15.8 Persiapan Presentasi	172
15.9 Final Commit dan Documentation	173

15.10Panduan Agentic Coding Tools (Opsional)	174
15.11Output yang Diharapkan	174
15.12Tautan Referensi	175
15.13Persiapan untuk Minggu 16 (Presentasi)	175
15.14Latihan Mandiri	175
Bab 16 Presentasi, Refleksi & Finalisasi Proyek Akhir	176
16.1 Peralatan yang Dibutuhkan	176
16.2 Persiapan Sebelum Presentasi	176
16.3 Presentasi Kelompok	177
16.4 Tanya-Jawab Individual	180
16.5 Refleksi Individu	181
16.6 Finalisasi Repository	182
16.7 Struktur Repository Final	183
16.8 Deliverable Akhir Proyek	184
16.9 Troubleshooting	184
16.10Tautan Referensi	185
16.11Latihan Mandiri	185

Daftar Gambar

1.1	VS Code dengan file <code>index.html</code> terbuka.	3
1.2	Peramban menampilkan hasil “Halo, Dunia!”	3
1.3	Dashboard GitHub setelah berhasil login.	4
1.4	Halaman repository GitHub <code>latihan-web-1</code>	5

Daftar Tabel

1.1	Tiga lapis utama halaman web	1
1.2	Ekstensi VS Code yang diperlukan	2
1.3	Masalah umum dan solusinya	7
2.1	Shortcut alat dasar Figma	9
2.2	Tingkatan detail desain	10
2.3	Tiga heuristik utama	11
2.4	Jenis hierarki visual	12
2.5	Aspek aksesibilitas untuk pemula	12
2.6	Template evaluasi heuristik	13
2.7	Masalah umum Figma dan solusinya	14
3.1	Penjelasan struktur dasar HTML	16
3.2	Perbandingan tag semantic dan non-semantic	18
3.3	Masalah umum HTML dan solusinya	26
4.1	Jenis input form HTML	31
4.2	Atribut validasi form	32
4.3	Atribut tabel untuk aksesibilitas	36
4.4	Atribut elemen video	38
4.5	Atribut elemen audio	38
4.6	Masalah umum form, tabel, dan multimedia	46
5.1	Masalah umum dan solusinya	56
6.1	Properti utama <i>flexbox</i>	58
6.2	Masalah umum <i>flexbox</i> dan solusinya	68
7.1	Breakpoint umum untuk <i>responsive design</i>	70
7.2	Masalah umum CSS <i>grid</i> dan <i>responsive design</i>	82
8.1	Masalah umum Git dan solusinya	92
9.1	Masalah umum <i>branch</i> , PR, dan GitHub Pages	102
10.1	Perbandingan kata kunci deklarasi variabel	104
10.2	Metode DOM Selection	110
11.1	Perbandingan onclick dan addEventListener	116
11.2	Event yang sering digunakan	116
11.3	Ringkasan cara mengubah DOM	120
12.1	Kelebihan Tailwind CSS	129
12.2	Pemetaan properti CSS ke class Tailwind	136

13.1 Alasan popularitas React	142
13.2 Perbedaan sintaks JSX dan HTML	145
13.3 Istilah penting dalam React	155
13.4 Topik dan keterampilan yang diperoleh dari Minggu 10–13	155
14.1 10 Usability Heuristics — Nielsen Norman Group	158
14.2 Masalah Umum dan Solusi — Minggu 14	163
15.1 Masalah Umum Deployment — GitHub Pages	172
15.2 Masalah Umum Implementasi dan Solusi	172
16.1 Contoh Pertanyaan Tanya-Jawab Individual	180
16.2 Masalah Umum dan Solusi — Minggu 16	184

Bab 1 Pengantar Web & Setup Lingkungan Kerja

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- menjelaskan alur kerja *design-to-code* dalam pengembangan web;
- menginstal dan mengonfigurasi VS Code beserta ekstensi yang diperlukan;
- membuat akun GitHub dan melakukan *push* pertama ke *repository*;
- membuka file HTML di peramban menggunakan *Live Server*.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- menginstal VS Code dan ekstensi pendukung;
- membuat file HTML pertama dan melihat hasilnya di peramban;
- membuat akun GitHub serta *repository* pertama;
- melakukan *push* kode ke GitHub.

1.1 Pengenalan Web

Website adalah kumpulan halaman yang diakses melalui peramban (*browser*) seperti Chrome, Firefox, atau Edge. Setiap halaman web terdiri dari tiga lapis utama:

Tabel 1.1: Tiga lapis utama halaman web

Lapis	Bahasa	Fungsi
Struktur	HTML	“Tulang” — menentukan elemen apa yang ada di halaman
Tampilan	CSS	“Kulit” — menentukan warna, ukuran, posisi, font
Interaksi	JavaScript	“Otot” — menentukan apa yang terjadi saat diklik/di-scroll

Catatan

Mahasiswa tidak perlu menghafal ketiga lapis ini sekarang. Cukup ketahui bahwa ada tiga lapis; di minggu-minggu berikutnya kita akan pelajari satu per satu.

1.2 Alur Kerja *Design-to-Code*

Alur kerja pengembangan web mengikuti urutan berikut:

1. **Ide / Kebutuhan** — menentukan tujuan dan isi halaman.
2. **Figma** — mendesain tampilan visual.
3. **VS Code** — menulis kode HTML, CSS, dan JavaScript.
4. **Peramban** — melihat dan menguji hasil.

5. **GitHub** — menyimpan kode agar bisa diakses kapan saja.

Intinya: desain di Figma → implementasi di VS Code → lihat hasil di peramban → simpan di GitHub.

1.3 Instalasi VS Code & Ekstensi

1.3.1 Download & Instal VS Code

Langkah Kerja

1. Buka peramban, kunjungi <https://code.visualstudio.com/>.
2. Klik tombol **Download for Windows** (atau sesuai OS).
3. Jalankan installer, klik **Next** di setiap langkah, pastikan opsi berikut diceklis:
 - Add “Open with Code” action to Windows Explorer file context menu
 - Add “Open with Code” action to Windows Explorer directory context menu
 - Register Code as an editor for supported file types
4. Klik **Install**, tunggu selesai, lalu klik **Finish**.

1.3.2 Instal Ekstensi Penting

Buka VS Code, tekan **Ctrl + Shift + X** untuk membuka panel *Extensions*. Cari dan instal ekstensi berikut:

Tabel 1.2: Ekstensi VS Code yang diperlukan

No.	Nama Ekstensi	Fungsi
1	Live Server (Ritwick Dey)	Preview halaman HTML otomatis di peramban saat file disimpan
2	HTML CSS Support	Autocomplete kelas CSS di file HTML
3	Prettier – Code formatter	Format kode otomatis agar rapi
4	VS Code Intellicode	Saran kode cerdas dari AI

1.3.3 Verifikasi Instalasi

Langkah Kerja

1. Buka VS Code.
2. Buat folder baru: **File** → **Open Folder** → **New Folder**, beri nama latihan-web-1.
3. Di dalam folder itu, buat file baru bernama `index.html`.
4. Ketik **!** lalu tekan **Tab** — VS Code akan mengisi template HTML secara otomatis (*snippets Emmet*).
5. Ubah isi `<title>` menjadi Halo Dunia.
6. Tambahkan di dalam `<body>`:

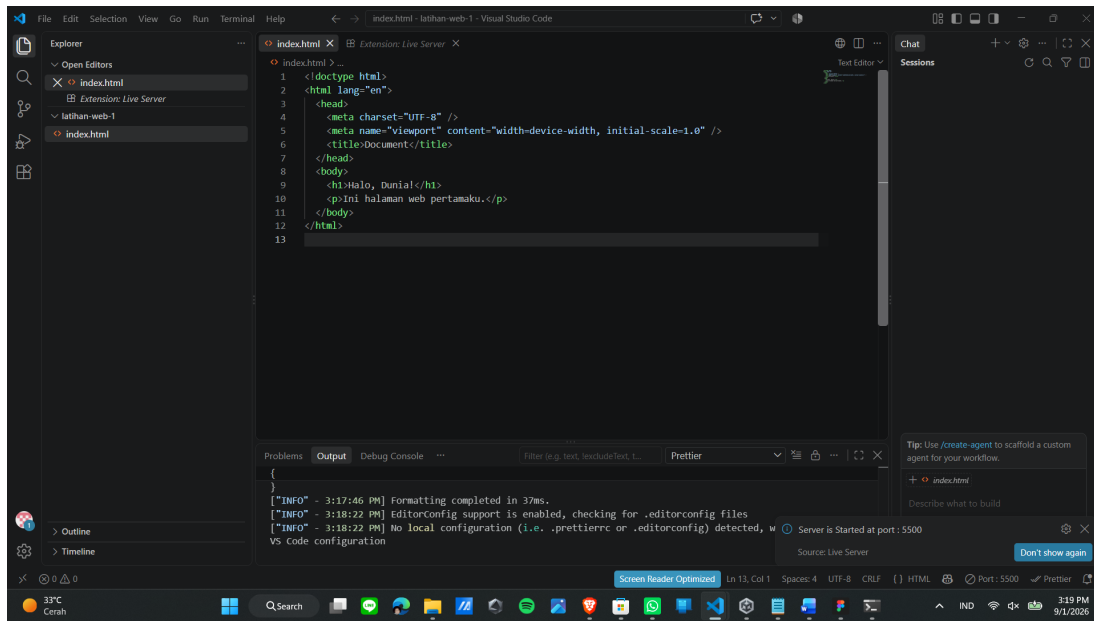
Listing 1.1: Kode “Halo Dunia”

```
<h1>Halo, Dunia!</h1>
<p>Ini halaman web pertamaku.</p>
```

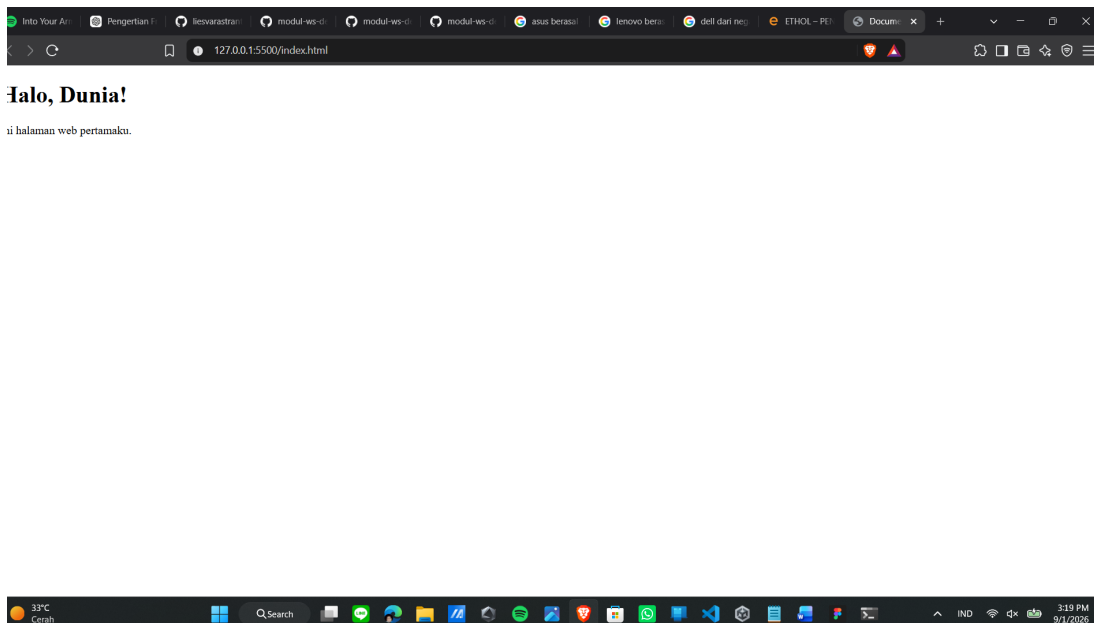
Langkah Kerja

1. Klik kanan di editor → **Open with Live Server**.

2. Peramban akan terbuka secara otomatis menampilkan teks “Halo, Dunia!”.



Gambar 1.1: VS Code dengan file index.html terbuka.



Gambar 1.2: Peramban menampilkan hasil “Halo, Dunia!”

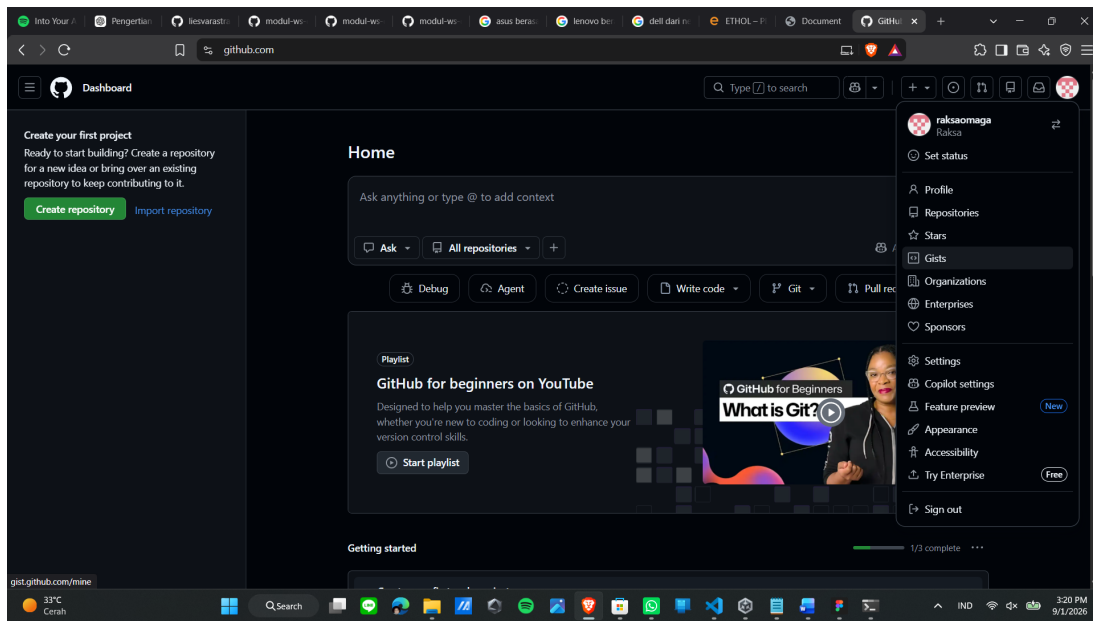
1.4 Akun GitHub & Repository Pertama

1.4.1 Buat Akun GitHub

Langkah Kerja

1. Buka <https://github.com>.
2. Klik Sign up.
3. Isi email, password, dan username (gunakan format: nama-nim, misal: budi-2301001).

4. Selesaikan verifikasi.



Gambar 1.3: Dashboard GitHub setelah berhasil login.

1.4.2 Buat Repository Baru

Langkah Kerja

1. Di dashboard GitHub, klik tombol + (pojok kanan atas) → **New repository**.
2. Isi form berikut:
 - **Repository name:** latihan-web-1
 - **Description:** Latihan pertama --- Halo Dunia
 - Pilih **Public**
 - Centang **Add a README file**
3. Klik **Create repository**.

1.4.3 Hubungkan Folder Lokal ke GitHub (*Push Pertama*)

Peringatan

Pastikan Git sudah terinstal di laptop. Jika belum, unduh dari <https://git-scm.com/download/win> dan instal dengan pengaturan default.

Buka **Terminal** di VS Code (**Ctrl + \$** atau menu **Terminal** → **New Terminal**), lalu ketik perintah berikut satu per satu:

Listing 1.2: Perintah push pertama ke GitHub

```
# 1. Inisialisasi Git di folder lokal
git init

# 2. Hubungkan ke remote repository
# (ganti NAMA_USER dengan username GitHub-mu)
```

```
git remote add origin https://github.com/NAMA_USER/latihan-web-1.git

# 3. Tambahkan semua file untuk di-commit
git add .

# 4. Buat commit pertama
git commit -m "Commit pertama: Halo Dunia"

# 5. Push ke GitHub
git branch -M main
git push -u origin main
```

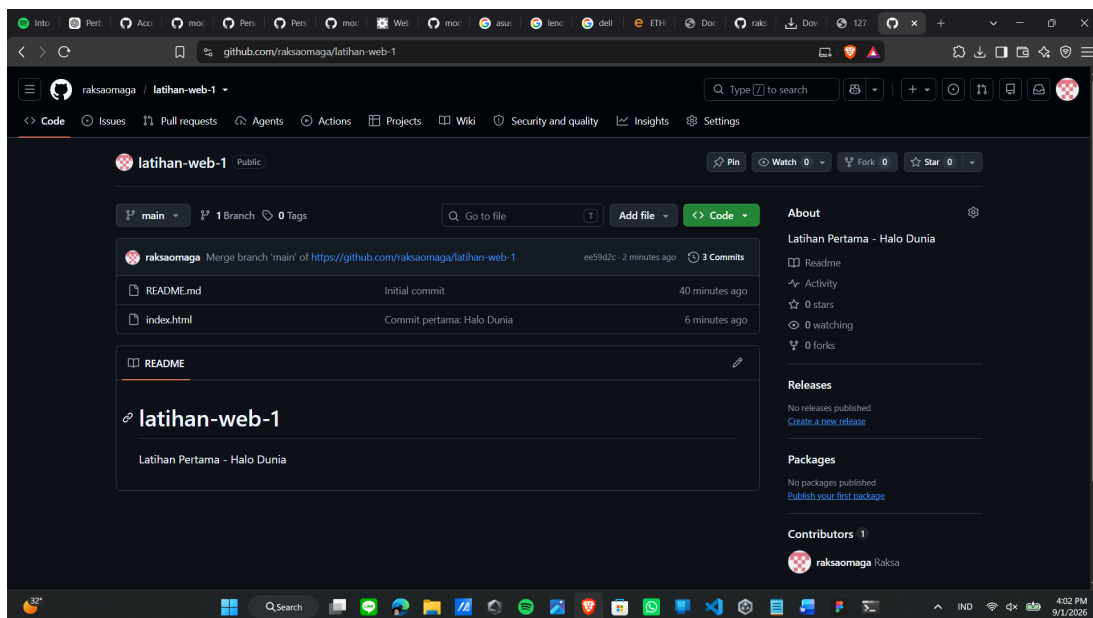
Catatan

Jika Git meminta *credential* (username & password), gunakan **Personal Access Token** sebagai password:

1. Buka <https://github.com/settings/tokens>.
2. Klik **Generate new token (classic)**.
3. Beri nama token, centang **repo**, lalu klik **Generate**.
4. Salin token dan gunakan sebagai password saat Git meminta.

1.4.4 Verifikasi

Buka https://github.com/NAMA_USER/latihan-web-1 di peramban. Pastikan file `index.html` terlihat di daftar file.



Gambar 1.4: Halaman repository GitHub latihan-web-1.

1.5 Latihan Mandiri

Latihan 1.1 Perbarui Halaman

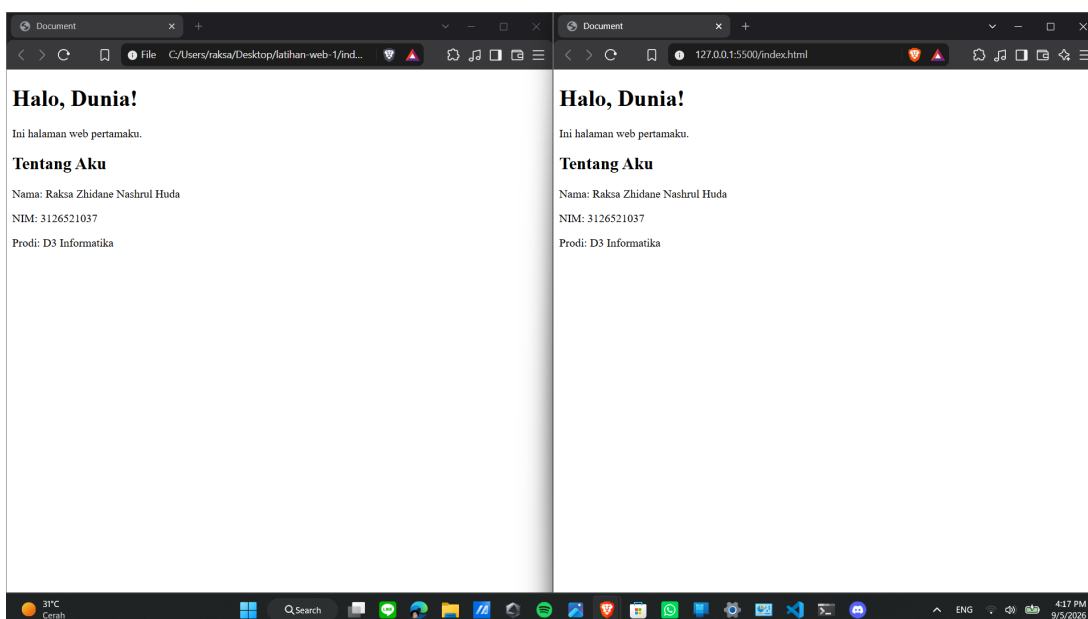
Buka file `index.html` di VS Code. Tambahkan elemen berikut di bawah `<p>` yang sudah ada:

```
<h2>Tentang Aku</h2>
<p>Nama: [nama lengkap kamu]</p>
<p>NIM: [NIM kamu]</p>
<p>Prodi: D3 Informatika</p>
```

Simpan file (**Ctrl + S**), periksa peramban, lalu *commit* dan *push* perubahan ke GitHub.

Latihan 1.2 Eksperimen *Live Server*

1. Matikan *Live Server* (klik **Port: 5500** → **Stop** di pojok kanan bawah VS Code).
2. Buka file `index.html` langsung dari File Explorer (klik ganda).
3. Perhatikan URL di peramban — apakah berbeda dari URL *Live Server*? (`file:///...` vs `127.0.0.1:5500`)
4. Ubah salah satu teks di VS Code → simpan → muat ulang peramban secara manual.
5. Aktifkan kembali *Live Server*. Ubah teks lagi → simpan → perhatikan apakah peramban memperbarui diri secara otomatis.



Perbandingan

URL: *Live Server* vs file langsung. Kiri: `127.0.0.1:5500/index.html`; kanan: `file:///C:/.../index.html`.

1.6 Troubleshooting

1.7 Rangkuman

- Alur kerja *design-to-code*: Figma → VS Code → Peramban → GitHub.

Tabel 1.3: Masalah umum dan solusinya

Masalah	Solusi
<i>Live Server</i> tidak muncul di menu kanan-klik	Pastikan ekstensi “Live Server” sudah terinstal dan VS Code di- <i>restart</i>
Peramban menampilkan <i>This site can't be reached</i>	Pastikan <i>Live Server</i> sudah berjalan (lihat <i>status bar</i> pojok kanan bawah VS Code)
Git menolak <i>push</i> karena “repository not found”	Periksa URL <i>remote</i> : <code>git remote -v</code> , lalu perbaiki dengan <code>git remote set-url origin URL_BARU</code>
Git meminta <i>password</i> tapi tidak bisa login	Gunakan Personal Access Token (bukan <i>password</i> GitHub biasa)
VS Code menampilkan error “command not found: git”	Instal Git dari https://git-scm.com/download/win , lalu <i>restart</i> VS Code
File <i>index.html</i> tidak muncul di GitHub	Pastikan sudah menjalankan <code>git add .</code> , <code>git commit</code> , dan <code>git push</code>

- VS Code + ekstensi (*Live Server*, Prettier, dll.) sudah terinstal.
- File HTML pertama berhasil dibuat dan dilihat di peramban.
- Akun GitHub dan *repository* pertama sudah dibuat.
- *Push* pertama ke GitHub berhasil dilakukan.
- *Live Server* memberikan *preview* otomatis tanpa *reload*.

Bab 2 Figma Dasar & Pengantar UI/UX

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- mengoperasikan antarmuka dasar Figma (frame, shape, text, layers);
- membuat wireframe sederhana untuk satu halaman web;
- menjelaskan tiga konsep dasar UI/UX: heuristik, hierarchy, dan aksesibilitas;
- menerapkan konsep UI/UX ke dalam desain wireframe.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- mengenal antarmuka dan alat dasar Figma;
- membuat wireframe halaman “Profil Diri” untuk perangkat *mobile*;
- mempelajari tiga konsep UI/UX dan menerapkannya pada wireframe.

2.1 Pengenalan Figma

2.1.1 Akses Figma

Langkah Kerja

1. Buka <https://www.figma.com> di peramban.
2. Klik **Log in**, masuk dengan akun yang sudah dibuat (atau daftar jika belum).
3. Kamu akan melihat halaman **Drafts** — ini adalah ruang kerja *default*.

Tips

Figma bisa digunakan langsung di peramban, tapi lebih nyaman menggunakan **Figma Desktop App** (unduh dari <https://www.figma.com/downloads/>).

2.1.2 Membuat File & Frame Pertama

Langkah Kerja

1. Klik **New design file** (tombol biru di pojok kanan atas).
2. Di *canvas* kosong, tekan **F** di keyboard → arahkan mouse ke *canvas* → *drag* untuk membuat sebuah **Frame**.
3. Di panel **Design** (kanan), atur ukuran frame: pilih preset **Web 1920** (1920 × 1080 px) atau **MacBook Air**.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-02/figma-frame-kosong.png`

Keterangan: Canvas Figma dengan satu frame kosong berukuran Web 1920.

Instruksi mahasiswa: Pastikan panel Design di kanan menunjukkan ukuran frame yang benar.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-02/figma-frame-kosong.png` tersedia.

2.1.3 Alat Dasar Figma

Pelajari toolbar di bagian atas *canvas*:

Tabel 2.1: Shortcut alat dasar Figma

Shortcut	Alat	Fungsi
V	Move	Memilih & memindahkan objek
F	Frame	Membuat <i>frame</i> (artboard)
R	Rectangle	Membuat kotak persegi
O	Ellipse	Membuat lingkaran/oval
T	Text	Membuat teks
L	Line	Membuat garis
P	Pen	Membuat bentuk bebas
Ctrl + Z	Undo	Membatalkan aksi terakhir

Latihan cepat: Di dalam *frame* yang sudah dibuat, coba:

1. Tekan **R** → *drag* di dalam *frame* untuk membuat kotak.
2. Tekan **T** → klik di dalam *frame* → ketik “Hello Figma”.
3. Tekan **V** → klik objek untuk memilih → *drag* untuk memindahkan.

2.1.4 Panel Layers & Properties

Di panel kiri terdapat **Layers** — setiap objek yang dibuat akan muncul sebagai *layer*.

- **Klik ganda** *layer* untuk mengganti nama (misal: “Rectangle 1” menjadi “Header Background”).
- **Urutan layer** = urutan di atas *canvas* (layer di atas menimpa layer di bawah).

Di panel kanan terdapat **Design Properties** — ubah:

- **Fill:** warna isi objek
- **Stroke:** garis tepi
- **Corner Radius:** sudut membulat
- **Font & Size:** untuk objek teks

Tips

Ubah warna kotak yang sudah dibuat menjadi biru, dan ubah teks “Hello Figma” menjadi ukuran 24 px untuk berlatih mengatur *properties*.

2.2 Membuat Wireframe Halaman Web

Catatan

Wireframe adalah sketsa rendah detail yang menunjukkan struktur dan penempatan elemen halaman. Bukan desain final.

Tabel 2.2: Tingkatan detail desain

Tingkat Detail	Nama	Fungsi
Rendah	Wireframe	Menunjukkan layout & struktur (tanpa warna/font detail)
Sedang	Mockup	Menambahkan warna, font, gambar (visual lebih realistis)
Tinggi	Prototype	Interaktif — bisa diklik dan dinavigasi

Kita mulai dari **wireframe** (tingkat rendah) karena fokusnya adalah struktur, bukan tampilan cantik.

2.2.1 Langkah 1: Buat Frame untuk *Mobile*

Langkah Kerja

1. Tekan **F** → pilih preset **iPhone 14** (390 × 844 px) di panel kanan.
2. *Rename layer frame* ini menjadi **Wireframe - Profil Diri (Mobile)**.

2.2.2 Langkah 2: Header / Navigasi

Langkah Kerja

1. Tekan **R** → buat *rectangle* di bagian atas *frame* (lebar penuh 390 px, tinggi ~60 px). *Rename layer* jadi **Header**. Warna: abu-abu terang (#E0E0E0).
2. Tekan **T** → klik di atas *rectangle* header → ketik Nama Kamu. Posisikan di kiri *header*. Font: regular, 16 px.

2.2.3 Langkah 3: Hero / Foto Profil

Langkah Kerja

1. Tekan **O** → buat lingkaran di bawah *header* (120 × 120 px, posisi center horizontal). Warna: abu-abu (#CCCCCC). *Rename layer*: **Foto Profil**.
2. Tekan **T** → di bawah lingkaran, ketik nama lengkap dan keterangan program studi. Font: **bold** untuk nama, regular untuk keterangan.

2.2.4 Langkah 4: Bagian “Tentang Saya”

Langkah Kerja

1. Tekan **T** → ketik **Tentang Saya** sebagai judul *section*. Font: bold, 18 px.
2. Tekan **R** → buat 2–3 *rectangle* kecil di bawah judul sebagai *placeholder* paragraf (lebar ~350 px, tinggi ~16 px × 3 baris). Warna: abu-abu sangat terang (#F0F0F0).

2.2.5 Langkah 5: Bagian “Keahlian”

Langkah Kerja

1. Ketik judul Keahlian.
2. Buat 3–4 *rectangle* kecil sebagai *placeholder* “skill card” (ukuran $\sim 160 \times 80$ px, sejajar horizontal atau *grid* 2×2). Di dalam masing-masing, tambahkan teks Skill 1, Skill 2, dst.

2.2.6 Langkah 6: Footer

Langkah Kerja

1. Tekan **R** → buat *rectangle* di bagian bawah *frame* (lebar penuh, tinggi ~ 50 px). Warna: abu-abu gelap (#999999).
2. Tambahkan teks: © 2026 Nama Kamu.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-02/wireframe-profil-diri.png`

Keterangan: Wireframe halaman “Profil Diri” di Figma.

Instruksi mahasiswa: Pastikan terlihat section: Header, Foto Profil, Tentang Saya, Keahlian, dan Footer dengan rapi.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-02/wireframe-profil-diri.png` tersedia.

2.3 Tiga Konsep Dasar UI/UX

2.3.1 Heuristik (Evaluasi Kegunaan)

Heuristik adalah aturan umum desain yang membantu agar antarmuka pengguna (*UI*) mudah digunakan. Berikut tiga heuristik paling relevan untuk pemula:

Tabel 2.3: Tiga heuristik utama

Heuristik	Penjelasan	Contoh Penerapan di Wireframe
Visibility of System Status	Pengguna harus tahu apa yang sedang terjadi	Tambahkan indikator <i>loading</i> saat halaman memuat
Match Between System and Real World	Gunakan bahasa yang dipahami pengguna	Judul “Tentang Saya” lebih jelas daripada “Bio Data Seksi 4”
Error Prevention	Cegah kesalahan pengguna sebelum terjadi	Form input harus punya <i>placeholder</i> /label yang jelas

Catatan

Ada 10 heuristik Nielsen Norman Group secara lengkap. Tiga di atas adalah yang paling sering ditemui dan paling mudah diterapkan untuk pemula.

2.3.2 Hierarchy (Hierarki Visual)

Hierarchy adalah pengaturan elemen agar mata pembaca tahu mana yang dilihat duluan.

Tabel 2.4: Jenis hierarki visual

Jenis	Fungsi	Contoh
Hierarki ukuran	Elemen lebih besar = lebih penting	Nama besar di atas, deskripsi kecil di bawah
Hierarki warna	Warna kontras menarik perhatian	Tombol “Hubungi Saya” berwarna biru di antara elemen abu-abu

Latihan di Figma: Pastikan di wireframe yang sudah dibuat:

1. Nama di bagian Hero **lebih besar** dari teks deskripsi.
2. Judul section (Tentang Saya, Keahlian) **lebih besar** dari konten di bawahnya.
3. Jika belum, ubah ukuran teks di Figma untuk mencerminkan hierarchy.

2.3.3 Aksesibilitas (Accessibility)

Aksesibilitas adalah desain yang bisa digunakan oleh semua orang, termasuk yang memiliki keterbatasan.

Tabel 2.5: Aspek aksesibilitas untuk pemula

Aspek	Yang Harus Diperhatikan
Kontras warna	Teks harus terbaca dengan latar belakang. Rasio kontras minimal 4.5:1 untuk teks normal.
Ukuran teks	Minimal 16 px untuk teks <i>body</i> . Jangan gunakan ukuran yang terlalu kecil.
Struktur heading	Gunakan heading secara berurutan (<h1> → <h2> → <h3>), jangan loncat-loncat.

Tips

Gunakan **WebAIM Contrast Checker** (<https://webaim.org/resources/contrastchecker/>) untuk mengecek kontras warna.

Latihan di Figma:

1. Di wireframe, pastikan warna teks di atas *background* cukup kontras.
2. Jika menggunakan warna abu-abu di atas putih, pastikan tidak terlalu tipis/pucat.

2.4 Latihan Mandiri

Latihan 2.1 Wireframe Halaman “Portofolio” (Desktop)

Buat wireframe baru untuk halaman portofolio di Figma dengan spesifikasi:

- **Frame:** Web 1920 (1920 × 1080 px).
- **Section yang wajib ada:**
 1. **Navbar** — logo di kiri, 3–4 menu di kanan (Home, Portofolio, About, Contact).
 2. **Hero** — judul besar + tombol “Lihat Proyek”.
 3. **Grid Proyek** — minimal 6 kotak (*card*) yang mewakili proyek.
 4. **Tentang Saya** — foto *placeholder* + teks deskripsi.
 5. **Footer** — hak cipta & ikon media sosial *placeholder*.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-02/wireframe-portofolio.png`

Keterangan: Wireframe halaman Portfolio di Figma (tampilan penuh).

Instruksi mahasiswa: Pastikan terlihat Navbar, Hero, Grid Proyek, Tentang Saya, dan Footer.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-02/wireframe-portofolio.png` tersedia.

Latihan 2.2 Evaluasi Heuristik pada Website Nyata

1. Buka salah satu website berikut: `google.com`, `tokopedia.com`, atau `instagram.com` (halaman login).
2. Evaluasi website tersebut menggunakan tiga heuristik yang sudah dipelajari dan catat hasilnya:

Tabel 2.6: Template evaluasi heuristik

Heuristik	Ya/Tidak	Bukti / Contoh
Visibility of System Status		
Match Between System and Real World		
Error Prevention		

Latihan 2.3 Cek Kontras Aksesibilitas

1. Pilih 2 kombinasi warna dari wireframe yang kamu buat.
2. Buka <https://webaim.org/resources/contrastchecker/>.
3. Masukkan warna teks dan *background*, cek rasio kontras.
4. Pastikan minimal satu kombinasi memenuhi **WCAG AA** (4.5:1 untuk teks normal).

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-02/webaim-contrast-checker.png`

Keterangan: WebAIM Contrast Checker dengan hasil pengecekan.

Instruksi mahasiswa: Pastikan terlihat rasio kontras dan status “Pass” atau “Fail”.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-02/webaim-contrast-checker.png` tersedia.

2.5 Troubleshooting

Tabel 2.7: Masalah umum Figma dan solusinya

Masalah	Solusi
Figma tidak bisa dibuka di peramban	Gunakan Figma Desktop App, atau coba peramban Chrome/Firefox versi terbaru
Frame tidak bisa dibuat	Tekan shortcut F (huruf F, bukan <i>function key</i>) untuk aktifkan alat Frame
Objek tidak bisa dipindahkan	Pastikan alat Move (V) aktif, bukan alat lain
Layer tidak muncul di panel	<i>Scroll</i> panel Layers di sebelah kiri <i>canvas</i> — mungkin tersembunyi di bawah
Perubahan tidak tersimpan	Figma <i>auto-save</i> , tapi pastikan koneksi internet stabil. Cek ikon <i>cloud</i> di pojok kanan atas
Tidak bisa membuat duplikat	Pilih objek → tekan Ctrl + D (Windows) atau Cmd + D (Mac)

2.6 Rangkuman

- Mengoperasikan Figma: membuat *frame*, *shapes*, teks, mengatur *properties*.
- Membuat wireframe sederhana untuk halaman web (5 section: Header, Foto, Tentang Saya, Keahlian, Footer).
- Tiga konsep dasar UI/UX: Heuristik, Hierarchy, Aksesibilitas.
- Menerapkan *hierarchy* dan aksesibilitas di wireframe.
- Mengecek kontras warna dengan WebAIM Contrast Checker.

Bab 3 HTML Dasar: Struktur & Semantic Tag

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- menjelaskan struktur dasar dokumen HTML (*doctype*, *html*, *head*, *body*);
- menggunakan elemen HTML dasar: *heading*, *paragraf*, *tautan*, *gambar*, dan *daftar*;
- membedakan dan menggunakan *semantic tag* (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`, dsb.);
- menulis HTML yang valid menggunakan W3C Validator;
- menghubungkan file HTML ke file CSS eksternal secara dasar.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- membuat dokumen HTML dengan struktur yang benar;
- berlatih menggunakan elemen dasar HTML;
- membangun halaman profil dengan *semantic tag*;
- menghubungkan file HTML ke file CSS;
- memvalidasi kode HTML melalui W3C Validator.

3.1 Struktur Dokumen HTML

3.1.1 Proyek Baru

Langkah Kerja

1. Di VS Code, buat folder baru: **File** → **Open Folder** → **New Folder**, beri nama latihan-web-3.
2. Buat file baru: `index.html`.

3.1.2 Struktur Dasar

Ketik kode berikut di `index.html`:

Listing 3.1: Struktur dasar dokumen HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Latihan HTML --- Minggu 3</title>
</head>
<body>
  <h1>Halo dari HTML!</h1>
</body>
</html>
```

Penjelasan tiap baris:

Tabel 3.1: Penjelasan struktur dasar HTML

Baris	Fungsi
<!DOCTYPE html>	Memberitahu peramban bahwa ini dokumen HTML5
<html lang="id">	Elemen root; lang="id" menandakan bahasa Indonesia
<head>	Metadata — tidak terlihat di halaman, tapi penting untuk peramban & SEO
<meta charset>	Set karakter <i>encoding</i> (agar huruf seperti é, ü muncul benar)
<meta name="viewport">	Agar halaman responsif di <i>mobile</i>
<title>	Judul yang muncul di tab peramban
<body>	Konten yang terlihat oleh pengguna

3.2 Elemen-Elemen Dasar HTML

Langkah Kerja

1. Tambahkan kode berikut di dalam <body>, di bawah <h1>.

Listing 3.2: Kumpulan elemen HTML dasar

```
<!-- Heading: h1 sampai h6 -->
<h1>Ini Heading 1 (Judul Utama)</h1>
<h2>Ini Heading 2 (Judul Section)</h2>
<h3>Ini Heading 3 (Sub-judul)</h3>

<!-- Paragraph -->
<p>Ini adalah paragraf biasa. HTML menggunakan tag
    untuk mendefinisikan konten.</p>
<p>Paragraf kedua. Setiap tag &lt;p> akan membuat
    baris baru.</p>

<!-- Link (Anchor) -->
<p>Kunjungi <a href="https://developer.mozilla.org"
    target="_blank">MDN Web Docs</a> untuk belajar
    HTML lebih lanjut.</p>

<!-- Image (gambar dari internet) -->


<!-- List tidak berurut (Unordered List) -->
<h3>Tools yang Kita Gunakan:</h3>
<ul>
```

```

<li>VS Code --- editor kode</li>
<li>Live Server --- preview otomatis</li>
<li>Browser --- melihat hasil</li>
<li>GitHub --- menyimpan kode</li>
</ul>

<!-- List berurut (Ordered List) -->
<h3>Langkah Instalasi:</h3>
<ol>
  <li>Download VS Code dari website resmi</li>
  <li>Install ekstensi Live Server</li>
  <li>Buat file index.html</li>
  <li>Klik kanan &rarr; Open with Live Server</li>
</ol>

<!-- Horizontal Rule -->
<hr>

<!-- Strong & Emphasis -->
<p>HTML itu <strong>sangat penting</strong> untuk
  dipelajari karena fondasi semua website.</p>
<p>Kamu pasti <em>bisa</em> menguasai ini!</p>

```

Simpan file (**Ctrl + S**) → buka dengan *Live Server*.

Tips

Periksa di peramban: apakah terlihat 3 *heading* (h1, h2, h3) dengan ukuran berbeda, 2 paragraf, 1 tautan, 1 gambar *placeholder*, 1 daftar *bullet*, 1 daftar bernomor, 1 garis horizontal, 1 teks tebal, dan 1 teks miring?

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-03/browser-elemen-dasar.png

Keterangan: Peramban yang menampilkan hasil elemen HTML dasar.

Instruksi mahasiswa: Pastikan terlihat heading, paragraph, link, gambar, lists, strong, em.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan \includegraphics setelah aset
assets/images/minggu-03/browser-elemen-dasar.png tersedia.

3.3 Semantic HTML

Catatan

Semantic HTML adalah penggunaan tag yang tidak hanya mendeskripsikan tampilan, tapi juga **makna** konten. Ini penting untuk SEO, aksesibilitas, dan keterbacaan kode.

Perbedaan *semantic* vs *non-semantic*:

Tabel 3.2: Perbandingan tag semantic dan non-semantic

Non-Semantic	Semantic	Kapan Menggunakan
<div> untuk semua <i>section</i>	<header> untuk bagian atas	Gunakan tag semantik saat konten punya peran jelas
<div> untuk daftar menu	<nav> untuk navigasi	<nav> memberitahu peramban ini bagian navigasi
<div> untuk konten utama	<main> untuk konten utama	Hanya ada satu <main> per halaman
<div> untuk footer	<footer> untuk bagian bawah	Setiap halaman sebaiknya punya <footer>
<div> untuk <i>section</i>	<section> atau <article>	<section> = bagian tematik; <article> = konten mandiri

3.3.1 Rekonstruksi dengan Semantic Tag

Buat file baru bernama `profil.html` di folder yang sama. Ketik kode berikut:

Listing 3.3: Halaman profil dengan semantic HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Profil Diri --- Latihan Semantic HTML</title>
</head>
<body>

  <!-- HEADER: Berisi navigasi -->
  <header>
    <h1>Portfolio Saya</h1>
    <nav>
      <ul>
        <li><a href="#tentang">Tentang</a></li>
        <li><a href="#keahlian">Keahlian</a></li>
        <li><a href="#proyek">Proyek</a></li>
        <li><a href="#kontak">Kontak</a></li>
      </ul>
    </nav>
  </header>
```

```

<!-- MAIN: Konten utama halaman -->
<main>

  <!-- SECTION: Tentang Saya -->
  <section id="tentang">
    <h2>Tentang Saya</h2>
    
    <p>Halo! Nama saya [Nama Lengkap], mahasiswa
      semester 1 D3 Informatika di [Nama Kampus].
      Saya tertarik dengan dunia desain web dan
      ingin belajar membangun website yang menarik
      dan mudah digunakan.</p>
  </section>

  <!-- SECTION: Keahlian -->
  <section id="keahlian">
    <h2>Keahlian</h2>
    <ul>
      <li>HTML & dasar CSS</li>
      <li>Figma (wireframing)</li>
      <li>Microsoft Office</li>
      <li>Dasar pemrograman (belajar)</li>
    </ul>
  </section>

  <!-- SECTION: Proyek -->
  <section id="proyek">
    <h2>Proyek</h2>

    <!-- ARTICLE: Konten mandiri tentang 1 proyek -->
    <article>
      <h3>Website Profil Diri</h3>
      <p>Website statis pertama saya yang dibuat
        dengan HTML dan CSS. Berisi informasi
        profil diri, daftar keahlian, dan cara
        menghubungi saya.</p>
      <p><strong>Teknologi:</strong> HTML, CSS</p>
    </article>

    <article>
      <h3>Landing Page Produk</h3>
      <p>Halaman promosi produk sederhana yang
        dibuat sebagai latihan layout dengan
        Flexbox.</p>
      <p><strong>Teknologi:</strong> HTML, CSS</p>
    </article>
  </section>

```

```

        </article>
    </section>

    <!-- SECTION: Kontak -->
    <section id="kontak">
        <h2>Kontak</h2>
        <p>Email: <a href="mailto:nama@email.com">
            nama@email.com</a></p>
        <p>GitHub: <a href="https://github.com/username"
            target="_blank">github.com/username</a></p>
    </section>

</main>

<!-- FOOTER -->
<footer>
    <p>&copy; 2026 [Nama Lengkap]. Dibuat untuk
        mata kuliah Desain Web.</p>
</footer>

</body>
</html>

```

Simpan file → buka dengan *Live Server*.

3.3.2 Perbandingan dengan Non-Semantic

Listing 3.4: Perbandingan non-semantic vs semantic

```

<!-- Non-Semantic: semua pakai div -->
<div class="header">
    <div class="nav">
        <div class="menu-item">Tentang</div>
    </div>
</div>
<div class="main">
    <div class="section">...</div>
</div>
<div class="footer">...</div>

<!-- Semantic: menggunakan tag yang tepat -->
<header>
    <nav>
        <a href="#tentang">Tentang</a>
    </nav>
</header>
<main>
    <section>...</section>
</main>

```

```
<footer>...</footer>
```

Catatan

Kenapa *semantic* lebih baik?

1. **Screen reader** (untuk tunanetra) bisa membaca struktur halaman dengan benar.
2. **Search engine** (Google) lebih mudah memahami konten halaman.
3. **Developer lain** (dan kamu 3 bulan lagi) lebih mudah membaca kode.

3.4 Elemen Pendukung Penting

3.4.1 Anchor (Tautan) — Detail Atribut

Listing 3.5: Berbagai jenis tautan

```
<!-- Link ke halaman lain -->
<a href="https://www.google.com">Google</a>

<!-- Link ke email -->
<a href="mailto:budi@email.com">Kirim Email</a>

<!-- Link ke section di halaman yang sama (anchor link) -->
<a href="#keahlian">Lihat Keahlian</a>

<!-- Buka di tab baru -->
<a href="https://www.google.com"
  target="_blank" rel="noopener_noreferrer">
  Google (tab baru)
</a>

<!-- Gambar sebagai link -->
<a href="https://www.google.com">
  
</a>
```

3.4.2 Image — Detail Atribut

Listing 3.6: Berbagai cara menampilkan gambar

```
<!-- Gambar dasar -->


<!-- Gambar dari URL -->


<!-- Gambar dengan figure & caption -->
```

```
<figure>
  
  <figcaption>Foto profil saya --- diambil tahun 2026
</figcaption>
</figure>
```

Peringatan

Aturan penting untuk alt pada gambar:

- **Selalu** isi atribut alt.
- alt harus mendeskripsikan gambar, bukan hanya “gambar” atau “foto”.
- Jika gambar murni dekoratif, gunakan alt="" (kosong).

3.4.3 Tabel Dasar (Preview)

Listing 3.7: Struktur tabel HTML sederhana

```
<table>
  <caption>Jadwal Kuliah Semester 1</caption>
  <thead>
    <tr>
      <th>Hari</th>
      <th>Mata Kuliah</th>
      <th>Jam</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Senin</td>
      <td>Desain Web</td>
      <td>08:00 &ndash; 10:00</td>
    </tr>
    <tr>
      <td>Rabu</td>
      <td>Algoritma</td>
      <td>10:00 &ndash; 12:00</td>
    </tr>
  </tbody>
</table>
```

Catatan

Tabel akan dibahas lebih dalam pada Bab 4.

3.5 Menghubungkan HTML ke CSS (Dasar)

Catatan

CSS detail akan dipelajari di minggu-minggu berikutnya. Ini hanya pengenalan cara menghubungkan file.

3.5.1 Buat File CSS

Di folder yang sama, buat file baru: `style.css`. Ketik kode berikut:

Listing 3.8: File CSS dasar untuk halaman profil

```
/* Reset dasar */
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    line-height: 1.6;
    color: #333;
}

/* Header */
header {
    background-color: #2c3e50;
    color: white;
    padding: 20px;
    text-align: center;
}

header nav ul {
    list-style: none;
    padding: 0;
    display: flex;
    justify-content: center;
    gap: 20px;
}

header nav a {
    color: white;
    text-decoration: none;
}

/* Main content */
main {
    max-width: 800px;
    margin: 0 auto;
    padding: 20px;
}

section {
```

```

    margin-bottom: 40px;
}

/* Footer */
footer {
    background-color: #2c3e50;
    color: white;
    text-align: center;
    padding: 10px;
}

```

3.5.2 Hubungkan ke HTML

Di file `profil.html`, tambahkan tag `<link>` di dalam `<head>`:

Listing 3.9: Menambahkan tautan CSS di HTML

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Profil Diri --- Latihan Semantic HTML</title>
  <link rel="stylesheet" href="style.css">
</head>

```

Simpan kedua file → buka `profil.html` via *Live Server*.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-03/profil-dengan-css.png`

Keterangan: Halaman `profil.html` di peramban dengan CSS.

Instruksi mahasiswa: Pastikan terlihat *header* berwarna gelap, layout terpusat, dan *footer* berwarna gelap.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-03/profil-dengan-css.png` tersedia.

3.6 Validasi HTML

Langkah Kerja

1. Buka https://validator.w3.org/#validate_by_input.
2. Pilih tab **Direct Input**.
3. Salin seluruh isi `profil.html` → tempel ke kolom teks.
4. Klik **Check**.
5. Pastikan hasil: “Document checking completed. No errors or warnings to show.”

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-03/w3c-validasi.png`

Keterangan: W3C HTML Validator yang menunjukkan hasil validasi tanpa error.

Instruksi mahasiswa: Pastikan terlihat pesan “No errors or warnings to show”.

[**Tempatkan tangkapan layar di sini**]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-03/w3c-validasi.png` tersedia.

Jika ada error, perbaiki berdasarkan pesan yang diberikan validator, lalu cek ulang.

3.7 Latihan Mandiri

Latihan 3.1 Halaman “Tips Belajar HTML”

Buat file baru bernama `tips.html` dengan struktur *semantic* HTML yang lengkap. Halaman ini berisi:

1. **Header** dengan judul “Tips Belajar HTML untuk Pemula” dan `<nav>` (minimal 3 *anchor link* ke *section* di bawah).
2. **Section “Mengapa HTML Penting”** — 2 paragraf penjelasan.
3. **Section “Tips Pertama”** — *ordered list* berisi minimal 5 tips.
4. **Section “Kesalahan Umum”** — *unordered list* berisi minimal 4 kesalahan, masing-masing dijelaskan dalam paragraf terpisah.
5. **Section “Sumber Belajar”** — *unordered list* berisi minimal 3 tautan ke website belajar HTML (MDN, W3Schools, dsb.). Setiap tautan harus menggunakan `target="_blank"` dan `rel="noopener noreferrer"`.
6. **Footer** dengan hak cipta tahun 2026.

Persyaratan teknis: gunakan hanya tag *semantic*; tidak boleh ada `<div>` sebagai *section placeholder*; semua gambar harus punya `alt` yang deskriptif; file harus valid di W3C Validator.

Latihan 3.2 Push ke GitHub

1. Inisialisasi Git di folder `latihan-web-3` dan buat *repository* baru di GitHub (public, dengan README).
2. *Push* kode:

Listing 3.10: Perintah push ke GitHub

```
git init
git remote add origin https://github.com/NAMA_USER/latihan-web-3.git
git add .
git commit -m "Latihan HTML minggu 3: semantic tags"
git branch -M main
git push -u origin main
```

Tips

Pastikan file `index.html`, `profil.html`, `tips.html`, dan `style.css` terlihat di *repository* GitHub.

3.8 Troubleshooting

Tabel 3.3: Masalah umum HTML dan solusinya

Masalah	Solusi
Gambar tidak muncul di peramban	Periksa path <code>src</code> — gunakan URL lengkap untuk gambar <i>online</i> , atau pastikan nama file benar untuk gambar lokal
Tautan tidak berfungsi (<i>anchor</i>)	Pastikan <code>href="#section-id"</code> cocok dengan <code>id="section-id"</code> pada <i>section</i> target
W3C Validator menampilkan error	Baca pesan error — biasanya tag tidak ditutup (<code></p></code>) atau <i>nesting</i> salah
CSS tidak terlihat	Pastikan <code><link rel="stylesheet" href="style.css"></code> ada di <code><head></code> , dan nama file CSS benar
Halaman tidak responsif di <i>mobile</i>	Tag <code><meta name="viewport" ...></code> harus ada di <code><head></code>
Teks berantakan / paragraf tidak terpisah	Pastikan setiap paragraf dibungkus tag <code><p></code> , bukan hanya enter di kode

3.9 Rangkuman

- Struktur dasar dokumen HTML: `<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`.
- Elemen dasar: *heading*, paragraf, tautan, gambar, daftar, `<strong>`, `<em>`, `<hr>`.
- *Semantic tags*: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`.
- Perbedaan *semantic* vs *non-semantic* HTML.
- Menghubungkan HTML ke file CSS eksternal.
- Validasi HTML menggunakan W3C Validator.

Tips

Cheat Sheet Tag Semantic:

- `<header>` → Bagian atas halaman (judul, *nav*)
- `<nav>` → Navigasi / menu
- `<main>` → Konten utama (hanya 1 per halaman)
- `<section>` → Bagian tematik dalam `<main>`
- `<article>` → Konten mandiri (bisa dipindah ke halaman lain)
- `<aside>` → Konten sampingan (*sidebar*)
- `<footer>` → Bagian bawah halaman
- `<figure>` → Gambar + *caption*
- `<figcaption>` → *Caption* untuk `<figure>`

Bab 4 HTML Lanjutan: Form, Tabel & Multimedia

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- membuat form HTML dengan berbagai jenis *input* (*text*, *email*, *password*, *checkbox*, *radio*, *select*, *textarea*);
- menggunakan `<label>`, `<fieldset>`, dan validasi form dasar;
- membuat tabel HTML yang terstruktur dan aksesibel (`<thead>`, `<tbody>`, `scope`);
- menyisipkan elemen multimedia (gambar, video, audio) ke halaman HTML;
- menggabungkan semua elemen ke dalam satu halaman yang rapi.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- membangun form pendaftaran akun dengan berbagai jenis *input*;
- membuat tabel data mahasiswa dengan struktur aksesibel;
- menyisipkan gambar, video, dan audio ke halaman HTML;
- menggabungkan semua elemen ke dalam satu halaman praktikum.

4.1 Form HTML

Catatan

Form adalah cara pengguna mengirim data ke server. Kita fokus pada struktur form-nya dulu; pengiriman data (*submit*) akan dipelajari saat belajar JavaScript.

4.1.1 Live Coding: Form Pendaftaran Akun

Langkah Kerja

1. Di folder latihan-web-3 (atau buat folder baru latihan-web-4), buat file `form.html`.
2. Ketik struktur HTML dasar + hubungkan ke `style.css` (dari Bab 3), atau buat style *inline* baru.
3. Ketik kode form berikut di dalam `<body>`.

Listing 4.1: Form pendaftaran akun

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Form Pendaftaran --- Latihan HTML Lanjutan</title>
  <style>
    body {
      font-family: Arial, sans-serif;
```

```
        max-width: 600px;
        margin: 0 auto;
        padding: 20px;
        line-height: 1.6;
    }
    h1 { color: #2c3e50; }
    form {
        background: #f9f9f9;
        padding: 20px;
        border-radius: 8px;
    }
    label {
        display: block;
        margin-top: 12px;
        font-weight: bold;
    }
    input, select, textarea {
        width: 100%;
        padding: 8px;
        margin-top: 4px;
        border: 1px solid #ccc;
        border-radius: 4px;
        box-sizing: border-box;
    }
    fieldset {
        border: 1px solid #ccc;
        border-radius: 8px;
        padding: 12px;
        margin-top: 16px;
    }
    legend {
        font-weight: bold;
        padding: 0 8px;
    }
    .radio-group label,
    .checkbox-group label {
        display: inline;
        font-weight: normal;
    }
    button {
        margin-top: 16px;
        padding: 10px 20px;
        background: #2c3e50;
        color: white;
        border: none;
        border-radius: 4px;
        font-size: 16px;
        cursor: pointer;
    }
```

```

    }
    button:hover { background: #34495e; }
</style>
</head>
<body>

<h1>Form Pendaftaran Akun</h1>
<p>Isi form berikut untuk mendaftar.
    Tanda <span style="color:red;">*</span>
    wajib diisi.</p>

<form action="#" method="post">

    <!-- FIELDSET 1: Data Diri -->
    <fieldset>
        <legend>Data Diri</legend>

        <label for="nama">Nama Lengkap
            <span style="color:red;">*</span></label>
        <input type="text" id="nama" name="nama"
            placeholder="Masukkan_nama_lengkap"
            required>

        <label for="email">Email
            <span style="color:red;">*</span></label>
        <input type="email" id="email" name="email"
            placeholder="contoh@email.com"
            required>

        <label for="password">Password
            <span style="color:red;">*</span></label>
        <input type="password" id="password"
            name="password"
            placeholder="Minimal_8_karakter"
            minlength="8" required>

        <label for="tanggal">Tanggal Lahir</label>
        <input type="date" id="tanggal" name="tanggal">

        <label for="jk">Jenis Kelamin
            <span style="color:red;">*</span></label>
        <div class="radio-group">
            <input type="radio" id="laki" name="jk"
                value="laki-laki" required>
            <label for="laki">Laki-laki</label>

            <input type="radio" id="perempuan"
                name="jk" value="perempuan">

```

```

        <label for="perempuan">Perempuan</label>
    </div>
</fieldset>

<!-- FIELDSET 2: Informasi Akademik -->
<fieldset>
    <legend>Informasi Akademik</legend>

    <label for="prodi">Program Studi
        <span style="color:red;">*</span></label>
    <select id="prodi" name="prodi" required>
        <option value="">-- Pilih Prodi --</option>
        <option value="d3-informatika">
            D3 Informatika</option>
        <option value="d3-multimedia">
            D3 Multimedia</option>
        <option value="d3-akuntansi">
            D3 Akuntansi</option>
        <option value="d3-administrasi">
            D3 Administrasi Bisnis</option>
    </select>

    <label for="angkatan">Angkatan</label>
    <input type="number" id="angkatan"
        name="angkatan" min="2020" max="2030"
        placeholder="Contoh: 2026">

    <label>Hobi / Minat (boleh pilih lebih dari
        1):</label>
    <div class="checkbox-group">
        <input type="checkbox" id="hobi-web"
            name="hobi" value="web-design">
        <label for="hobi-web">Desain Web</label>
        <br>
        <input type="checkbox" id="hobi-mobile"
            name="hobi" value="mobile-app">
        <label for="hobi-mobile">Mobile App</label>
        <br>
        <input type="checkbox" id="hobi-ai"
            name="hobi" value="ai">
        <label for="hobi-ai">AI / Machine
            Learning</label>
        <br>
        <input type="checkbox" id="hobi-network"
            name="hobi" value="networking">
        <label for="hobi-network">Jaringan</label>
    </div>
</fieldset>

```

```

<!-- FIELDSET 3: Pesan -->
<fieldset>
  <legend>Catatan Tambahan</legend>

  <label for="pesan">Tulis pesan / motivasi
    kamu:</label>
  <textarea id="pesan" name="pesan" rows="5"
    placeholder="Tulis di sini...">
  </textarea>
</fieldset>

<!-- Tombol Submit -->
<button type="submit">Daftar Sekarang</button>

</form>

</body>
</html>

```

Simpan → buka dengan *Live Server*.

4.1.2 Jenis-Jenis Input

Tabel 4.1: Jenis input form HTML

Jenis Input	Fungsi	Contoh Penggunaan
<code>type="text"</code>	Teks biasa (satu baris)	Nama, judul
<code>type="email"</code>	Teks + validasi format email	Alamat email
<code>type="password"</code>	Teks tersembunyi (bullet)	Password
<code>type="number"</code>	Hanya angka + tombol naik/turun	Usia, tahun
<code>type="date"</code>	<i>Date picker</i> (kalender)	Tanggal lahir
<code>type="radio"</code>	Pilih satu dari beberapa opsi	Jenis kelamin
<code>type="checkbox"</code>	Pilih beberapa opsi	Hobi, minat
<code>type="tel"</code>	Nomor telepon	No. HP
<code>type="url"</code>	Teks + validasi format URL	Website
<code>type="file"</code>	Upload file	Upload foto profil
<code><select></code>	<i>Dropdown</i> (daftar pilihan)	Program studi
<code><textarea></code>	Teks panjang (multi-baris)	Pesan, komentar

4.1.3 Pentingnya Label

Listing 4.2: Perbandingan label benar dan salah

```

<!-- Benar: label terhubung ke input via "for" = id -->
<label for="nama">Nama Lengkap</label>
<input type="text" id="nama" name="nama">

<!-- Salah: label tidak terhubung -->
<label>Nama Lengkap</label>
<input type="text">

```

Catatan

Kenapa harus pakai label?

1. **Aksesibilitas:** *Screen reader* bisa membaca “Nama Lengkap” saat input difokuskan.
2. **UX:** Klik pada label akan memfokuskan kursor ke input yang sesuai.
3. **Validasi:** Peramban menampilkan pesan error yang lebih informatif.

4.1.4 Atribut Validasi Dasar

Tabel 4.2: Atribut validasi form

Atribut	Fungsi	Contoh
required	Input wajib diisi	Semua field penting
minlength / maxlength	Batas panjang teks	Password minimal 8 karakter
min / max	Batas angka	Tahun 2020–2030
placeholder	Teks petunjuk di dalam input	“Masukkan email”
pattern	Pola <i>regex</i> untuk validasi	Format telepon tertentu
disabled	Nonaktifkan input	Field yang belum tersedia
readonly	Boleh dilihat, tidak bisa diedit	Field otomatis

4.1.5 Verifikasi Form

Coba:

1. Klik tombol **Daftar Sekarang** tanpa mengisi apapun → peramban harus menampilkan pesan error “Please fill out this field” pada field `required`.
2. Isi *email* dengan format salah (misal: bukanemail) → tekan *Submit* → peramban harus menampilkan pesan error tentang format email.
3. Isi *password* kurang dari 8 karakter → peramban harus menolak.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-04/form-pendaftaran.png`

Keterangan: Form pendaftaran di peramban.

Instruksi mahasiswa: Pastikan terlihat semua field (Data Diri, Info Akademik, Pesan) dan tombol “Daftar Sekarang”.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-04/form-pendaftaran.png` tersedia.

4.2 Tabel HTML

Catatan

Tabel digunakan untuk menampilkan data berbentuk baris dan kolom. Penting untuk data terstruktur seperti jadwal, perbandingan harga, statistik, dsb.

4.2.1 Live Coding: Tabel Sederhana

Buat file baru: `tabel.html`. Ketik kode berikut:

Listing 4.3: Tabel data mahasiswa dengan `colspan` dan `rowspan`

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Tabel Data --- Latihan HTML Lanjutan</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 800px;
      margin: 20px auto;
    }
    table {
      width: 100%;
      border-collapse: collapse;
      margin: 20px 0;
    }
    th, td {
      border: 1px solid #ddd;
      padding: 10px 12px;
      text-align: left;
    }
    thead { background-color: #2c3e50; color: white; }
    tbody tr:nth-child(even) {
      background-color: #f2f2f2;
    }
    caption {
```

```

        font-size: 1.2em;
        font-weight: bold;
        margin-bottom: 10px;
    }
</style>
</head>
<body>

<h1>Data Mahasiswa D3 Informatika</h1>

<!-- TABLE 1: Tabel Dasar -->
<table>
    <caption>Jadwal Kuliah Semester 1</caption>
    <thead>
        <tr>
            <th scope="col">No</th>
            <th scope="col">Mata Kuliah</th>
            <th scope="col">Hari</th>
            <th scope="col">Jam</th>
            <th scope="col">SKS</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>1</td>
            <td>Desain Web</td>
            <td>Senin</td>
            <td>08:00 &ndash; 10:00</td>
            <td>3</td>
        </tr>
        <tr>
            <td>2</td>
            <td>Algoritma Pemrograman</td>
            <td>Selasa</td>
            <td>10:00 &ndash; 12:00</td>
            <td>3</td>
        </tr>
        <tr>
            <td>3</td>
            <td>Basis Data</td>
            <td>Rabu</td>
            <td>13:00 &ndash; 15:00</td>
            <td>3</td>
        </tr>
        <tr>
            <td>4</td>
            <td>Matematika Diskrit</td>
            <td>Kamis</td>

```

```

        <td>08:00 &ndash; 10:00</td>
        <td>3</td>
    </tr>
    <tr>
        <td>5</td>
        <td>Bahasa Inggris Teknik</td>
        <td>Jumat</td>
        <td>10:00 &ndash; 12:00</td>
        <td>2</td>
    </tr>
</tbody>
<tfoot>
    <tr>
        <td colspan="4"><strong>Total SKS</strong></td>
        <td><strong>14</strong></td>
    </tr>
</tfoot>
</table>

```

```
<hr>
```

```
<!-- TABLE 2: Tabel dengan colspan & rowspan -->
```

```
<h2>Perbandingan Nilai UTS</h2>
```

```

<table>
    <thead>
        <tr>
            <th rowspan="2" scope="col">Mata Kuliah</th>
            <th colspan="3" scope="colgroup">Nilai Per Kelompok</th>
        </tr>
        <tr>
            <th scope="col">Kelompok A</th>
            <th scope="col">Kelompok B</th>
            <th scope="col">Kelompok C</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Desain Web</td>
            <td>85</td>
            <td>78</td>
            <td>90</td>
        </tr>
        <tr>
            <td>Algoritma</td>
            <td>72</td>
            <td>88</td>
            <td>81</td>
        </tr>
    </tbody>
</table>

```

```

        </tr>
        <tr>
            <td>Basis Data</td>
            <td>91</td>
            <td>76</td>
            <td>85</td>
        </tr>
    </tbody>
</table>

</body>
</html>

```

Simpan → buka dengan *Live Server*.

4.2.2 Struktur Tabel

Listing 4.4: Struktur dasar tabel HTML

```

<table>
  <caption> Judul tabel (wajib ada untuk aksesibilitas)
  <thead> Bagian kepala tabel (baris judul kolom)
    <tr> Table row (baris)
    <th> Table header (judul kolom/baris)
  <tbody> Bagian isi tabel (data)
    <tr>
    <td> Table data (sel data)
  <tfoot> Bagian kaki tabel (total, ringkasan)
    <tr>
    <td>
</table>

```

4.2.3 Atribut Penting untuk Aksesibilitas

Tabel 4.3: Atribut tabel untuk aksesibilitas

Atribut	Fungsi	Contoh
scope="col"	Menandai header sebagai kolom	<th scope="col">Nama</th>
scope="row"	Menandai header sebagai baris	<th scope="row">Senin</th>
colspan="N"	Sel melebar N kolom	<td colspan="3">Gabungan</td>
rowspan="N"	Sel melebar N baris	<th rowspan="2">Header</th>

Catatan

Atribut scope membantu *screen reader* menavigasi tabel dengan benar.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-04/tabel-data.png`

Keterangan: Tabel data mahasiswa di peramban.

Instruksi mahasiswa: Pastikan terlihat 2 tabel: Jadwal Kuliah dan Perbandingan Nilai, dengan *styling* yang rapi.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-04/tabel-data.png` tersedia.

4.3 Multimedia: Gambar, Video, Audio

4.3.1 Gambar — Review & Lanjutan

Kita sudah belajar `<img>` di Bab 3. Sekarang tambahkan `<figure>`:

Listing 4.5: Gambar dengan figure dan figcaption

```
<figure>
  
  <figcaption>Gambar 1 --- Pemandangan alam sebagai
    contoh konten visual</figcaption>
</figure>
```

Atribut tambahan `loading="lazy"`: gambar hanya dimuat saat pengguna *scroll* ke bagian tersebut (hemat *bandwidth* & mempercepat *loading* halaman pertama).

4.3.2 Video (HTML5)

Listing 4.6: Menyisipkan video HTML5

```
<!-- Video dari URL langsung -->
<video width="640" height="360" controls>
  <source src="video-tutorial.mp4" type="video/mp4">
  Browser kamu tidak mendukung elemen video.
</video>

<!-- Contoh dengan video publik -->
<video width="640" height="360" controls
  poster="https://placeholder.co/640x360">
  <source src="https://www.w3schools.com/html/mov_bbb.mp4"
    type="video/mp4">
  Browser kamu tidak mendukung elemen video.
```

```
</video>
```

Tabel 4.4: Atribut elemen video

Atribut	Fungsi
controls	Menampilkan tombol play/pause/volume
autoplay	Video otomatis dimuat (hindari — mengganggu pengguna)
loop	Video diulang terus-menerus
muted	Video tanpa suara
poster	Gambar <i>thumbnail</i> sebelum video dimainkan
width / height	Ukuran tampilan video

4.3.3 Audio (HTML5)

Listing 4.7: Menyisipkan elemen audio

```
<audio controls>
  <source src="https://www.w3schools.com/html/horse.mp3"
    type="audio/mpeg">
  Browser kamu tidak mendukung elemen audio.
</audio>
```

Tabel 4.5: Atribut elemen audio

Atribut	Fungsi
controls	Menampilkan tombol play/pause/volume
autoplay	Audio otomatis diputar (sangat tidak disarankan)
loop	Audio diulang terus-menerus
muted	Audio tanpa suara

4.3.4 Iframe — Embed Konten Eksternal

Listing 4.8: Menyisipkan video YouTube via iframe

```
<!-- Embed YouTube video -->
<iframe width="560" height="315"
  src="https://www.youtube.com/embed/dQw4w9WgXcQ"
  title="Contoh video YouTube"
  frameborder="0"
  allow="accelerometer; autoplay; clipboard-write;
  encrypted-media; gyroscope; picture-in-picture"
  allowfullscreen>
</iframe>
```

Peringatan

Selalu isi atribut `title` pada `<iframe>` untuk aksesibilitas.

4.4 Praktik: Menggabungkan Semua Elemen

Langkah Kerja

1. Buat file praktikum-minggu-4.html yang menggabungkan semua elemen yang sudah dipelajari.
2. Gunakan struktur *semantic* HTML (`<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`).
3. Sertakan section form, tabel, dan multimedia dalam satu halaman.

Listing 4.9: Halaman praktikum minggu 4 (ringkasan struktur)

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Praktikum Minggu 4 --- Form, Tabel &
    Multimedia</title>
  <style>
    * { box-sizing: border-box; }
    body {
      font-family: Arial, sans-serif;
      max-width: 900px;
      margin: 0 auto;
      padding: 20px;
      line-height: 1.6;
      color: #333;
    }
    h1 {
      color: #2c3e50;
      border-bottom: 2px solid #2c3e50;
      padding-bottom: 8px;
    }
    h2 { color: #34495e; margin-top: 40px; }
    nav {
      background: #2c3e50;
      padding: 12px;
      border-radius: 8px;
      margin-bottom: 20px;
    }
    nav a {
      color: white;
      text-decoration: none;
      margin: 0 12px;
    }
    nav a:hover { text-decoration: underline; }
```

```
table {
    width: 100%;
    border-collapse: collapse;
    margin: 16px 0;
}
th, td {
    border: 1px solid #ddd;
    padding: 8px 12px;
    text-align: left;
}
thead { background: #2c3e50; color: white; }
caption {
    font-weight: bold;
    margin-bottom: 8px;
}
form {
    background: #f9f9f9;
    padding: 20px;
    border-radius: 8px;
}
label {
    display: block;
    margin-top: 10px;
    font-weight: bold;
}
input, select, textarea {
    width: 100%;
    padding: 8px;
    margin-top: 4px;
    border: 1px solid #ccc;
    border-radius: 4px;
}
button {
    margin-top: 14px;
    padding: 10px 24px;
    background: #2c3e50;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-size: 16px;
}
button:hover { background: #34495e; }
figure {
    margin: 20px 0;
    text-align: center;
}
figcaption {
```

```

        font-style: italic;
        margin-top: 6px;
        color: #666;
    }
    footer {
        background: #2c3e50;
        color: white;
        text-align: center;
        padding: 12px;
        border-radius: 8px;
        margin-top: 40px;
    }
</style>
</head>
<body>

<!-- HEADER & NAV -->
<header>
    <h1>Praktikum Minggu 4</h1>
    <nav>
        <a href="#form">Form</a>
        <a href="#tabel">Tabel</a>
        <a href="#multimedia">Multimedia</a>
    </nav>
</header>

<main>

<!-- SECTION: FORM -->
<section id="form">
    <h2>1. Form Pendaftaran</h2>
    <form action="#" method="post">
        <label for="p-nama">Nama Lengkap
            <span style="color:red;">*</span></label>
        <input type="text" id="p-nama" name="nama"
            placeholder="Nama_kamu" required>

        <label for="p-email">Email
            <span style="color:red;">*</span></label>
        <input type="email" id="p-email"
            name="email"
            placeholder="email@contoh.com"
            required>

        <label for="p-prodi">Program Studi</label>
        <select id="p-prodi" name="prodi">
            <option value="">-- Pilih --</option>
            <option value="d3-informatika">

```

```

        D3 Informatika</option>
        <option value="d3-multimedia">
        D3 Multimedia</option>
    </select>

    <label for="p-pesan">Pesan</label>
    <textarea id="p-pesan" name="pesan"
        rows="4"
        placeholder="Tulis pesan...">
    </textarea>

    <button type="submit">Kirim</button>
</form>
</section>

<!-- SECTION: TABEL -->
<section id="tabel">
    <h2>2. Tabel Perbandingan</h2>
    <table>
        <caption>Perbandingan Framework CSS</caption>
        <thead>
            <tr>
                <th scope="col">Fitur</th>
                <th scope="col">CSS Biasa</th>
                <th scope="col">Tailwind CSS</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>Cara penulisan</td>
                <td>File terpisah (.css)</td>
                <td>Class inline di HTML</td>
            </tr>
            <tr>
                <td>Learning curve</td>
                <td>Rendah</td>
                <td>Sedang</td>
            </tr>
            <tr>
                <td>Ukuran file akhir</td>
                <td>Terkontrol manual</td>
                <td>Di-browse otomatis</td>
            </tr>
        </tbody>
    </table>
</section>

<!-- SECTION: MULTIMEDIA -->

```

```

<section id="multimedia">
  <h2>3. Multimedia</h2>

  <figure>
    
    <figcaption>Gambar 1 --- Demonstrasi
        elemen <img> dengan
        <figure></figcaption>
  </figure>

  <h3>Video</h3>
  <video width="100%" controls
    poster="https://placeholder.co/640x360">
    <source
      src="https://www.w3schools.com/html/mov_bbb.mp4"
      type="video/mp4">
    Browser kamu tidak mendukung elemen video.
  </video>

  <h3>Audio</h3>
  <audio controls style="width:100%;">
    <source
      src="https://www.w3schools.com/html/horse.mp3"
      type="audio/mpeg">
    Browser kamu tidak mendukung elemen audio.
  </audio>
</section>

</main>

<footer>
  <p>&copy; 2026 --- Praktikum Minggu 4 |
    D3 Informatika</p>
</footer>

</body>
</html>

```

Tips

Pastikan di peramban:

- Tautan *anchor* (#form, #tabel, #multimedia) berfungsi.
- Form bisa diisi dan menampilkan pesan error jika field *required* kosong.
- Tabel muncul dengan benar.

- Gambar muncul dengan *caption*.
- Video dan audio bisa diputar.
- *Footer* muncul di bagian bawah.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-04/praktikum-section-form.png`

Keterangan: Section Form di halaman praktikum.

Instruksi mahasiswa: Pastikan terlihat label, input field, select *dropdown*, textarea, dan tombol *submit*.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-04/praktikum-section-form.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-04/praktikum-section-tabel-multimedia.png`

Keterangan: Section Tabel dan Multimedia.

Instruksi mahasiswa: Pastikan terlihat tabel perbandingan, gambar dengan *caption*, video *player*, dan audio *player*.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-04/praktikum-section-tabel-multimedia.png` tersedia.

4.5 Validasi & Push ke GitHub

4.5.1 Validasi HTML

Langkah Kerja

1. Buka https://validator.w3.org/#validate_by_input.
2. Salin-tempel isi `praktikum-minggu-4.html` → klik **Check**.
3. Perbaiki semua error hingga hasilnya bersih.

4.5.2 Push ke GitHub

Listing 4.10: Perintah push ke GitHub

```
# Inisialisasi (jika folder baru)
git init
git remote add origin https://github.com/NAMA_USER/latihan-web-4.git
```

```
# Commit dan push
git add .
git commit -m "Latihan HTML minggu 4: form, tabel, multimedia"
git branch -M main
git push -u origin main
```

4.6 Latihan Mandiri

Latihan 4.1 Form “Survei Kepuasan”

Buat halaman `survei.html` yang berisi form survei kepuasan mahasiswa terhadap layanan kampus. Field yang wajib ada:

1. **Nama** — `type="text", required`
2. **NIM** — `type="number", required, minlength="8"`
3. **Email** — `type="email", required`
4. **Fakultas** — `<select>` (minimal 4 pilihan), `required`
5. **Semester saat ini** — `type="number", min="1" max="8"`
6. **Tingkat kepuasan** — `type="radio" (1–5), required, gunakan <fieldset> + <legend>`
7. **Saran/keluhan** — `<textarea>`, `rows="5"`
8. **Setuju dihubungi** — `type="checkbox"`

Gunakan minimal 2 `<fieldset>` untuk mengelompokkan field. Setiap field harus punya `<label>` yang terhubung (`for = id`). Gunakan `<main>`, `<header>`, `<footer>` (*semantic HTML*).

Latihan 4.2 Tabel “Data Buku”

Buat halaman `buku.html` yang berisi tabel daftar buku dengan kolom: No, Judul Buku, Pengarang, Tahun, Harga. Persyaratan:

- Minimal 8 baris data (buku).
- Gunakan `<thead>`, `<tbody>`, `<tfoot>` (`<tfoot>` untuk total rata-rata harga).
- Gunakan `scope="col"` pada header kolom.
- Gunakan `colspan` di *footer* untuk kolom “Total”.
- Tambahkan `<caption>` pada tabel.
- Sertakan minimal 1 gambar (misal: sampul buku *placeholder*) di atas tabel dengan `<figure>` + `<figcaption>`.

Latihan 4.3 Push & Screenshot

1. *Push* semua file ke GitHub.
2. Ambil *screenshot* hasil latihan (form & tabel).
3. Pastikan tidak ada *warning* di W3C Validator.

Tabel 4.6: Masalah umum form, tabel, dan multimedia

Masalah	Solusi
Form tidak melakukan apa-apa saat <i>submit</i>	<code>action="#"</code> hanya <i>placeholder</i> — tidak ada <i>backend</i> . Validasi peramban tetap berjalan meskipun tidak ada server
<i>Radio button</i> bisa dipilih semua	Pastikan semua <i>radio button</i> punya nama yang sama (<code>name="jk"</code>), sehingga hanya satu yang bisa dipilih
Label tidak memfokuskan input saat diklik	Pastikan atribut <code>for</code> pada <code><label></code> = <code>id</code> pada <code><input></code> (harus sama persis)
Tampilan tabel berantakan di <i>mobile</i>	Tabel memang tidak responsif secara <i>default</i> — akan dipelajari cara membuatnya responsif di CSS
Video/audio tidak muncul	Periksa URL/path file. Jika dari URL eksternal, pastikan URL masih aktif
<code><caption></code> tidak terlihat	Pastikan <code><caption></code> adalah anak langsung <code><table></code> dan berada sebelum <code><thead></code>
Tombol <i>submit</i> me- <i>reload</i> halaman	Normal — karena <code>action="#"</code> ; saat belajar JavaScript, <i>reload</i> akan dicegah dengan <code>event.preventDefault()</code>

4.7 Troubleshooting

4.8 Rangkuman

- Membuat form HTML dengan berbagai jenis *input*.
- Menggunakan `<label>`, `<fieldset>`, `<legend>` untuk form yang rapi & aksesibel.
- Atribut validasi: `required`, `minlength`, `min`, `max`, `placeholder`.
- Membuat tabel dengan `<table>`, `<caption>`, `<thead>`, `<tbody>`, `<tfoot>`.
- Atribut tabel: `scope`, `colspan`, `rowspan`.
- Menyisipkan multimedia: `<img>`, `<figure>`, `<video>`, `<audio>`, `<iframe>`.

Tips

Cheat Sheet Form Input Types:

- `text` → Teks biasa
- `email` → Email (validasi format)
- `password` → Teks tersembunyi
- `number` → Angka + tombol `+/-`
- `date` → *Date picker* (kalender)
- `tel` → Nomor telepon
- `url` → URL (validasi format)
- `radio` → Pilih 1 dari beberapa
- `checkbox` → Pilih beberapa
- `file` → *Upload* file
- `color` → *Color picker*
- `range` → Slider
- `hidden` → Tersembunyi
- `submit` → Tombol *submit*
- `reset` → Tombol *reset* form

Bab 5 CSS Dasar: Selector, Box Model, Typography

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- menjelaskan peran CSS dalam halaman web;
- menulis CSS dengan tiga cara: *inline*, *internal*, dan eksternal;
- menggunakan berbagai jenis *selector* (tag, class, id, *descendant*);
- memahami dan menerapkan konsep *box model* (*margin*, *border*, *padding*, *content*);
- menerapkan dasar *typography*: *font*, ukuran, warna, dan spasi teks.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- menyiapkan folder proyek latihan-css;
- menulis CSS dengan tiga pendekatan dan membandingkannya;
- mempraktikkan *selector* tag, class, id, dan *descendant*;
- membuat elemen kotak untuk memahami *box model*;
- mengatur tipografi pada halaman web.

5.1 Pengantar CSS

CSS (*Cascading Style Sheets*) adalah bahasa untuk mengatur **tampilan** halaman web. HTML menentukan **struktur**, sedangkan CSS menentukan **penampilan**.

Catatan

CSS tidak mengubah isi halaman—hanya cara halaman itu ditampilkan.

5.1.1 Box Model

Setiap elemen HTML dianggap sebagai sebuah **kotak** yang terdiri dari empat lapisan:

Listing 5.1: Ilustrasi box model

```
+-----+
|      margin      |
| +-----+ |
| |      border   | | | | | |
| | +-----+ | |
| | | padding    | | |
| | | +-----+ | | |
| | | | content  | | | |
| | | | +-----+ | | |
| | | +-----+ | |
| | +-----+ |
| +-----+ |
+-----+
```

- **Content** — isi elemen (teks/gambar).
- **Padding** — ruang di dalam *border*, antara *border* dan *content*.
- **Border** — garis pembatas elemen.
- **Margin** — ruang di luar *border*, antara elemen dan elemen lain.

5.2 Praktikum 1 — Menyiapkan Proyek

Langkah Kerja

1. Buka VSCode, lalu buat folder baru bernama `latihan-css`.
2. Di dalam folder tersebut, buat file baru bernama `index.html`.
3. Ketik struktur dasar HTML berikut.

Listing 5.2: Struktur dasar HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Latihan CSS</title>
</head>
<body>
  <h1>Selamat Datang di Latihan CSS</h1>
  <p>Ini adalah paragraf pertama saya.</p>
  <p class="highlight">
    Ini paragraf dengan class highlight.
  </p>
  <p id="spesial">
    Ini paragraf dengan id spesial.
  </p>
</body>
</html>
```

Langkah Kerja

1. Klik kanan pada file `index.html`, pilih **Open with Live Server**.
2. Buka browser; pastikan halaman tampil. Anda akan melihat teks polos tanpa *styling*.

Catatan

Belum ada CSS, jadi halaman tampil apa adanya (default browser).

5.3 Praktikum 2 — Cara Menulis CSS

Ada tiga cara menulis CSS. Kita akan mencoba semuanya.

5.3.1 Inline CSS (langsung di tag)

Langkah Kerja

1. Ubah tag `<h1>` menjadi:

Listing 5.3: Inline CSS pada tag `h1`

```
<h1 style="color: blue; text-align: center;">
  Selamat Datang di Latihan CSS
</h1>
```

Langkah Kerja

1. Simpan (Ctrl+S) dan lihat hasilnya di browser. Judul menjadi biru dan di tengah.

Catatan

Inline CSS hanya berlaku untuk satu elemen. Cara ini jarang dipakai karena sulit dirawat.

5.3.2 Internal CSS (di dalam tag `<style>`)

Langkah Kerja

1. Di dalam `<head>`, tambahkan tag `<style>`:

Listing 5.4: Internal CSS

```
<head>
  <meta charset="UTF-8">
  <title>Latihan CSS</title>
  <style>
    p {
      color: green;
      font-size: 18px;
    }
  </style>
</head>
```

Langkah Kerja

1. Simpan dan lihat hasilnya. Semua paragraf menjadi hijau.

5.3.3 Eksternal CSS (file terpisah)

Tips

Gunakan **eksternal CSS** untuk proyek nyata karena mudah dirawat dan dipakai ulang.

Langkah Kerja

1. Buat file baru bernama `style.css` di folder yang sama.
2. Ketik isi berikut.

Listing 5.5: Isi `style.css`

```
body {
  font-family: Arial, sans-serif;
```

```
background-color: #f4f4f4;
margin: 20px;
}
```

Langkah Kerja

1. Hubungkan file CSS ke HTML. Di dalam <head>, tambahkan:

Listing 5.6: Penghubung eksternal CSS

```
<link rel="stylesheet" href="style.css">
```

Langkah Kerja

1. Hapus tag <style> dan atribut style yang tadi dibuat, supaya bersih.
2. Simpan dan lihat hasilnya. Latar belakang menjadi abu-abu muda dan teks memakai *font* Arial.

5.4 Praktikum 3 — Selector

Selector menentukan **elemen mana** yang akan diberi *style*.

Langkah Kerja

1. Buka file `style.css` dan tambahkan aturan berikut.

Listing 5.7: Berbagai jenis selector

```
/* 1. Selector tag: semua elemen <h1> */
h1 {
    color: navy;
    text-align: center;
}

/* 2. Selector class: elemen dengan class="highlight" */
.highlight {
    background-color: yellow;
    font-weight: bold;
}

/* 3. Selector id: elemen dengan id="spesial" */
#spesial {
    color: red;
    font-size: 24px;
}

/* 4. Selector descendant: <p> di dalam <div> */
div p {
    color: purple;
}
```

Langkah Kerja

1. Tambahkan sebuah <div> di dalam <body>:

Listing 5.8: Div dengan paragraf descendant

```
<div>
  <p>Ini paragraf di dalam div, warnanya ungu.</p>
</div>
```

Langkah Kerja

1. Simpan dan amati hasilnya di browser.
2. Uji pemahaman—ubah warna pada masing-masing *selector* dan lihat perubahannya.

Tips

- **Class** (.nama) bisa dipakai banyak elemen → paling sering digunakan.
- **Id** (#nama) hanya untuk satu elemen unik.
- **Selector tag** memengaruhi semua elemen dengan tag tersebut.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/bab-05/hasil-selector.png

Keterangan: Hasil index.html setelah menerapkan selector (judul navy, paragraf hijau, *highlight* kuning, id merah, paragraf ungu di dalam div).

Instruksi mahasiswa: Buka index.html di peramban, lalu bandingkan tampilan Anda dengan contoh.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan \includegraphics setelah aset assets/images/bab-05/hasil-selector.png tersedia.

5.5 Praktikum 4 — Box Model

Langkah Kerja

1. Buat file baru box.html dengan isi berikut.

Listing 5.9: Struktur box.html

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Box Model</title>
  <link rel="stylesheet" href="box.css">
</head>
<body>
  <div class="kotak">Kotak 1</div>
  <div class="kotak">Kotak 2</div>
</body>
</html>
```

Langkah Kerja

1. Buat file `box.css`:

Listing 5.10: Gaya box model pada `.kotak`

```
.kotak {
  width: 200px;
  height: 100px;
  background-color: lightblue;
  border: 3px solid navy;
  padding: 20px;
  margin: 15px;
  text-align: center;
  line-height: 100px;
}
```

Langkah Kerja

1. Buka `box.html` dengan Live Server. Amati jarak antar kotak (*margin*), ruang dalam kotak (*padding*), dan garis tepi (*border*).
2. Ubah nilai-nilai berikut satu per satu dan amati perubahannya.

Listing 5.11: Variasi nilai box model

```
.kotak {
  padding: 5px;           /* coba ubah dari 20px */
  margin: 50px;          /* coba ubah dari 15px */
  border: 10px dashed red; /* coba ubah */
}
```

Langkah Kerja

1. Gunakan **DevTools** (F12 → tab *Elements* → tab *Computed*) untuk melihat kotak model secara visual.

Tips

Buat kotak dengan `border-radius: 50%` dan amati hasilnya—kotak menjadi lingkaran.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-05/box-model-devtools.png`

Keterangan: DevTools → tab *Computed* yang menampilkan *box model* salah satu kotak.

Instruksi mahasiswa: Buka DevTools, pilih elemen `.kotak`, lalu lihat diagram *computed style*.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-05/box-model-devtools.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-05/hasil-box.html.png`

Keterangan: Hasil `box.html` dengan dua kotak biru muda ber-*border* navy dan jarak antar kotak.

Instruksi mahasiswa: Buka `box.html` di peramban dan bandingkan tampilan Anda.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-05/hasil-box.html.png` tersedia.

5.6 Praktikum 5 — Typography

Langkah Kerja

1. Buat file `tipografi.html`:

Listing 5.12: Struktur `tipografi.html`

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Tipografi</title>
  <link rel="stylesheet" href="tipografi.css">
</head>
<body>
  <h1>Judul Halaman</h1>
  <h2>Sub Judul</h2>
  <p class="paragraf-1">
    Ini paragraf pertama dengan pengaturan
    font dan spasi.
  </p>
  <p class="paragraf-2">
    Ini paragraf kedua dengan gaya yang berbeda.
  </p>
</body>
</html>
```

Langkah Kerja

1. Buat file `tipografi.css`:

Listing 5.13: Gaya tipografi

```
body {
  font-family: 'Segoe UI', Tahoma, Geneva,
    Verdana, sans-serif;
  font-size: 16px;
  color: #333;
  line-height: 1.6;
```

```

}

h1 {
  font-size: 2.5em;
  font-weight: bold;
  color: #2c3e50;
  letter-spacing: 1px;
}

h2 {
  font-size: 1.8em;
  color: #34495e;
}

.paragraf-1 {
  font-style: italic;
  text-align: justify;
}

.paragraf-2 {
  font-weight: bold;
  text-transform: uppercase;
  text-decoration: underline;
}

```

Langkah Kerja

1. Buka di Live Server dan amati perbedaannya.
2. Coba ubah nilai font-size, color, line-height, dan text-align untuk melihat efeknya.

Tips

Gunakan situs seperti coolors.co atau colorhunt.co untuk memilih palet warna yang serasi.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/bab-05/hasil-tipografi.html.png

Keterangan: Hasil tipografi.html dengan judul, sub judul, dan dua paragraf bergaya berbeda.

Instruksi mahasiswa: Buka tipografi.html di peramban dan bandingkan.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan \includegraphics setelah aset
assets/images/bab-05/hasil-tipografi.html.png tersedia.

5.7 Latihan Mandiri

Latihan 5.1 Profil Saya

Buat halaman profil sederhana dengan ketentuan berikut.

- Buat file `profil.html` dan `profil.css`.
- Halaman berisi judul “Profil Saya”, satu foto/*placeholder* gambar, nama, NIM, program studi, dan paragraf singkat tentang diri Anda.
- Terapkan *selector* tag, class, dan id (minimal masing-masing 1).
- Terapkan *box model*: *padding* dan *margin* pada kartu profil.
- Gunakan *border* dengan *border-radius* agar sudut membulat.
- Atur *typography*: *font* berbeda untuk judul dan isi, warna yang serasi.

Struktur HTML yang disarankan:

Listing 5.14: Kerangka kartu profil

```
<div class="kartu">
  
  <h1 id="nama">Nama Anda</h1>
  <p class="info">NIM: 12345678</p>
  <p class="info">
    Program Studi: D3 Informatika
  </p>
  <p class="deskripsi">Tentang saya...</p>
</div>
```

5.8 Hasil yang Diharapkan

Setelah menyelesaikan semua praktikum, Anda memiliki:

- Folder `latihan-css` berisi minimal 6 file: `index.html`, `style.css`, `box.html`, `box.css`, `tipografi.html`, `tipografi.css`.
- Halaman `index.html` dengan judul biru navy di tengah, paragraf hijau, satu paragraf berlatar kuning, satu paragraf merah, dan satu paragraf ungu.
- Halaman `box.html` dengan dua kotak biru muda ber-*border* navy, berjarak satu sama lain.
- Halaman `tipografi.html` dengan judul besar, sub judul, dan dua paragraf bergaya berbeda.
- Halaman `profil.html` (latihan mandiri) berupa kartu profil yang rapi.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-05/hasil-profil.html.png`

Keterangan: Hasil latihan mandiri `profil.html` (kartu profil).

Instruksi mahasiswa: Buat `profil.html` sesuai ketentuan latihan, lalu bandingkan tampilan Anda.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-05/hasil-profil.html.png` tersedia.

5.9 Troubleshooting

Tabel 5.1: Masalah umum dan solusinya

Masalah	Penyebab	Solusi
CSS tidak berpengaruh	File CSS tidak terhubung	Pastikan ada <code><link rel="stylesheet" href="style.css"></code> dan nama file benar
Perubahan tidak muncul di browser	Browser belum di-refresh	Simpan file (Ctrl+S) dan <i>re-fresh</i> browser, atau pastikan Live Server aktif
<i>Selector</i> tidak bekerja	Nama class/id salah ketik	Periksa penulisan <code>.nama</code> (class) vs <code>#nama</code> (id)
Jarak antar elemen aneh	<i>Margin/padding</i> belum dipahami	Buka DevTools → <i>Computed</i> untuk melihat <i>box model</i>
<i>Font</i> tidak berubah	<i>Font</i> tidak tersedia di sistem	Gunakan <i>font</i> umum seperti Arial, sans-serif sebagai <i>fallback</i>
Gambar tidak muncul	Path salah	Pastikan file gambar ada di folder yang sama dan nama file benar

5.10 Rangkuman

- CSS mengatur tampilan, HTML mengatur struktur.
- Gunakan **eksternal CSS** untuk proyek nyata.
- *Selector*: tag, class (`.`), id (`#`), *descendant*.
- *Box model*: *content* → *padding* → *border* → *margin*.
- *Typography*: atur *font*, ukuran, warna, dan spasi teks agar mudah dibaca.

Bab 6 CSS Layout: Flexbox

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- menjelaskan konsep *flexbox* sebagai alat *layout* satu dimensi;
- menggunakan properti `display: flex` pada kontainer;
- mengatur arah, pembungkusan, dan perataan item *flex*;
- mengatur ukuran dan urutan item *flex*;
- membangun *layout* sederhana (*navbar*, kartu, galeri) dengan *flexbox*.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- menyiapkan folder proyek latihan-*flexbox*;
- mempraktikkan properti utama *flexbox*;
- membangun *navbar* dan galeri foto dengan *flexbox*.

6.1 Pengantar Flexbox

Flexbox (*Flexible Box*) adalah cara mengatur **tata letak satu dimensi**—baik baris (horizontal) maupun kolom (vertikal). *Flexbox* sangat berguna untuk menyusun elemen secara sejajar dan merata.

6.1.1 Konsep Utama

- **Flex container** — elemen induk yang diberi `display: flex`.
- **Flex item** — elemen anak di dalam *flex container*.

Listing 6.1: Ilustrasi flex container dan flex items

```
+-----+
| Flex Container (display: flex)          |
| +-----+ +-----+ +-----+          |
| | item 1 | | item 2 | | item 3 |          |
| +-----+ +-----+ +-----+          |
+-----+
```

6.1.2 Properti Penting

6.2 Praktikum 1 — Menyiapkan Proyek

Langkah Kerja

1. Buat folder latihan-*flexbox* di VSCode.
2. Buat file `index.html`.

Tabel 6.1: Properti utama *flexbox*

Properti	Fungsi
<code>display: flex</code>	Mengaktifkan <i>flexbox</i> pada kontainer
<code>flex-direction</code>	Arah item: <code>row</code> , <code>column</code> , <code>row-reverse</code> , <code>column-reverse</code>
<code>justify-content</code>	Perataan item pada sumbu utama (horizontal untuk <code>row</code>)
<code>align-items</code>	Perataan item pada sumbu silang (vertikal untuk <code>row</code>)
<code>flex-wrap</code>	Membolehkan item pindah baris (<code>wrap</code>)
<code>gap</code>	Jarak antar item
<code>flex</code>	Mengatur ukuran relatif item

Listing 6.2: Struktur `index.html`

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Latihan Flexbox</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <div class="item">1</div>
    <div class="item">2</div>
    <div class="item">3</div>
  </div>
</body>
</html>

```

Langkah Kerja

1. Buat file `style.css`:

Listing 6.3: Gaya dasar *flexbox*

```

.container {
  display: flex;
  background-color: #ddd;
  padding: 10px;
}

.item {
  background-color: lightblue;
  border: 2px solid navy;
  padding: 20px;
  margin: 5px;
  font-size: 20px;
}

```

Langkah Kerja

1. Buka dengan Live Server. Ketiga item tampil **sejajar horizontal** karena `display: flex`.

6.3 Praktikum 2 — Flex Direction

Langkah Kerja

1. Tambahkan properti berikut pada `.container`:

Listing 6.4: `flex-direction: row` (default)

```
.container {
  display: flex;
  flex-direction: row;
  background-color: #ddd;
  padding: 10px;
}
```

Langkah Kerja

1. Ubah `row` menjadi `column` dan amati hasilnya (item tersusun vertikal).
2. Coba juga `row-reverse` dan `column-reverse`. Amati urutan item berubah.

Catatan

`flex-direction` menentukan **sumbu utama**. `row` = sumbu utama horizontal, `column` = sumbu utama vertikal.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-06/flex-direction-row.png`

Keterangan: Hasil `index.html` dengan 3 item sejajar horizontal (`flex-direction: row`).

Instruksi mahasiswa: Buka `index.html` di peramban dan amati posisi item.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-06/flex-direction-row.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-06/flex-direction-column.png`

Keterangan: Hasil `index.html` setelah `flex-direction: column` (item tersusun vertikal).

Instruksi mahasiswa: Ubah `flex-direction` menjadi `column`, simpan, dan perbarui peramban.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-06/flex-direction-column.png` tersedia.

6.4 Praktikum 3 — Justify Content & Align Items

Langkah Kerja

1. Kembalikan flex-direction menjadi row.
2. Tambahkan justify-content dengan berbagai nilai. Coba satu per satu.

Listing 6.5: justify-content

```
.container {
  display: flex;
  justify-content: center;
  background-color: #ddd;
  padding: 10px;
}
```

Amati perbedaan:

- flex-start — item di kiri.
- flex-end — item di kanan.
- center — item di tengah.
- space-between — jarak merata, item ujung menempel tepi.
- space-around — jarak merata dengan ruang di tepi.
- space-evenly — jarak benar-benar merata.

Langkah Kerja

1. Atur tinggi kontainer dan coba align-items.

Listing 6.6: align-items

```
.container {
  display: flex;
  height: 200px;
  align-items: center;
  background-color: #ddd;
  padding: 10px;
}
```

Tips

justify-content mengatur sumbu utama, align-items mengatur sumbu silang. Jangan tertukar!

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/bab-06/justify-space-between.png

Keterangan: Hasil justify-content: space-between (item ujung menempel tepi).

Instruksi mahasiswa: Ubah nilai justify-content, simpan, dan amati posisi item di peramban.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-06/justify-space-between.png` tersedia.

6.5 Praktikum 4 — Flex Wrap & Gap

Langkah Kerja

1. Tambahkan lebih banyak item di HTML (misal 6 item).
2. Atur lebar item agar sempit:

Listing 6.7: Lebar tetap pada item

```
.item {
  width: 150px;
  background-color: lightblue;
  border: 2px solid navy;
  padding: 20px;
  margin: 5px;
}
```

Langkah Kerja

1. Tanpa flex-wrap, item akan menyempit agar muat satu baris. Tambahkan:

Listing 6.8: flex-wrap dan gap

```
.container {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  background-color: #ddd;
  padding: 10px;
}
```

Langkah Kerja

1. Hapus margin pada `.item` dan gunakan gap pada kontainer. Amati hasilnya lebih rapi.

Catatan

gap lebih modern dan rapi daripada margin untuk jarak antar item *flex*.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-06/flex-wrap.png`

Keterangan: Hasil flex-wrap: wrap dengan 6 item yang pindah baris.

Instruksi mahasiswa: Ubah ukuran peramban dan amati item berpindah baris secara otomatis.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-06/flex-wrap.png` tersedia.

6.6 Praktikum 5 — Properti Flex pada Item

Langkah Kerja

1. Buat file baru `flex-item.html`:

Listing 6.9: Struktur `flex-item.html`

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Flex Item</title>
  <link rel="stylesheet" href="flex-item.css">
</head>
<body>
  <div class="container">
    <div class="item_a">A</div>
    <div class="item_b">B</div>
    <div class="item_c">C</div>
  </div>
</body>
</html>
```

Langkah Kerja

1. Buat `flex-item.css`:

Listing 6.10: Proporsi flex item

```
.container {
  display: flex;
  background-color: #ddd;
  padding: 10px;
}

.item {
  background-color: lightgreen;
  border: 2px solid green;
  padding: 20px;
  margin: 5px;
}

/* Item A mengambil 1 bagian */
.a { flex: 1; }
```

```
/* Item B mengambil 2 bagian */
.b { flex: 2; }

/* Item C mengambil 1 bagian */
.c { flex: 1; }
```

Langkah Kerja

1. Buka di Live Server. Item B tampil dua kali lebih lebar dari A dan C.
2. Coba ubah nilai flex pada masing-masing item dan amati perbandingan lebarnya.

Tips

`flex: 1` berarti “ambil ruang yang tersisa secara proporsional”. Ini sangat berguna untuk *layout* yang fleksibel.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-06/flex-item.html.png`

Keterangan: Hasil `flex-item.html` dengan item B lebih lebar (`flex: 2`).

Instruksi mahasiswa: Buka `flex-item.html` di peramban dan bandingkan lebar masing-masing item.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-06/flex-item.html.png` tersedia.

6.7 Praktikum 6 — Membangun Navbar dengan Flexbox

Langkah Kerja

1. Buat file `navbar.html`:

Listing 6.11: Struktur `navbar.html`

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Navbar Flexbox</title>
  <link rel="stylesheet" href="navbar.css">
</head>
<body>
  <nav class="navbar">
    <div class="logo">MySite</div>
    <ul class="menu">
      <li><a href="#">Beranda</a></li>
      <li><a href="#">Tentang</a></li>
      <li><a href="#">Kontak</a></li>
```

```

        </ul>
        <a href="#" class="btn">Login</a>
    </nav>
</body>
</html>

```

Langkah Kerja

1. Buat navbar.css:

Listing 6.12: Gaya navbar dengan flexbox

```

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    background-color: #2c3e50;
    color: white;
    padding: 15px 30px;
}

.logo {
    font-size: 24px;
    font-weight: bold;
}

.menu {
    display: flex;
    list-style: none;
    gap: 20px;
}

.menu a {
    color: white;
    text-decoration: none;
}

.menu a:hover {
    color: #3498db;
}

.btn {
    background-color: #3498db;
    color: white;
    padding: 8px 16px;
}

```

```
border-radius: 5px;
text-decoration: none;
}
```

Langkah Kerja

1. Buka di Live Server. Anda memiliki *navbar* dengan logo di kiri, menu di tengah, dan tombol *login* di kanan.
2. Coba ubah `justify-content` menjadi `center` dan amati perubahannya.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-06/navbar.html.png`

Keterangan: Hasil `navbar.html` (logo kiri, menu tengah, tombol kanan).

Instruksi mahasiswa: Buka `navbar.html` di peramban dan amati posisi elemen.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-06/navbar.html.png` tersedia.

6.8 Praktikum 7 — Galeri Foto dengan Flexbox

Langkah Kerja

1. Buat file `galeri.html`:

Listing 6.13: Struktur `galeri.html`

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Galeri Flexbox</title>
  <link rel="stylesheet" href="galeri.css">
</head>
<body>
  <h1>Galeri Foto</h1>
  <div class="galeri">
    <div class="foto">Foto 1</div>
    <div class="foto">Foto 2</div>
    <div class="foto">Foto 3</div>
    <div class="foto">Foto 4</div>
    <div class="foto">Foto 5</div>
    <div class="foto">Foto 6</div>
  </div>
</body>
</html>
```

Langkah Kerja

1. Buat galeri.css:

Listing 6.14: Gaya galeri dengan flexbox

```
.galeri {
    display: flex;
    flex-wrap: wrap;
    gap: 15px;
    justify-content: center;
}

.foto {
    width: 200px;
    height: 150px;
    background-color: #3498db;
    color: white;
    display: flex;
    justify-content: center;
    align-items: center;
    border-radius: 8px;
    font-size: 18px;
}
```

Langkah Kerja

1. Buka di Live Server. Foto-foto tersusun rapi dan otomatis pindah baris saat jendela dipersempit.
2. Persempit jendela browser dan amati bagaimana item berpindah baris (karena `flex-wrap: wrap`).

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-06/galeri.html.png`

Keterangan: Hasil `galeri.html` dengan foto-foto tersusun rapi.

Instruksi mahasiswa: Buka `galeri.html` di peramban, lalu ubah ukuran jendela untuk melihat *wrapping*.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-06/galeri.html.png` tersedia.

6.9 Latihan Mandiri

Latihan 6.1 Kartu Produk

Buat halaman kartu produk dengan *flexbox*. Ketentuan:

- Buat file `produk.html` dan `produk.css`.
- Halaman berisi judul “Produk Kami” dan 3 kartu produk sejajar.
- Setiap kartu berisi nama produk, harga, dan tombol “Beli”.

- Gunakan `display: flex` untuk menyusun kartu sejajar.
- Gunakan `justify-content: space-between` atau `center`.
- Gunakan `gap` untuk jarak antar kartu.
- Gunakan `flex-direction: column` di dalam kartu agar isi tersusun vertikal.
- Gunakan `align-items: center` untuk menengahkan isi kartu.

Struktur HTML yang disarankan:

Listing 6.15: Kerangka kartu produk

```
<div class="produk-list">
  <div class="kartu">
    <h3>Produk A</h3>
    <p class="harga">Rp 50.000</p>
    <button>Beli</button>
  </div>
  <div class="kartu">
    <h3>Produk B</h3>
    <p class="harga">Rp 75.000</p>
    <button>Beli</button>
  </div>
  <div class="kartu">
    <h3>Produk C</h3>
    <p class="harga">Rp 100.000</p>
    <button>Beli</button>
  </div>
</div>
```

6.10 Hasil yang Diharapkan

Setelah menyelesaikan semua praktikum, Anda memiliki:

- Folder latihan-flexbox berisi file: `index.html`, `style.css`, `flex-item.html`, `flex-item.css`, `navbar.html`, `navbar.css`, `galeri.html`, `galeri.css`.
- Halaman `index.html` dengan 3 item sejajar yang bisa diubah arah dan perataannya.
- Halaman `flex-item.html` dengan item B dua kali lebih lebar dari A dan C.
- Halaman `navbar.html` dengan *navbar* rapi: logo kiri, menu tengah, tombol kanan.
- Halaman `galeri.html` dengan foto-foto yang otomatis pindah baris saat jendela dipersempit.
- Halaman `produk.html` (latihan mandiri) dengan 3 kartu produk sejajar.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-06/produk.html.png`

Keterangan: Hasil latihan mandiri `produk.html` (3 kartu produk sejajar).

Instruksi mahasiswa: Buat `produk.html` sesuai ketentuan latihan, lalu bandingkan tampilan Anda.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-06/produk.html.png` tersedia.

6.11 Troubleshooting

Tabel 6.2: Masalah umum *flexbox* dan solusinya

Masalah	Penyebab	Solusi
Item tidak sejajar	<code>display: flex</code> belum ditambahkan	Pastikan properti <code>display: flex</code> ada di kontainer, bukan di item
<code>justify-content</code> tidak berpengaruh	Salah sumbu	<code>justify-content</code> bekerja pada sumbu utama; jika <code>flex-direction: column</code> , ia mengatur vertikal
Item menyempit saat jendela kecil	<code>flex-wrap</code> belum diaktifkan	Tambahkan <code>flex-wrap: wrap</code> pada kontainer
Jarak antar item tidak muncul	Menggunakan <code>margin</code> tapi <code>gap</code> tidak ada	Gunakan <code>gap</code> pada kontainer untuk jarak yang konsisten
Item tidak menengahkan teks	<code>align-items/justify-content</code> belum diatur	Untuk menengahkan, atur keduanya menjadi <code>center</code>
<i>Layout</i> berantakan	Ada <code>margin/padding</code> default browser	Tambahkan <code>* { margin: 0; padding: 0; box-sizing: border-box; }</code> di awal CSS

6.12 Rangkuman

- *Flexbox* mengatur *layout* **satu dimensi** (baris atau kolom).
- `display: flex` pada kontainer mengaktifkan *flexbox*.
- `flex-direction` menentukan sumbu utama.
- `justify-content` mengatur sumbu utama, `align-items` mengatur sumbu silang.
- `flex-wrap: wrap` memungkinkan item pindah baris.
- `gap` mengatur jarak antar item.
- `flex: 1` membuat item mengambil ruang secara proporsional.

Bab 7 CSS Layout: Grid & Responsive Design

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- menjelaskan konsep CSS *grid* untuk *layout* dua dimensi;
- membuat *layout grid* dengan baris dan kolom;
- menggunakan *media query* untuk membuat halaman responsif;
- menerapkan prinsip *mobile-first*;
- membangun halaman yang tampil baik di layar besar maupun kecil.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- menyiapkan folder proyek latihan-grid;
- mempraktikkan properti *grid* (*grid-template-columns*, *grid-template-rows*, *grid-column*);
- membangun *layout* halaman lengkap (header, sidebar, konten, footer);
- menerapkan *media query* untuk tampilan responsif.

7.1 Pengantar CSS Grid

CSS Grid adalah sistem *layout* **dua dimensi**—mengatur baris **dan** kolom sekaligus. Berbeda dengan *flexbox* yang satu dimensi, *grid* sangat cocok untuk *layout* halaman penuh (header, sidebar, konten, footer).

Listing 7.1: Ilustrasi grid container dan cells

```
+-----+
| Grid Container (display: grid)          |
| +-----+ +-----+ +-----+          |
| | col 1 | | col 2 | | col 3 | <- baris 1 |
| +-----+ +-----+ +-----+          |
| +-----+ +-----+ +-----+          |
| | col 1 | | col 2 | | col 3 | <- baris 2 |
| +-----+ +-----+ +-----+          |
+-----+
```

7.1.1 Media Query

Media query memungkinkan CSS **berubah** berdasarkan ukuran layar perangkat. Ini adalah kunci dari *responsive design*.

Listing 7.2: Sintaks media query

```
@media (max-width: 768px) {
  /* Aturan ini hanya berlaku saat layar <= 768px */
}
```

7.1.2 Breakpoint Umum

Tabel 7.1: Breakpoint umum untuk *responsive design*

Breakpoint	Target Perangkat
max-width: 600px	Ponsel
max-width: 768px	Tablet
max-width: 992px	Laptop kecil
min-width: 1200px	Desktop besar

7.2 Praktikum 1 — Menyiapkan Proyek

Langkah Kerja

1. Buat folder latihan-grid di VSCode.
2. Buat file index.html:

Listing 7.3: Struktur index.html

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Latihan Grid</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="grid">
    <div class="cell">1</div>
    <div class="cell">2</div>
    <div class="cell">3</div>
    <div class="cell">4</div>
    <div class="cell">5</div>
    <div class="cell">6</div>
  </div>
</body>
</html>
```

Langkah Kerja

1. Buat file style.css:

Listing 7.4: Grid tiga kolom

```
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
```

```
.cell {
  background-color: lightcoral;
  border: 2px solid darkred;
  padding: 30px;
  text-align: center;
  font-size: 20px;
}
```

Langkah Kerja

1. Buka dengan Live Server. Enam sel tersusun dalam **3 kolom × 2 baris**.

7.3 Praktikum 2 — Mengatur Kolom dan Baris

Langkah Kerja

1. Ubah grid-template-columns menjadi berbagai variasi:

Listing 7.5: Kolom tidak sama lebar

```
.grid {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
  gap: 10px;
}
```

Langkah Kerja

1. Coba juga dengan satuan tetap:

Listing 7.6: Kolom tetap + fleksibel

```
.grid {
  display: grid;
  grid-template-columns: 200px 1fr;
  gap: 10px;
}
```

Langkah Kerja

1. Tambahkan grid-template-rows:

Listing 7.7: Mengatur tinggi baris

```
.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: 100px 200px;
  gap: 10px;
}
```

Langkah Kerja

1. Amati bagaimana tinggi baris berbeda.

Tips

1fr berarti “1 fraksi dari ruang yang tersedia”. 1fr 2fr berarti kolom kedua dua kali lebih lebar dari kolom pertama.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/bab-07/grid-1fr2fr.png

Keterangan: Hasil index.html dengan grid-template-columns: 1fr 2fr 1fr (kolom tengah lebih lebar).

Instruksi mahasiswa: Ubah nilai grid-template-columns, simpan, dan amati perubahan di peramban.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset assets/images/bab-07/grid-1fr2fr.png tersedia.

7.4 Praktikum 3 — Menempatkan Item di Grid

Langkah Kerja

1. Buat file layout.html:

Listing 7.8: Struktur layout.html

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Layout Grid</title>
  <link rel="stylesheet" href="layout.css">
</head>
<body>
  <div class="layout">
    <header class="header">Header</header>
    <nav class="nav">Navigasi</nav>
    <main class="main">Konten Utama</main>
    <aside class="aside">Sidebar</aside>
    <footer class="footer">Footer</footer>
  </div>
</body>
</html>
```

Langkah Kerja

1. Buat layout.css:

Listing 7.9: Layout halaman dengan grid

```
.layout {
  display: grid;
```

```
    grid-template-columns: 1fr 3fr;
    grid-template-rows: auto auto 1fr auto;
    gap: 10px;
    min-height: 100vh;
}

.header {
    grid-column: 1 / -1;
    background-color: #2c3e50;
    color: white;
    padding: 20px;
}

.nav {
    background-color: #3498db;
    color: white;
    padding: 20px;
}

.main {
    background-color: #ecf0f1;
    padding: 20px;
}

.aside {
    background-color: #e67e22;
    color: white;
    padding: 20px;
}

.footer {
    grid-column: 1 / -1;
    background-color: #2c3e50;
    color: white;
    padding: 20px;
    text-align: center;
}
```

Langkah Kerja

1. Buka di Live Server. Anda memiliki *layout* halaman lengkap: header di atas, sidebar kiri, konten tengah, dan footer di bawah.
2. Coba ubah `grid-template-columns` menjadi `1fr 2fr` dan amati perubahannya.

Catatan

`grid-column: 1 / -1` berarti item membentang dari kolom 1 sampai kolom terakhir.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/bab-07/layout.html.png

Keterangan: Hasil layout.html (header, sidebar, konten, footer).

Instruksi mahasiswa: Buka layout.html di peramban dan amati struktur *layout* halaman.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset assets/images/bab-07/layout.html.png tersedia.

7.5 Praktikum 4 — Pengenalan Media Query

Langkah Kerja

1. Buat file responsive.html:

Listing 7.10: Struktur responsive.html

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Responsive</title>
  <link rel="stylesheet" href="responsive.css">
</head>
<body>
  <h1>Halaman Responsif</h1>
  <div class="konten">
    <div class="kartu">Kartu 1</div>
    <div class="kartu">Kartu 2</div>
    <div class="kartu">Kartu 3</div>
  </div>
</body>
</html>
```

Langkah Kerja

1. Buat responsive.css:

Listing 7.11: Grid responsif dengan media query

```
body {
  font-family: Arial, sans-serif;
  margin: 20px;
}

.konten {
  display: grid;
```

```

    grid-template-columns: 1fr 1fr 1fr;
    gap: 15px;
}

.kartu {
    background-color: #3498db;
    color: white;
    padding: 40px;
    text-align: center;
    border-radius: 8px;
}

/* Media query: saat layar <= 768px (tablet) */
@media (max-width: 768px) {
    .konten {
        grid-template-columns: 1fr 1fr;
    }
}

/* Media query: saat layar <= 480px (ponsel) */
@media (max-width: 480px) {
    .konten {
        grid-template-columns: 1fr;
    }
    h1 {
        font-size: 20px;
    }
}

```

Langkah Kerja

1. Buka di Live Server. Persempit dan lebarkan jendela browser. Amati bagaimana jumlah kolom berubah: 3 → 2 → 1.

Tips

Gunakan DevTools (F12) → ikon perangkat (*toggle device toolbar*) untuk mensimulasikan layar ponsel/tablet.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/bab-07/responsive-desktop.png

Keterangan: Hasil responsive.html di layar desktop (3 kolom).

Instruksi mahasiswa: Buka responsive.html di peramban dengan lebar penuh.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-07/responsive-desktop.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-07/responsive-mobile.png`

Keterangan: Hasil `responsive.html` di layar ponsel (1 kolom)—gunakan DevTools *device toolbar*.

Instruksi mahasiswa: Aktifkan DevTools → *toggle device toolbar*, lalu pilih ukuran ponsel.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-07/responsive-mobile.png` tersedia.

7.6 Praktikum 5 — Mobile-First

Prinsip **mobile-first**: tulis CSS untuk layar kecil dulu, lalu gunakan `min-width` untuk layar yang lebih besar.

Langkah Kerja

1. Buat file `mobile-first.css`:

Listing 7.12: Pendekatan mobile-first

```
/* Default: untuk ponsel (layar kecil) */
.konten {
    display: grid;
    grid-template-columns: 1fr;
    gap: 15px;
}

.kartu {
    background-color: #27ae60;
    color: white;
    padding: 40px;
    text-align: center;
    border-radius: 8px;
}

/* Layar >= 600px (tablet) */
@media (min-width: 600px) {
    .konten {
        grid-template-columns: 1fr 1fr;
    }
}
```

```

}

/* Layar >= 992px (desktop) */
@media (min-width: 992px) {
  .konten {
    grid-template-columns: 1fr 1fr 1fr;
  }
}

```

Langkah Kerja

1. Buat file `mobile-first.html` yang memakai CSS ini (salin dari `responsive.html`, ganti nama file CSS).
2. Buka di Live Server dan amati: di layar kecil tampil 1 kolom, makin lebar makin banyak kolom.

Catatan

Perbedaan: *mobile-first* memakai `min-width` (dari kecil ke besar), sedangkan pendekatan biasa memakai `max-width` (dari besar ke kecil). Keduanya sah; *mobile-first* lebih disarankan.

7.7 Praktikum 6 — Membuat Layout Responsif Lengkap

Langkah Kerja

1. Buat file `halaman.html`:

Listing 7.13: Struktur `halaman.html`

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Halaman Responsif Lengkap</title>
  <link rel="stylesheet" href="halaman.css">
</head>
<body>
  <header class="header">
    <h1>Website Saya</h1>
    <nav>
      <ul class="menu">
        <li><a href="#">Beranda</a></li>
        <li><a href="#">Tentang</a></li>
        <li><a href="#">Kontak</a></li>
      </ul>
    </nav>
  </header>

  <main class="konten">
    <section class="artikel">

```

```

        <h2>Artikel Utama</h2>
        <p>Ini adalah konten utama halaman.
        Teks ini akan menyesuaikan lebar
        layar.</p>
    </section>
    <aside class="sidebar">
        <h3>Sidebar</h3>
        <p>Informasi tambahan.</p>
    </aside>
</main>

<footer class="footer">
    <p>&copy; 2026 Website Saya</p>
</footer>
</body>
</html>

```

Langkah Kerja

1. Buat halaman.css (mobile-first):

Listing 7.14: Gaya halaman responsif (mobile-first)

```

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    line-height: 1.6;
}

.header {
    background-color: #2c3e50;
    color: white;
    padding: 20px;
    text-align: center;
}

.menu {
    list-style: none;
    display: flex;
    justify-content: center;
    gap: 20px;
    margin-top: 10px;
}

.menu a {
    color: white;
}

```

```
    text-decoration: none;
}

.konten {
    display: grid;
    grid-template-columns: 1fr;
    gap: 20px;
    padding: 20px;
    max-width: 1200px;
    margin: 0 auto;
}

.artikel {
    background-color: #ecf0f1;
    padding: 20px;
    border-radius: 8px;
}

.sidebar {
    background-color: #bdc3c7;
    padding: 20px;
    border-radius: 8px;
}

.footer {
    background-color: #2c3e50;
    color: white;
    text-align: center;
    padding: 15px;
}

/* Tablet: 2 kolom untuk konten */
@media (min-width: 600px) {
    .konten {
        grid-template-columns: 2fr 1fr;
    }
}

/* Desktop: menu di kanan header */
@media (min-width: 992px) {
    .header {
        display: flex;
        justify-content: space-between;
        align-items: center;
        text-align: left;
    }
    .menu {
        margin-top: 0;
    }
}
```

```
}
}
```

Langkah Kerja

1. Buka di Live Server. Uji di berbagai ukuran layar (gunakan DevTools *device toolbar*).
2. Amati:
 - Di ponsel: konten 1 kolom, header di tengah.
 - Di tablet: konten 2 kolom (artikel + sidebar).
 - Di desktop: header jadi sejajar (logo kiri, menu kanan).

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-07/halaman-desktop.png`

Keterangan: Hasil `halaman.html` di layar desktop (konten 2 kolom, header sejajar).

Instruksi mahasiswa: Buka `halaman.html` dengan lebar layar penuh.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-07/halaman-desktop.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-07/halaman-mobile.png`

Keterangan: Hasil `halaman.html` di layar ponsel (konten 1 kolom, header di tengah).

Instruksi mahasiswa: Aktifkan DevTools → *device toolbar* dan pilih ukuran ponsel.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-07/halaman-mobile.png` tersedia.

7.8 Latihan Mandiri

Latihan 7.1 Portofolio Responsif

Buat halaman portofolio yang responsif. Ketentuan:

- Buat file `portofolio.html` dan `portofolio.css`.
- Halaman berisi header dengan nama Anda dan menu navigasi, bagian “Tentang Saya”, bagian “Proyek” berisi minimal 6 kartu proyek, dan footer.
- Gunakan CSS *grid* untuk menyusun kartu proyek.
- Gunakan *media query (mobile-first)* agar: ponsel 1 kolom, tablet 2 kolom, desktop 3 kolom.
- Gunakan *flexbox* untuk menu navigasi.

Struktur HTML yang disarankan:

Listing 7.15: Kerangka portofolio.html

```

<header class="header">
  <h1>Nama Anda</h1>
  <nav>
    <ul class="menu">
      <li><a href="#">Beranda</a></li>
      <li><a href="#">Tentang</a></li>
      <li><a href="#">Proyek</a></li>
    </ul>
  </nav>
</header>

<section class="tentang">
  <h2>Tentang Saya</h2>
  <p>...</p>
</section>

<section class="proyek">
  <h2>Proyek Saya</h2>
  <div class="grid-proyek">
    <div class="kartu">Proyek 1</div>
    <!-- ... 6 kartu ... -->
  </div>
</section>

<footer class="footer">
  <p>&copy; 2026 Nama Anda</p>
</footer>

```

7.9 Hasil yang Diharapkan

Setelah menyelesaikan semua praktikum, Anda memiliki:

- Folder latihan-grid berisi file: index.html, style.css, layout.html, layout.css, responsive.html, responsive.css, mobile-first.html, mobile-first.css, halaman.html, halaman.css.
- Halaman index.html dengan 6 sel dalam 3 kolom.
- Halaman layout.html dengan *layout* lengkap: header, sidebar, konten, footer.
- Halaman responsive.html yang berubah dari 3 → 2 → 1 kolom saat layar dipersempit.
- Halaman halaman.html yang responsif dari ponsel hingga desktop.
- Halaman portofolio.html (latihan mandiri) dengan kartu proyek yang menyesuaikan jumlah kolom.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/bab-07/portofolio.html.png

Keterangan: Hasil latihan mandiri portofolio.html di layar desktop (3 kolom kartu).

Instruksi mahasiswa: Buat portofolio.html sesuai ketentuan latihan, lalu bandingkan tampilan Anda.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-07/portofolio.html.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-07/portofolio-mobile.png`

Keterangan: Hasil latihan mandiri `portofolio.html` di layar ponsel (1 kolom kartu).

Instruksi mahasiswa: Gunakan DevTools → *device toolbar* untuk menguji tampilan ponsel.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-07/portofolio-mobile.png` tersedia.

7.10 Troubleshooting

Tabel 7.2: Masalah umum CSS *grid* dan *responsive design*

Masalah	Penyebab	Solusi
<i>Grid</i> tidak tampil	<code>display: grid</code> belum ditambahkan	Pastikan properti <code>display: grid</code> ada di kontainer
Item tidak pindah kolom saat layar kecil	<i>Media query</i> salah atau tidak ada	Pastikan ada <code>@media (max-width: ...)</code> atau <code>@media (min-width: ...)</code>
<i>Media query</i> tidak berpengaruh	Urutan <i>media query</i> salah	Letakkan <i>media query</i> setelah aturan dasar; untuk <code>min-width</code> , urutkan dari kecil ke besar
<i>Layout</i> berantakan di ponsel	Tidak ada <i>viewport meta tag</i>	Pastikan ada <code><meta name="viewport" content="width=device-width, initial-scale=1.0"></code>
Kolom tidak sama lebar	Menggunakan satuan tetap	Gunakan <code>1fr</code> untuk kolom yang fleksibel
Item menumpuk	<code>grid-template-columns</code> tidak sesuai	Periksa jumlah kolom yang didefinisikan
Perubahan tidak terlihat	Browser <i>cache</i>	<i>Refresh</i> dengan <code>Ctrl+Shift+R</code>

7.11 Rangkuman

- CSS *grid* mengatur *layout* **dua dimensi** (baris + kolom).
- `grid-template-columns` dan `grid-template-rows` menentukan struktur *grid*.

- 1fr = satu fraksi ruang yang tersedia.
- `grid-column: 1 / -1` membuat item membentang semua kolom.
- *Media query* membuat halaman **responsif** terhadap ukuran layar.
- **Mobile-first**: tulis CSS ponsel dulu, lalu tingkatkan dengan `min-width`.
- Selalu sertakan `<meta name="viewport">` untuk halaman responsif.

Bab 8 Git & GitHub Dasar: Init, Commit, Push, Pull

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- menjelaskan kegunaan Git dan GitHub;
- menginstal dan mengonfigurasi Git di komputer;
- menginisialisasi *repository* Git (`git init`);
- melakukan *commit* dengan pesan yang baik;
- membuat *repository* di GitHub dan menghubungkannya;
- melakukan *push* dan *pull*.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- memeriksa instalasi Git dan mengonfigurasi identitas;
- menginisialisasi *repository* dan melakukan *commit* pertama;
- membuat *repository* di GitHub dan menghubungkannya dengan *repository* lokal;
- mempraktikkan alur kerja `git add` → `git commit` → `git push` → `git pull`.

8.1 Apa itu Git?

Git adalah **sistem kontrol versi** (*version control system*). Git melacak setiap perubahan pada file, sehingga Anda bisa:

- melihat riwayat perubahan;
- kembali ke versi sebelumnya;
- bekerja bersama orang lain tanpa menimpa pekerjaan.

8.2 Apa itu GitHub?

GitHub adalah **layanan hosting** untuk *repository* Git di internet. GitHub memungkinkan Anda menyimpan kode di cloud, berkolaborasi, dan (nanti) men-*deploy* website.

8.3 Alur Kerja Dasar

Listing 8.1: Alur kerja Git

```
Working Directory -> (git add) -> Staging Area
-> (git commit) -> Repository Lokal
-> (git push) -> GitHub
```

- **Working directory** — file yang sedang Anda edit.
- **Staging area** — file yang “dipilih” untuk di-*commit*.
- **Repository lokal** — riwayat *commit* di komputer Anda.

- **GitHub** — salinan *repository* di internet.

8.4 Praktikum 1 — Memeriksa Instalasi Git

Langkah Kerja

1. Buka terminal di VSCode (menu *Terminal* → *New Terminal*).
2. Ketik perintah berikut untuk memeriksa versi Git.

Listing 8.2: Memeriksa versi Git

```
git --version
```

Langkah Kerja

1. Jika muncul versi (mis. `git version 2.40.0`), Git sudah terinstal. Jika belum, unduh dari <https://git-scm.com/downloads>, instal dengan pengaturan default, lalu ulangi Langkah 2.

Catatan

Saat instalasi, pilih editor default sesuai keinginan (mis. VSCode) dan biarkan opsi lain default.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-08/git-version.png`

Keterangan: Hasil `git --version` di terminal (menunjukkan versi Git terinstal).

Instruksi mahasiswa: Jalankan `git --version` di terminal dan pastikan muncul nomor versi.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-08/git-version.png` tersedia.

8.5 Praktikum 2 — Konfigurasi Git

Sebelum mulai, Git perlu tahu **siapa Anda**. Ini akan tercatat di setiap *commit*.

Langkah Kerja

1. Di terminal, ketik (ganti dengan nama dan email Anda):

Listing 8.3: Konfigurasi identitas Git

```
git config --global user.name "NamaAnda"
git config --global user.email "email@contoh.com"
```

Langkah Kerja

1. Gunakan email yang sama dengan akun GitHub Anda.
2. Periksa konfigurasi:

Listing 8.4: Memeriksa konfigurasi

```
git config --list
```

Langkah Kerja

1. Pastikan `user.name` dan `user.email` muncul dengan benar.

Tips

Konfigurasi `--global` berlaku untuk semua proyek di komputer Anda. Lakukan sekali saja.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-08/git-config-list.png`

Keterangan: Hasil `git config --list` (menampilkan `user.name` dan `user.email`).

Instruksi mahasiswa: Jalankan `git config --list` dan pastikan identitas Anda tercatat.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-08/git-config-list.png` tersedia.

8.6 Praktikum 3 — Inisialisasi Repository

Langkah Kerja

1. Buka folder proyek yang sudah ada, misalnya `latihan-css` dari Bab 5.
2. Di terminal, pastikan Anda berada di folder proyek. Periksa dengan:

Listing 8.5: Memeriksa direktori kerja

```
pwd
```

Langkah Kerja

1. Inisialisasi *repository* Git:

Listing 8.6: Inisialisasi repository

```
git init
```

Langkah Kerja

1. Periksa status *repository*:

Listing 8.7: Memeriksa status repository

```
git status
```

Catatan

`git init` membuat folder tersembunyi `.git` yang berisi riwayat versi. Jangan hapus folder ini.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-08/git-status-untracked.png`

Keterangan: Hasil `git status` setelah `git init` (file untracked).

Instruksi mahasiswa: Jalankan `git status` setelah `git init` dan amati daftar file.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-08/git-status-untracked.png` tersedia.

8.7 Praktikum 4 — Membuat File .gitignore

Langkah Kerja

1. Buat file bernama `.gitignore` di folder proyek.
2. Isi dengan file/folder yang **tidak perlu** di-*commit*:

Listing 8.8: Isi `.gitignore`

```
node_modules/
.DS_Store
*.log
```

Langkah Kerja

1. Simpan file.

Catatan

`.gitignore` memberi tahu Git file mana yang harus diabaikan. Berguna untuk file sementara atau sensitif.

8.8 Praktikum 5 — Commit Pertama

Langkah Kerja

1. Tambahkan semua file ke *staging area*:

Listing 8.9: Menambahkan file ke staging area

```
git add .
```

Langkah Kerja

1. Periksa status:

Listing 8.10: Memeriksa status setelah `git add`

```
git status
```

Langkah Kerja

1. File yang berwarna hijau (*staged*) siap di-*commit*. Lakukan *commit* pertama:

Listing 8.11: Commit pertama

```
git commit -m "Proyek awal latihan CSS"
```

Langkah Kerja

1. Periksa riwayat *commit*:**Listing 8.12:** Melihat riwayat commit

```
git log --oneline
```

Tips**Tips penulisan pesan *commit* yang baik:**

- Singkat dan jelas (maksimal ~50 karakter).
- Jelaskan **apa** yang diubah, bukan sekadar “update”.
- Contoh: “Menambahkan halaman profil”, “Memperbaiki bug navbar”, “Mengubah warna tombol”.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-08/git-status-staged.png`

Keterangan: Hasil `git status` setelah `git add .` (file *staged*, berwarna hijau).

Instruksi mahasiswa: Jalankan `git add .` lalu `git status` dan amati status file.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-08/git-status-staged.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-08/git-log.png`

Keterangan: Hasil `git log --oneline` (riwayat *commit*).

Instruksi mahasiswa: Jalankan `git log --oneline` dan pastikan *commit* pertama tercatat.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-08/git-log.png` tersedia.

8.9 Praktikum 6 — Membuat Repository di GitHub

Langkah Kerja

1. Buka <https://github.com> dan *login*.
2. Klik tombol + di pojok kanan atas, lalu pilih **New repository**.
3. Isi:
 - **Repository name:** `latihan-css` (atau nama proyek Anda).
 - **Description:** (opsional) “Latihan CSS minggu 5”.
 - **Visibility:** pilih **Public**.
 - **Jangan centang** “Add a README file” (karena kita sudah punya file).

Langkah Kerja

1. Klik **Create repository**.
2. Anda akan melihat halaman berisi perintah untuk menghubungkan *repository* lokal ke GitHub. Simpan halaman ini—kita akan menggunakannya di praktikum berikutnya.

Catatan

Jangan tutup halaman ini dulu. Perhatikan bagian “... or push an existing repository from the command line”.

8.10 Praktikum 7 — Menghubungkan dan Push**Langkah Kerja**

1. Kembali ke terminal VSCode.
2. Tambahkan *remote* (alamat *repository* GitHub). Ganti USERNAME dan NAMA-REPO sesuai milik Anda:

Listing 8.13: Menambahkan remote origin

```
git remote add origin https://github.com/USERNAME/NAMA-REPO.git
```

Langkah Kerja

1. Periksa *remote*:

Listing 8.14: Memeriksa remote

```
git remote -v
```

Langkah Kerja

1. *Push commit* ke GitHub. Ganti main dengan nama branch Anda jika berbeda:

Listing 8.15: Push ke GitHub

```
git branch -M main
git push -u origin main
```

Langkah Kerja

1. Jika diminta *login*, ikuti petunjuk (*browser* akan terbuka untuk autentikasi GitHub).
2. Buka halaman *repository* di GitHub dan *refresh*. File-file Anda sekarang ada di GitHub!

Catatan

`-u` (*upstream*) membuat Git mengingat bahwa branch main terhubung ke origin/main, sehingga *push* berikutnya cukup `git push`.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-08/github-repo-setelah-push.png`

Keterangan: Halaman *repository* di GitHub setelah *push* pertama (berisi file proyek).

Instruksi mahasiswa: Buka halaman *repository* di GitHub dan pastikan file sudah muncul.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-08/github-repo-setelah-push.png` tersedia.

8.11 Praktikum 8 — Mengubah File dan Commit Lagi

Langkah Kerja

1. Ubah salah satu file di proyek Anda (mis. tambahkan paragraf baru di `index.html`).
2. Lihat perubahan yang terjadi:

Listing 8.16: Melihat perubahan

```
git status
git diff
```

Langkah Kerja

1. Tambahkan dan *commit* perubahan:

Listing 8.17: Commit perubahan baru

```
git add .
git commit -m "Menambahkan paragraf baru di index"
```

Langkah Kerja

1. *Push* ke GitHub:

Listing 8.18: Push ke GitHub

```
git push
```

Langkah Kerja

1. *Refresh* halaman GitHub dan lihat bahwa perubahan sudah masuk.

Catatan

Alur yang benar: `git add` → `git commit` → `git push`. Ulangi setiap kali selesai mengerjakan sesuatu.

8.12 Praktikum 9 — Pull (Mengambil Perubahan)

Langkah Kerja

1. Buka halaman *repository* di GitHub.
2. Klik file `index.html`, lalu klik ikon pensil (*edit*) di kanan atas.
3. Ubah sesuatu (mis. ganti judul), lalu klik **Commit changes** di bagian bawah.
4. Kembali ke terminal VSCode. Ambil perubahan dari GitHub:

Listing 8.19: Pull dari GitHub

```
git pull
```

Langkah Kerja

1. Buka `index.html` di VSCode—perubahan yang Anda buat di GitHub sekarang ada di komputer lokal.

Catatan

`git pull` mengambil perubahan terbaru dari GitHub ke komputer Anda. Lakukan ini sebelum mulai bekerja jika bekerja bersama tim.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-08/git-pull.png`

Keterangan: Hasil `git pull` yang berhasil mengambil perubahan dari GitHub.

Instruksi mahasiswa: Jalankan `git pull` dan pastikan perubahan dari GitHub sudah sinkron.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-08/git-pull.png` tersedia.

8.13 Latihan Mandiri

Latihan 8.1 Repository Flexbox

- Buat *repository* baru di GitHub bernama `latihan-flexbox`.
- Hubungkan folder `latihan-flexbox` (dari Bab 6) ke *repository* tersebut.
- Lakukan *commit* dengan pesan yang jelas untuk semua file.
- *Push* ke GitHub.
- Ubah satu file, *commit*, dan *push* lagi.
- Buat perubahan di GitHub (edit langsung), lalu `git pull` di komputer.
- Periksa `git log --oneline` dan pastikan semua *commit* tercatat.

8.14 Hasil yang Diharapkan

Setelah menyelesaikan semua praktikum, Anda memiliki:

- Git terinstal dan terkonfigurasi dengan nama dan email Anda.
- *Repository* lokal `latihan-css` dengan minimal 2 *commit*.
- *Repository* di GitHub bernama `latihan-css` berisi semua file proyek.
- Kemampuan untuk melakukan `git add`, `git commit`, `git push`, dan `git pull`.
- Riwayat *commit* yang bisa dilihat di `git log --oneline` dan di halaman GitHub.

8.15 Troubleshooting

Tabel 8.1: Masalah umum Git dan solusinya

Masalah	Penyebab	Solusi
git bukan perintah yang dikenali	Git belum terinstal / tidak di PATH	Instal Git dari git-scm.com , <i>restart</i> terminal/VSCode
git push meminta <i>password</i> terus	Autentikasi belum diatur	Gunakan <i>login browser</i> saat diminta, atau atur <i>Personal Access Token</i>
fatal: remote origin already exists	<i>Remote</i> sudah pernah ditambahkan	Gunakan <code>git remote set-url origin <URL></code> untuk mengganti
fatal: not a git repository	Bukan di folder proyek	Pastikan berada di folder yang sudah git init (periksa dengan <code>pwd</code>)
<i>Push</i> ditolak (<i>rejected</i>)	<i>Remote</i> punya <i>commit</i> yang tidak ada di lokal	Jalankan <code>git pull</code> dulu, lalu <code>git push</code> lagi
File tidak muncul di GitHub	Belum di- <i>commit</i> / di- <i>push</i>	Jalankan <code>git add .</code> , <code>git commit -m "..."</code> , lalu <code>git push</code>
<i>Commit</i> tidak tercatat	Belum git add	Jalankan <code>git add .</code> sebelum <code>git commit</code>
Branch bernama master bukan main	Versi Git lama	Jalankan <code>git branch -M main</code> sebelum <i>push</i>

8.16 Rangkuman

- Git = kontrol versi lokal; GitHub = hosting *repository* di internet.
- `git init` — mulai *repository*.
- `git add .` — pindahkan file ke *staging area*.
- `git commit -m "pesan"` — simpan *snapshot* perubahan.
- `git push` — kirim *commit* ke GitHub.
- `git pull` — ambil perubahan dari GitHub.
- `git status` — lihat kondisi *repository*.
- `git log --oneline` — lihat riwayat *commit*.
- Tulis pesan *commit* yang jelas dan singkat.

Bab 9 GitHub Kolaborasi: Branch, Pull Request, GitHub Pages

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- membuat dan berpindah branch di Git;
- menggabungkan branch (*merge*);
- membuat *pull request* (PR) di GitHub;
- men-*deploy* website statis ke GitHub Pages;
- menjelaskan alur kerja kolaborasi dasar dengan Git/GitHub.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- membuat branch baru dan bekerja di atasnya;
- menggabungkan branch ke main dengan *merge*;
- membuat dan menggabungkan *pull request* di GitHub;
- men-*deploy* website statis ke GitHub Pages.

9.1 Apa itu Branch?

Branch adalah **cabang** dari riwayat *commit*. Branch memungkinkan Anda bekerja pada fitur baru **tanpa mengganggu** kode utama.

Listing 9.1: Ilustrasi branch

```
main:      A --- B --- C
              \
fitur-navbar: D --- E
```

- **main** (atau **master**) — branch utama, kode yang stabil.
- **Branch fitur** — cabang untuk mengerjakan fitur tertentu, lalu digabungkan kembali ke main.

9.2 Apa itu Pull Request (PR)?

Pull request adalah **permintaan untuk menggabungkan** perubahan dari satu branch ke branch lain (biasanya ke main). PR memungkinkan orang lain **meninjau** kode sebelum digabung.

9.3 Apa itu GitHub Pages?

GitHub Pages adalah layanan **hosting gratis** dari GitHub untuk website statis (HTML/CSS/JS). Website Anda akan dapat diakses publik melalui URL seperti:

Listing 9.2: Format URL GitHub Pages

```
https://USERNAME.github.io/NAMA-REPO/
```

9.4 Praktikum 1 — Membuat Branch

Langkah Kerja

1. Buka folder proyek latihan-css di VSCode (pastikan sudah menjadi *repository* Git dari Bab 8).
2. Periksa branch saat ini:

Listing 9.3: Melihat branch

```
git branch
```

Langkah Kerja

1. Anda akan melihat `* main` (tanda `*` menunjukkan branch aktif).
2. Buat branch baru bernama `fitur-navbar`:

Listing 9.4: Membuat branch baru

```
git branch fitur-navbar
```

Langkah Kerja

1. Pindah ke branch baru:

Listing 9.5: Berpindah ke branch baru

```
git checkout fitur-navbar
```

Langkah Kerja

1. Periksa lagi:

Listing 9.6: Memeriksa branch aktif

```
git branch
```

Catatan

Cara cepat: `git checkout -b fitur-navbar` membuat branch **dan** langsung pindah ke sana dalam satu perintah.

9.5 Praktikum 2 — Bekerja di Branch Baru

Langkah Kerja

1. Di branch `fitur-navbar`, buat file baru bernama `navbar.html` (bisa salin dari Bab 6).
2. Tambahkan dan *commit* perubahan:

Listing 9.7: Commit di branch fitur

```
git add .
git commit -m "Menambahkan halaman navbar"
```

Langkah Kerja

1. Periksa riwayat:

Listing 9.8: Riwayat commit di branch fitur

```
git log --oneline
```

Langkah Kerja

1. Sekarang pindah kembali ke main:

Listing 9.9: Kembali ke branch main

```
git checkout main
```

Langkah Kerja

1. Periksa file di folder—`navbar.html` **tidak ada** di branch main, karena perubahan hanya ada di branch `fitur-navbar`.

Catatan

Konsep penting: Setiap branch memiliki riwayat *commit* sendiri. Perubahan di satu branch tidak terlihat di branch lain sampai digabungkan.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-09/git-branch-list.png`

Keterangan: Hasil `git branch` yang menampilkan branch main dan fitur-navbar.

Instruksi mahasiswa: Jalankan `git branch` dan pastikan kedua branch terdaftar.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-09/git-branch-list.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-09/git-log-fitur.png`

Keterangan: Hasil `git log --oneline` di branch fitur-navbar (menampilkan *commit* tambahan).

Instruksi mahasiswa: Jalankan `git log --oneline` di branch fitur-navbar dan amati *commit* baru.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-09/git-log-fitur.png` tersedia.

9.6 Praktikum 3 — Menggabungkan Branch (Merge)

Langkah Kerja

1. Pastikan Anda berada di branch main:

Listing 9.10: Pindah ke branch main

```
git checkout main
```

Langkah Kerja

1. Gabungkan branch fitur-navbar ke main:

Listing 9.11: Merge branch fitur-navbar

```
git merge fitur-navbar
```

Langkah Kerja

1. Periksa file—navbar.html sekarang ada di main.
2. Periksa riwayat:

Listing 9.12: Riwayat setelah merge

```
git log --oneline
```

Langkah Kerja

1. *Push* main ke GitHub:

Listing 9.13: Push main ke GitHub

```
git push
```

Catatan

Setelah *merge*, branch fitur-navbar bisa dihapus jika tidak diperlukan: `git branch -d fitur-navbar`.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/bab-09/git-merge-success.png

Keterangan: Hasil git merge fitur-navbar yang berhasil.

Instruksi mahasiswa: Jalankan git merge fitur-navbar di branch main dan pastikan tidak ada konflik.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset
assets/images/bab-09/git-merge-success.png tersedia.

9.7 Praktikum 4 — Push Branch ke GitHub

Langkah Kerja

1. Buat branch baru lagi:

Listing 9.14: Membuat branch fitur-kontak

```
git checkout -b fitur-kontak
```

Langkah Kerja

1. Buat file `kontak.html` sederhana dan *commit*:

Listing 9.15: Commit file kontak

```
git add .
git commit -m "Menambahkan halaman kontak"
```

Langkah Kerja

1. *Push* branch baru ke GitHub:

Listing 9.16: Push branch fitur-kontak ke GitHub

```
git push -u origin fitur-kontak
```

Langkah Kerja

1. Buka halaman *repository* di GitHub. Klik dropdown **branch** (biasanya bertuliskan `main`). Anda akan melihat branch `fitur-kontak` di daftar.

Catatan

`-u origin fitur-kontak` membuat branch lokal terhubung ke branch di GitHub.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-09/github-dropdown-branch.png`

Keterangan: Halaman GitHub yang menampilkan dropdown branch (ada `fitur-kontak`).

Instruksi mahasiswa: Buka halaman *repository* di GitHub dan klik dropdown branch untuk melihat daftar branch.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-09/github-dropdown-branch.png` tersedia.

9.8 Praktikum 5 — Membuat Pull Request (PR)

Langkah Kerja

1. Di halaman *repository* GitHub, setelah *push* branch `fitur-kontak`, akan muncul banner kuning “Compare & pull request”. Klik banner tersebut.
2. Jika banner tidak muncul, klik tab **Pull requests**, lalu tombol **New pull request**.
3. Atur:
 - **base:** `main` (tujuan penggabungan).
 - **compare:** `fitur-kontak` (sumber perubahan).

Langkah Kerja

1. Isi judul PR, misalnya: “Menambahkan halaman kontak”.
2. Isi deskripsi PR:

Listing 9.17: Contoh deskripsi PR

Menambahkan halaman kontak sederhana dengan form.

- Form nama, **email**, dan pesan
- Styling dasar dengan CSS

Langkah Kerja

1. Klik **Create pull request**.
2. Amati halaman PR. Anda bisa melihat:
 - **Files changed** — perbedaan kode yang diusulkan.
 - **Commits** — daftar *commit* dalam PR.
 - **Conversation** — diskusi/*review*.

Tips

PR adalah tempat untuk **review** sebelum kode digabung. Rekan tim bisa memberi komentar pada baris kode tertentu.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-09/github-create-pr.png`

Keterangan: Halaman pembuatan *pull request* (base: main, compare: fitur-kontak).

Instruksi mahasiswa: Buka halaman PR di GitHub dan perhatikan perbandingan branch main dengan fitur-kontak.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-09/github-create-pr.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-09/github-pr-page.png`

Keterangan: Halaman PR yang menampilkan tab *Conversation*, *Commits*, dan *Files changed*.

Instruksi mahasiswa: Jelajahi tab-tab di halaman PR untuk memahami alur *review*.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-09/github-pr-page.png` tersedia.

9.9 Praktikum 6 — Merge Pull Request

Langkah Kerja

1. Di halaman PR yang sudah dibuat, klik tombol **Merge pull request**.
2. Klik **Confirm merge**.

3. Klik **Delete branch** untuk membersihkan branch fitur-kontak di GitHub (opsional).
4. Kembali ke terminal VSCode. Ambil perubahan yang sudah di-merge:

Listing 9.18: Ambil perubahan setelah merge PR

```
git checkout main
git pull
```

Langkah Kerja

1. Periksa bahwa kontak.html sekarang ada di main lokal.

Catatan

Alur kolaborasi lengkap:

1. Buat branch fitur.
2. Kerjakan dan *commit*.
3. *Push* branch ke GitHub.
4. Buat PR.
5. Review & merge.
6. *git pull* di lokal.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/bab-09/github-pr-merged.png

Keterangan: Halaman PR setelah di-merge (tombol “Merged”).

Instruksi mahasiswa: Klik **Merge pull request** di GitHub dan amati status PR berubah menjadi “Merged”.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset
assets/images/bab-09/github-pr-merged.png tersedia.

9.10 Praktikum 7 — Menyiapkan GitHub Pages

Langkah Kerja

1. Pastikan Anda berada di branch main dan semua perubahan sudah di-*push*.
2. Buka halaman *repository* di GitHub.
3. Klik tab **Settings**.
4. Di menu kiri, klik **Pages**.
5. Pada bagian **Build and deployment**:
 - **Source:** pilih **Deploy from a branch**.
 - **Branch:** pilih main dan folder / (root).
 - Klik **Save**.

Langkah Kerja

1. Tunggu beberapa saat (1–2 menit) hingga proses *build* selesai.
2. *Refresh* halaman. Akan muncul URL website Anda, misalnya:

Listing 9.19: Format URL GitHub Pages

```
https://USERNAME.github.io/latihan-css/
```

Langkah Kerja

1. Klik URL tersebut atau buka di browser. Website Anda sekarang **live di internet!**

Catatan

URL GitHub Pages mengikuti format `https://USERNAME.github.io/NAMA-REPO/`. Pastikan ada file `index.html` di *root repository* agar website menampilkan halaman utama.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-09/github-pages-settings.png`

Keterangan: Halaman Settings → Pages yang menampilkan konfigurasi *deploy* dari branch main.

Instruksi mahasiswa: Buka tab **Settings** → **Pages** di GitHub dan pastikan branch main terpilih.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-09/github-pages-settings.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-09/github-pages-live.png`

Keterangan: Website yang ter-*deploy* di GitHub Pages (tampil di browser).

Instruksi mahasiswa: Buka URL GitHub Pages di browser dan pastikan website Anda tampil.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-09/github-pages-live.png` tersedia.

9.11 Praktikum 8 — Memperbarui Website di GitHub Pages**Langkah Kerja**

1. Ubah salah satu file di proyek (mis. ganti judul di `index.html`).
2. *Commit* dan *push*:

Listing 9.20: Commit dan push pembaruan

```
git add .
git commit -m "Memperbarui judul halaman"
git push
```

Langkah Kerja

1. Tunggu beberapa saat, lalu *refresh* website Anda. Perubahan otomatis muncul.

Catatan

GitHub Pages otomatis men-*deploy* ulang setiap kali ada *push* ke branch main. Tidak perlu konfigurasi tambahan.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/bab-09/github-pages-mobile.png`

Keterangan: Website di GitHub Pages diakses dari ponsel (tampilan responsif).

Instruksi mahasiswa: Buka URL GitHub Pages di perangkat seluler untuk memverifikasi tampilan responsif.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/bab-09/github-pages-mobile.png` tersedia.

9.12 Latihan Mandiri

Latihan 9.1 Halaman Tentang

- Buat branch baru bernama `fitur-tentang`.
- Buat file `tentang.html` berisi halaman “Tentang” sederhana.
- *Commit* dan *push* branch ke GitHub.
- Buat *pull request* dari `fitur-tentang` ke `main`.
- *Merge* PR tersebut.
- `git pull` di komputer lokal.
- Pastikan GitHub Pages menampilkan halaman terbaru.
- Buka website Anda di ponsel dan pastikan tampil dengan baik.

9.13 Hasil yang Diharapkan

Setelah menyelesaikan semua praktikum, Anda memiliki:

- Kemampuan membuat, berpindah, dan menggabungkan branch.
- Pengalaman membuat dan *merge pull request* di GitHub.
- Website statis yang **ter-deploy** di GitHub Pages dengan URL publik.
- Pemahaman alur kerja kolaborasi: `branch` → `commit` → `push` → `PR` → `merge` → `pull`.
- *Repository* GitHub dengan riwayat *commit* yang rapi dan terstruktur.

Tabel 9.1: Masalah umum *branch*, PR, dan GitHub Pages

Masalah	Penyebab	Solusi
git checkout gagal	Ada perubahan yang belum di- <i>commit</i>	<i>Commit</i> atau <i>stash</i> perubahan dulu: <code>git stash</code>
<i>Merge conflict</i>	Dua branch mengubah file yang sama	Buka file yang konflik, pilih perubahan yang benar, hapus penanda <<<<<<, lalu <i>commit</i>
PR tidak muncul	Branch belum di- <i>push</i>	Jalankan <code>git push -u origin NAMA-BRANCH</code>
GitHub Pages 404	Tidak ada <code>index.html</code> di <i>root</i>	Pastikan file <code>index.html</code> ada di <i>root repository</i>
GitHub Pages belum muncul	Proses <i>build</i> masih berjalan	Tunggu 1–2 menit, lalu <i>refresh</i>
Website tidak update	<i>Push</i> belum dilakukan / <i>build</i> berjalan	Pastikan <code>git push</code> berhasil, tunggu <i>build</i> selesai
URL tidak bisa diakses	<i>Repository private</i>	GitHub Pages hanya untuk <i>repository public</i> (atau akun Pro)
<i>Merge conflict</i> saat <code>git pull</code>	Perubahan lokal dan <i>remote</i> bentrok	<i>Commit</i> perubahan lokal dulu, lalu <code>git pull</code>

9.14 Troubleshooting

9.15 Rangkuman

- **Branch** memungkinkan bekerja pada fitur tanpa mengganggu *main*.
- `git branch` — lihat branch; `git checkout -b nama` — buat & pindah branch.
- `git merge nama-branch` — gabungkan branch ke branch aktif.
- **Pull request** adalah permintaan penggabungan yang bisa direview.
- **GitHub Pages** men-*deploy* website statis secara gratis.
- Alur kolaborasi: *branch* → *commit* → *push* → PR → *merge* → *pull*.
- GitHub Pages otomatis update setiap *push* ke *main*.

Bab 10 JavaScript Dasar: Variabel, Fungsi, Kondisi & DOM Selection

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- menulis skrip JavaScript pertama dan menjalankannya di peramban;
- mendeklarasikan variabel dengan `let` dan `const`, serta membedakan tipe data dasar;
- mendefinisikan dan memanggil fungsi, termasuk *arrow function*;
- menggunakan percabangan `if/else` untuk pengambilan keputusan;
- memilih elemen HTML melalui *DOM selection* (`getElementById`, `querySelector`, dsb.).

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- membuat file `.html` yang mengandung skrip JavaScript;
- berlatih menulis variabel, fungsi, dan kondisi;
- memanipulasi elemen HTML melalui *DOM selection*.

10.1 Persiapan

Yang Anda butuhkan:

- VSCode (sudah terinstal dari minggu sebelumnya).
- File HTML dari minggu-minggu sebelumnya (atau buat file baru `index.html`).
- Peramban (Chrome atau Firefox).

Catatan

Untuk modul ini kita **TIDAK** membutuhkan Node.js. Kita menulis JavaScript langsung di dalam file HTML.

10.2 Menulis Skrip Pertama

Langkah Kerja

1. Buat file baru bernama `minggu10.html` di folder proyek Anda.
2. Isi dengan kode berikut.
3. Buka file ini di peramban (klik kanan di VSCode → **Open with Live Server**).
4. Buka **Console** peramban: tekan F12 → tab **Console**.

Listing 10.1: Skrip JavaScript pertama

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Minggu 10 -- JavaScript Pertama</title>
</head>
<body>
  <h1 id="judul">Halo, Dunia!</h1>
  <p id="paragraf">Ini adalah paragraf pertama saya.</p>

  <script>
    console.log("JavaScript berhasil dimuat!");
    document.getElementById("judul").style.color = "blue";
  </script>
</body>
</html>

```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-10/console-srip-pertama.png

Keterangan: Console peramban menampilkan pesan “JavaScript berhasil dimuat!”

Instruksi mahasiswa: Buka file minggu10.html di peramban, lalu buka Console (F12) dan bandingkan hasilnya.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset
assets/images/minggu-10/console-srip-pertama.png tersedia.

10.2.1 Yang Terjadi

- `<script>` di dalam HTML adalah tempat menulis kode JavaScript.
- `console.log(...)` mencetak pesan ke Console peramban—sangat berguna untuk *debugging*.
- `document.getElementById("judul")` mencari elemen HTML dengan `id="judul"`.
- `.style.color = "blue"` mengubah warna teks menjadi biru.

10.3 Variabel & Tipe Data

10.3.1 Deklarasi Variabel

Variabel adalah **wadah** untuk menyimpan data. Di JavaScript modern ada tiga cara mendeklarasikan variabel:

Tabel 10.1: Perbandingan kata kunci deklarasi variabel

Kata Kunci	Bisa Diubah?	Kapan Dipakai
<code>let</code>	Ya	Nilai yang mungkin berubah
<code>const</code>	Tidak	Nilai yang tetap (konstanta)
<code>var</code>	Ya	HINDARI—cara lama, bermasalah pada <i>scope</i>

Tips

Selalu pakai `const`. Ganti ke `let` hanya jika nilai perlu diubah. Jangan pakai `var`.

Langkah Kerja

1. Tambahkan script berikut **sebelum** tag `</script>` yang sudah ada.
2. *Refresh* halaman dan lihat hasilnya di Console.

Listing 10.2: Deklarasi variabel dan perubahan nilai

```
// Variabel dengan const (tidak bisa diubah)
const nama = "Budi";
const umur = 19;

// Variabel dengan let (bisa diubah)
let IPK = 3.5;

console.log("Nama:", nama);
console.log("Umur:", umur);
console.log("IPK:", IPK);

// Mengubah nilai let
IPK = 3.8;
console.log("IPK setelah update:", IPK);
```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-10/console-variabel.png`

Keterangan: Console menampilkan output nama, umur, IPK, dan IPK setelah *update*.

Instruksi mahasiswa: Bandingkan output Console Anda dengan contoh di atas.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-10/console-variabel.png` tersedia.

10.3.2 Tipe Data Dasar**Listing 10.3:** Tipe data dasar JavaScript

```
// String (teks) -- pakai tanda kutip
const pesan = "Halo Dunia";

// Number (angka) -- tanpa tanda kutip
const nilai = 100;

// Boolean (benar/salah)
const lulus = true;
```

```
// Array (daftar)
const buku = ["HTML", "CSS", "JavaScript"];

// Object (kumpulan properti)
const mahasiswa = {
  nama: "Budi",
  nim: "12345",
  jurusan: "Informatika"
};

console.log(pesan);           // "Halo Dunia"
console.log(nilai);           // 100
console.log(lulus);           // true
console.log(buku[0]);         // "HTML" (indeks mulai dari 0)
console.log(mahasiswa.nama); // "Budi"
```

10.4 Fungsi

Fungsi adalah **blok kode yang bisa dipanggil berulang kali**. Bayangkan seperti resep masakan—Anda menulis resepnya sekali, lalu bisa dipakai kapan saja.

Langkah Kerja

1. Tambahkan kode berikut ke dalam tag `<script>`.
2. *Refresh* halaman, lihat hasil di Console.

Listing 10.4: Fungsi deklarasi, nilai kembali, dan *arrow function*

```
// Fungsi sederhana -- deklarasi
function sapa(nama) {
  console.log("Halo, " + nama + "! Selamat belajar JavaScript.");
}

// Memanggil fungsi
sapa("Budi");
sapa("Ani");
sapa("Andi");

// Fungsi dengan nilai kembali (return)
function hitungLuas(panjang, lebar) {
  const luas = panjang * lebar;
  return luas;
}

const hasil = hitungLuas(10, 5);
console.log("Luas persegi panjang:", hasil);

// Fungsi panah (arrow function) -- cara modern
const kalikan = (a, b) => {
```

```

    return a * b;
};

console.log("3x4=", kalikan(3, 4));

```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-10/console-fungsi.png

Keterangan: Console menampilkan output pemanggilan fungsi `sapa()`, `hitungLuas()`, dan `kalikan()`.

Instruksi mahasiswa: Bandingkan output Console Anda dengan contoh di atas.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-10/console-fungsi.png` tersedia.

- `function sapa(nama)` mendefinisikan fungsi bernama `sapa` yang menerima satu parameter `nama`.
- `sapa("Budi")` memanggil fungsi dengan nilai `"Budi"` untuk parameter `nama`.
- `return luas` mengembalikan nilai hasil perhitungan agar bisa digunakan di luar fungsi.
- *Arrow function* `=>` adalah cara menulis fungsi yang lebih singkat.

10.5 Kondisi (If/Else)

Kondisi membuat program bisa **mengambil keputusan**—seperti “Jika hujan, bawa payung; jika tidak, bawa topi.”

Langkah Kerja

1. Tambahkan kode berikut ke dalam tag `<script>`.
2. *Refresh* dan lihat output di Console.

Listing 10.5: Percabangan if/else dan operator perbandingan

```

// Kondisi sederhana
const suhu = 32;

if (suhu > 30) {
    console.log("Panas! Minum air yang banyak.");
} else if (suhu >= 20) {
    console.log("Nyaman. Cukup untuk belajar.");
} else {
    console.log("Dingin! Pakai jaket.");
}

// Kondisi dengan string
const namaMahasiswa = "Budi";

if (namaMahasiswa === "Budi") {

```

```

    console.log("Halo_Budi,_selamat_datang!");
} else {
    console.log("Siapa_kamu?");
}

// Operator perbandingan
// === sama dengan (nilai DAN tipe sama)
// !== tidak sama dengan
// >  lebih besar dari
// <  lebih kecil dari
// >= lebih besar atau sama dengan
// <= lebih kecil atau sama dengan

console.log(5 === 5); // true
console.log(5 === "5"); // false (angka vs string)
console.log(5 == "5"); // true (hati-hati, bisa menyesatkan)

```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-10/console-kondisi.png

Keterangan: Console menampilkan output kondisi if/else dan perbandingan.

Instruksi mahasiswa: Bandingkan output Console Anda dengan contoh di atas.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset
assets/images/minggu-10/console-kondisi.png tersedia.

10.6 DOM Selection

DOM (Document Object Model) adalah representasi struktur HTML sebagai “pohon” yang bisa diakses oleh JavaScript. *DOM Selection* adalah cara JavaScript **mencari elemen** di halaman HTML.

Langkah Kerja

1. Ganti isi file minggu10.html dengan versi lengkap berikut.
2. Buka dengan Live Server dan lihat hasilnya.
3. Buka Console (F12 → Console) untuk melihat semua *log*.

Listing 10.6: Demonstrasi DOM Selection

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,_initial-scale=1.0">
  <title>Minggu 10 -- DOM Selection</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; }

```

```

        .kotak { border: 2px solid #333; padding: 10px;
                margin: 10px 0; border-radius: 5px; }
        .highlight { background-color: yellow; }
</style>
</head>
<body>
    <h1 id="judul">Belajar DOM Selection</h1>
    <p class="teks">Paragraf pertama -- class="teks"</p>
    <p class="teks">Paragraf kedua -- class="teks"</p>
    <p id="paragraf-khusus">Paragraf khusus dengan ID</p>

    <div class="kotak" id="kotak1">Kotak 1</div>
    <div class="kotak" id="kotak2">Kotak 2</div>
    <div class="kotak" id="kotak3">Kotak 3</div>

    <ul id="daftar-mahasiswa">
        <li>Budi</li>
        <li>Ani</li>
        <li>Andi</li>
        <li>Sari</li>
    </ul>

    <script>
        // Cara 1: getElementById
        const judul = document.getElementById("judul");
        console.log("Judul:", judul.textContent);
        judul.style.color = "purple";

        // Cara 2: querySelector
        const paragrafKhusus =
            document.querySelector("#paragraf-khusus");
        console.log("Paragraf_khusus:", paragrafKhusus.textContent);

        // Cara 3: querySelectorAll
        const semuaTeks = document.querySelectorAll(".teks");
        console.log("Jumlah_kotak_khusus:", semuaTeks.length);
        semuaTeks.forEach((el, index) => {
            console.log(`Teks ${index + 1}:`, el.textContent);
        });

        // Cara 4: getElementsByClassName
        const semuaKotak =
            document.getElementsByClassName("kotak");
        console.log("Jumlah_kotak:", semuaKotak.length);
        for (let i = 0; i < semuaKotak.length; i++) {
            semuaKotak[i].style.backgroundColor = "lightblue";
            semuaKotak[i].style.padding = "15px";
        }
    </script>

```

```
// Cara 5: selector campuran
const itemPertama = document.querySelector(
  "#daftar-mahasiswa_li:first-child"
);
console.log("Mahasiswa pertama:", itemPertama.textContent);
itemPertama.style.fontWeight = "bold";
itemPertama.style.color = "red";
</script>
</body>
</html>
```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-10/dom-selection-halaman.png

Keterangan: Halaman web setelah dieksekusi: judul ungu, kotak biru, item pertama daftar mahasiswa berwarna merah tebal.

Instruksi mahasiswa: Buka minggu10.html di peramban dan bandingkan tampilan Anda.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan \includegraphics setelah aset
assets/images/minggu-10/dom-selection-halaman.png tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-10/dom-selection-console.png

Keterangan: Console menampilkan semua *log* DOM selection.

Instruksi mahasiswa: Buka Console (F12) dan pastikan semua *log* muncul.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan \includegraphics setelah aset
assets/images/minggu-10/dom-selection-console.png tersedia.

10.6.1 Ringkasan Metode DOM Selection

Tabel 10.2: Metode DOM Selection

Metode	Mencari	Mengembalikan
getElementById("id")	1 elemen	Element
querySelector("sel")	1 elemen (pertama cocok)	Element
querySelectorAll("sel")	Semua elemen	NodeList
getElementsByClassName("cls")	Semua elemen	HTMLCollection

Tips

querySelector dan querySelectorAll paling fleksibel—bisa pakai semua *selector* CSS (id, class, tag, attribute, bahkan :nth-child).

10.7 Latihan

Latihan 10.1 Variabel dan Fungsi

Buat file baru latihan10-1.html dan buatlah:

1. Variabel `const` untuk menyimpan nama lengkap Anda dan NIM.
2. Variabel `let` untuk menyimpan nilai ujian (angka).
3. Fungsi `beriNilai(skor)` yang mencetak ke Console:
 - Jika skor ≥ 85 : "Excellent! Nilai Anda: [skor]"
 - Jika skor ≥ 70 : "Bagus! Nilai Anda: [skor]"
 - Jika skor < 70 : "Perlu belajar lagi. Nilai Anda: [skor]"
4. Panggil fungsi tersebut dengan tiga nilai berbeda.

Output yang diharapkan di Console:

```
Excellent! Nilai Anda: 90
Bagus! Nilai Anda: 75
Perlu belajar lagi. Nilai Anda: 60
```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-10/latihan1-expected.png`

Keterangan: Console menampilkan output pencarian nilai sesuai skor.

Instruksi mahasiswa: Jalankan file latihan10-1.html di peramban dan pastikan output sesuai.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-10/latihan1-expected.png` tersedia.

Latihan 10.2 DOM Selection

Buat file baru latihan10-2.html dengan HTML berikut:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Latihan 10-2</title>
</head>
<body>
```

```

<h1 id="sapa">Selamat Datang</h1>
<p class="info">Mata kuliah: Desain Web</p>
<p class="info">Dosen: [Nama Dosen]</p>
<p class="info">Hari: [Hari Kuliah]</p>
<div class="kartu">Kartu Mahasiswa</div>
<div class="kartu">Kartu UKT</div>
<div class="kartu">Kartu Perpustakaan</div>
</body>
</html>

```

Tugas JavaScript:

1. Ambil elemen #sapa dan ubah teksnya menjadi nama Anda.
2. Ambil semua elemen .info dan ubah warna setiap paragraf menjadi biru.
3. Ambil semua elemen .kartu dan tambahkan border 1px solid gray pada setiap kartu.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-10/latihan2-expected.png

Keterangan: Halaman setelah JavaScript berjalan—judul berganti nama, paragraf info biru, kartu ada border.

Instruksi mahasiswa: Jalankan file latihan10-2.html di peramban dan pastikan tampilan sesuai.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan \includegraphics setelah aset assets/images/minggu-10/latihan2-expected.png tersedia.

Latihan 10.3 Tantangan — Opsional

Buat file latihan10-3.html yang berisi:

- Sebuah *array* berisi minimal 5 nama mahasiswa.
- Fungsi cariMahasiswa(namaArray, namaDicari) yang mencetak ke Console:
 - "[nama] ditemukan di indeks [indeks]!" jika ketemu.
 - "[nama] tidak ditemukan di daftar." jika tidak ketemu.
- Panggil fungsi dengan nama yang ada di *array* dan nama yang tidak ada.

Output yang diharapkan di Console:

```

Budi ditemukan di indeks 0!
Siti tidak ditemukan di daftar.

```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-10/latihan3-expected.png`

Keterangan: Console menampilkan hasil pencarian nama dalam *array*.

Instruksi mahasiswa: Jalankan file `latihan10-3.html` di peramban.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-10/latihan3-expected.png` tersedia.

10.8 Troubleshooting

Peringatan

JavaScript tidak berjalan / Console kosong — pastikan `<script>` berada **sebelum** `</body>`, perhatikan huruf besar-kecil nama fungsi/variabel, dan tekan `Ctrl+S` sebelum *refresh*.

Catatan

“Uncaught TypeError: Cannot read properties of null” — elemen belum ada di DOM saat script dijalankan. Pindahkan script ke akhir `<body>` atau bungkus dengan `DOMContentLoaded`.

Listing 10.7: Penanganan elemen belum tersedia

```
// Salah -- elemen belum ada di DOM
document.getElementById("judul").style.color = "red";

// Benar -- pindahkan script ke akhir body
// atau pakai DOMContentLoaded
document.addEventListener("DOMContentLoaded", function() {
  document.getElementById("judul").style.color = "red";
});
```

Tips

Selector tidak menemukan elemen — buka DevTools → tab Elements → periksa apakah elemen yang dicari benar-benar ada di DOM. Pastikan `#` untuk ID dan `.` untuk *class*.

Bab 11 JavaScript: Event Handling & Manipulasi DOM

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- merespons interaksi pengguna (klik, input, *submit*) menggunakan *event handling*;
- membedakan penggunaan *onclick* dan *addEventListener*;
- memanipulasi konten DOM (*textContent*, *innerHTML*, *style*, *classList*);
- membuat dan menghapus elemen HTML secara dinamis.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- membuat halaman web interaktif dengan tombol dan form;
- membangun daftar tugas yang bisa ditambah/dihapus;
- menerapkan *toggle menu mobile* sebagai mini proyek.

Catatan

Pengingat cepat dari Minggu 10: *getElementById*, *querySelector*, *querySelectorAll*, *let*, *const*, *if/else*, *fungsi*.

11.1 Event Handling: onclick & addEventListener

Event adalah sesuatu yang terjadi pada halaman web—misal: pengguna mengklik tombol, mengetik di *input*, atau menggerakkan *mouse*. Kita bisa **mendengarkan** event ini dan menjalankan JavaScript sebagai respons.

Langkah Kerja

1. Buat file baru *minggu11.html* dengan kode berikut.
2. Buka dengan Live Server, klik kedua tombol, dan amati perubahan.

Listing 11.1: Event handling: onclick dan addEventListener

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Minggu 11 -- Event Handling</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; }
    button { padding: 10px 20px; font-size: 16px;
             cursor: pointer; margin: 5px; }
    .aktif { background-color: #4CAF50; color: white; }
    .output { background-color: #f0f0f0; padding: 15px;
             margin: 10px 0; border-radius: 5px; }
```

```

        min-height: 30px; }
    </style>
</head>
<body>
    <h1>Event Handling</h1>

    <!-- Cara 1: onclick langsung di HTML -->
    <h2>Cara 1: onclick di HTML</h2>
    <button onclick="sapa()">Klik Saya!</button>

    <!-- Cara 2: addEventListener di JavaScript -->
    <h2>Cara 2: addEventListener</h2>
    <button id="tombol-hitung">Hitung Klik</button>
    <span id="tampil-klik">0</span> kali diklik

    <div class="output" id="area-output">
        Output akan muncul di sini...
    </div>

    <script>
        // Cara 1: onclick
        function sapa() {
            document.getElementById("area-output").textContent =
                "Halo! Anda baru saja mengklik tombol sapa.";
        }

        // Cara 2: addEventListener (lebih disarankan)
        let jumlahKlik = 0;
        document.getElementById("tombol-hitung")
            .addEventListener("click", function() {
                jumlahKlik++;
                document.getElementById("tampil-klik")
                    .textContent = jumlahKlik;

                if (jumlahKlik >= 5) {
                    document.getElementById("area-output")
                        .textContent = "Wah, Anda sudah mengklik "
                            + jumlahKlik + " kali!";
                } else {
                    document.getElementById("area-output")
                        .textContent = "Klik ke-" + jumlahKlik
                            + ". Coba terus!";
                }
            });
    </script>
</body>
</html>

```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-11/event-basic.png`

Keterangan: Halaman dengan tombol “Hitung Klik” yang sudah diklik beberapa kali dan angka penghitung.

Instruksi mahasiswa: Buka `minggu11.html` di peramban, klik tombol beberapa kali, dan bandingkan.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-11/event-basic.png` tersedia.

11.1.1 Mengapa `addEventListener` Lebih Baik?

Tabel 11.1: Perbandingan `onclick` dan `addEventListener`

Aspek	<code>onclick</code>	<code>addEventListener</code>
Jumlah <i>handler</i>	Hanya 1	Banyak sekaligus
Pemisahan kode	JS tercampur di HTML	JS terpisah di <code><script></code>
Fleksibilitas	Terbatas	Bisa <i>add/remove listener</i>

Tips

Gunakan `addEventListener` untuk semua kebutuhan *event handling*.

11.1.2 Event Populer

Tabel 11.2: Event yang sering digunakan

Event	Kapan Terjadi
<code>click</code>	Elemen diklik
<code>dblclick</code>	Elemen diklik dua kali
<code>mouseover</code>	<i>Mouse</i> masuk ke elemen
<code>mouseout</code>	<i>Mouse</i> keluar dari elemen
<code>keydown</code>	Tombol <i>keyboard</i> ditekan
<code>input</code>	Nilai <i>input</i> berubah
<code>submit</code>	Form dikirim
<code>load</code>	Halaman selesai dimuat

11.2 Event pada Form & Input

Langkah Kerja

1. Tambahkan kode berikut **sebelum** `</body>` pada `minggu11.html` (ganti atau tambahkan *script* baru).
2. Coba isi form dan *submit*. Amati: jika field kosong → muncul pesan error; saat mengetik → muncul penghitung karakter *real-time*; setelah *submit* berhasil → form terkosongkan.

Listing 11.2: Form sederhana dengan validasi dan penghitung karakter

```

<h2>Form Sederhana</h2>
<form id="form-mahasiswa">
  <label for="input-nama">Nama:</label><br>
  <input type="text" id="input-nama"
    placeholder="Masukkan nama"><br><br>
  <label for="input-email">Email:</label><br>
  <input type="email" id="input-email"
    placeholder="contoh@email.com"><br><br>
  <button type="submit">Kirim</button>
</form>

<div class="output" id="hasil-form">
  Data form akan muncul di sini...
</div>

<script>
  // Event pada form submit
  document.getElementById("form-mahasiswa")
    .addEventListener("submit", function(event) {
      event.preventDefault();

      const nama = document.getElementById("input-nama").value;
      const email = document.getElementById("input-email").value;

      if (nama === "" || email === "") {
        document.getElementById("hasil-form").textContent =
          "Semua field wajib diisi!";
        document.getElementById("hasil-form")
          .style.backgroundColor = "#ffe0e0";
        return;
      }

      document.getElementById("hasil-form").textContent =
        "Data diterima -- Nama: " + nama
          + ", Email: " + email;
      document.getElementById("hasil-form")
        .style.backgroundColor = "#e0ffe0";

      // Kosongkan form
      document.getElementById("input-nama").value = "";
      document.getElementById("input-email").value = "";
    });

  // Event pada input -- real-time feedback
  document.getElementById("input-nama")
    .addEventListener("input", function(event) {
      const panjang = event.target.value.length;

```

```

    if (panjang > 0) {
        document.getElementById("hasil-form").textContent =
            "Anda_mengetik_" + panjang + "_karakter...";
        document.getElementById("hasil-form")
            .style.backgroundColor = "#f0f0f0";
    } else {
        document.getElementById("hasil-form").textContent =
            "Data_form_akan_muncul_di_sini...";
    }
});
</script>

```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-11/event-form.png

Keterangan: Form yang sudah diisi dengan data valid dan pesan sukses di area *output*.

Instruksi mahasiswa: Isi form dengan nama dan email yang valid, lalu klik “Kirim”.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset assets/images/minggu-11/event-form.png tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-11/event-form-validasi.png

Keterangan: Form yang dikosongkan lalu disubmit—muncul pesan validasi error.

Instruksi mahasiswa: Kosongkan form lalu klik “Kirim” tanpa mengisi apapun.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset assets/images/minggu-11/event-form-validasi.png tersedia.

11.3 Manipulasi DOM: Mengubah Konten

Langkah Kerja

1. Tambahkan kode berikut ke dalam `<body>` minggu11.html.
2. Klik setiap tombol satu per satu dan amati perubahan.

Listing 11.3: Empat cara mengubah konten DOM

```

<h2>Mengubah Konten DOM</h2>
<button id="btn-ubah-teks">Ubah Teks</button>
<button id="btn-ubah-html">Ubah HTML</button>
<button id="btn-ubah-style">Ubah Gaya</button>

```

```

<button id="btn-ubah-class">Toggle Class</button>

<div id="target-konten" class="output">
  Ini adalah elemen yang akan diubah.
</div>

<script>
  // 1. Mengubah teks (textContent)
  document.getElementById("btn-ubah-teks")
    .addEventListener("click", function() {
      document.getElementById("target-konten")
        .textContent = "Teks_sudah_berubah!";
    });

  // 2. Mengubah HTML (innerHTML)
  document.getElementById("btn-ubah-html")
    .addEventListener("click", function() {
      document.getElementById("target-konten")
        .innerHTML =
          "<strong>Teks_tebal</strong>_dan_"
          + "<em>text_miring</em>_"
          + "--_semua_dari_innerHTML!";
    });

  // 3. Mengubah gaya langsung (style)
  document.getElementById("btn-ubah-style")
    .addEventListener("click", function() {
      const el = document.getElementById("target-konten");
      el.style.backgroundColor = "#4CAF50";
      el.style.color = "white";
      el.style.padding = "20px";
      el.style.borderRadius = "10px";
      el.style.fontSize = "18px";
    });

  // 4. Toggle class (cara paling rapi untuk styling)
  document.getElementById("btn-ubah-class")
    .addEventListener("click", function() {
      document.getElementById("target-konten")
        .classList.toggle("aktif");
    });
</script>

```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-11/manipulasi-konten.png

Keterangan: Area *output* setelah diklik—tampilan berubah sesuai tombol yang ditekan.

Instruksi mahasiswa: Klik setiap tombol satu per satu dan bandingkan perubahan tampilan.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-11/manipulasi-konten.png` tersedia.

Tabel 11.3: Ringkasan cara mengubah DOM

Metode	Fungsi	Contoh
<code>textContent</code>	Mengubah teks mentah	<code>el.textContent = "Halo"</code>
<code>innerHTML</code>	Mengubah isi HTML	<code>el.innerHTML = "Halo"</code>
<code>style.prop</code>	Mengubah gaya CSS	<code>el.style.color = "red"</code>
<code>classList.toggle()</code>	Tambah/hapus class	<code>el.classList.toggle("aktif")</code>

Peringatan

Hindari `innerHTML` untuk konten dari pengguna (bisa menyebabkan **XSS**). Untuk teks biasa, gunakan `textContent`.

11.4 Manipulasi DOM: Membuat & Menghapus Elemen

Langkah Kerja

1. Tambahkan kode berikut ke dalam `<body>` `minggu11.html`.
2. Coba tambahkan beberapa item, hapus satu per satu, dan hapus semua.

Listing 11.4: Membuat dan menghapus elemen secara dinamis

```
<h2>Membuat & Menghapus Elemen</h2>
<input type="text" id="input-item"
  placeholder="Ketik item baru...">
<button id="btn-tambah">Tambah</button>
<button id="btn-hapus-terakhir">Hapus Terakhir</button>
<button id="btn-hapus-semua">Hapus Semua</button>

<ul id="daftar-item" style="list-style: none; padding: 0;">
</ul>

<script>
  function buatItem(teksItem) {
    const li = document.createElement("li");
    li.textContent = teksItem;
    li.style.padding = "8px 12px";
    li.style.margin = "5px 0";
    li.style.backgroundColor = "#e8f4f8";
    li.style.borderRadius = "4px";
```

```

    li.style.display = "flex";
    li.style.justifyContent = "space-between";

    const tombolHapus = document.createElement("button");
    tombolHapus.textContent = "x";
    tombolHapus.style.backgroundColor = "#ff6b6b";
    tombolHapus.style.color = "white";
    tombolHapus.style.border = "none";
    tombolHapus.style.padding = "2px 8px";
    tombolHapus.style.cursor = "pointer";
    tombolHapus.style.borderRadius = "3px";

    tombolHapus.addEventListener("click", function() {
        li.remove();
    });

    li.appendChild(tombolHapus);
    return li;
}

// Tambah item
document.getElementById("btn-tambah")
    .addEventListener("click", function() {
        const input = document.getElementById("input-item");
        const teks = input.value.trim();
        if (teks === "") {
            alert("Masukkan teks terlebih dahulu!");
            return;
        }
        document.getElementById("daftar-item")
            .appendChild(buatItem(teks));
        input.value = "";
        input.focus();
    });

// Tambah item dengan Enter
document.getElementById("input-item")
    .addEventListener("keydown", function(event) {
        if (event.key === "Enter") {
            document.getElementById("btn-tambah").click();
        }
    });

// Hapus item terakhir
document.getElementById("btn-hapus-terakhir")
    .addEventListener("click", function() {
        const daftar = document.getElementById("daftar-item");
        if (daftar.lastElementChild) {

```

```

        daftar.lastElementChild.remove();
    }
});

// Hapus semua item
document.getElementById("btn-hapus-semua")
    .addEventListener("click", function() {
        document.getElementById("daftar-item")
            .innerHTML = "";
    });
</script>

```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-11/manipulasi-elemen.png

Keterangan: Daftar item yang sudah ditambahkan beberapa item, dengan tombol hapus per item.

Instruksi mahasiswa: Buka minggu11.html di peramban, tambahkan item baru, lalu hapus satu per satu.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset
assets/images/minggu-11/manipulasi-elemen.png tersedia.

11.4.1 Cara Membuat & Menambah Elemen

Listing 11.5: Pola dasar pembuatan elemen DOM

```

// 1. Buat elemen baru
const elemenBaru = document.createElement("div");

// 2. Isi konten
elemenBaru.textContent = "Ini elemen baru";
elemenBaru.className = "kartu";

// 3. Masukkan ke DOM (sebagai anak dari elemen lain)
document.getElementById("induk").appendChild(elemenBaru);

// Atau sisipkan di posisi tertentu
document.getElementById("induk")
    .insertBefore(elemenBaru, referensiElemen);

// Hapus elemen
elemenBaru.remove();

```

11.5 Mini Proyek: Toggle Menu Mobile

Membuat tombol *hamburger* sederhana yang membuka/menutup menu navigasi—seperti yang sering Anda lihat di situs web pada tampilan *mobile*.

Langkah Kerja

1. Buat file baru `mini-proyek-menu.html`.
2. Buka dengan Live Server, lalu **resize browser ke bawah 600px** untuk melihat tampilan *mobile*.
3. Klik tombol `≡` dan amati menu terbuka. Klik `×` untuk menutup.

Listing 11.6: Mini proyek: toggle menu mobile

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Mini Proyek -- Toggle Menu</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: Arial, sans-serif; }

    .navbar {
      background-color: #333;
      padding: 15px 20px;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }
    .navbar .logo {
      color: white; font-size: 20px; font-weight: bold;
    }
    .hamburger {
      display: none; background: none; border: none;
      color: white; font-size: 24px; cursor: pointer;
      padding: 5px 10px;
    }
    .nav-menu {
      list-style: none;
      display: flex; gap: 15px;
    }
    .nav-menu a {
      color: white; text-decoration: none;
      padding: 5px 10px; border-radius: 3px;
    }
    .nav-menu a:hover { background-color: #555; }

    @media (max-width: 600px) {
      .hamburger { display: block; }
```

```

        .nav-menu {
            display: none;
            flex-direction: vertical;
            position: absolute; top: 60px;
            left: 0; right: 0;
            background-color: #444;
            padding: 10px 20px;
        }
        .nav-menu.tampil { display: flex; }
    }

    .konten { padding: 20px; }
</style>
</head>
<body>
    <nav class="navbar">
        <div class="logo">MyWeb</div>
        <button class="hamburger" id="hamburger">
            &#9776;
        </button>
        <ul class="nav-menu" id="nav-menu">
            <li><a href="#">Beranda</a></li>
            <li><a href="#">Tentang</a></li>
            <li><a href="#">Portofolio</a></li>
            <li><a href="#">Kontak</a></li>
        </ul>
    </nav>

    <div class="konten">
        <h2>Selamat Datang!</h2>
        <p>Coba ubah ukuran browser ke bawah 600px
            (atau buka di hp) untuk melihat menu
            hamburger muncul.</p>
    </div>

    <script>
        const tombolHamburger =
            document.getElementById("hamburger");
        const navMenu = document.getElementById("nav-menu");

        tombolHamburger.addEventListener("click", function() {
            navMenu.classList.toggle("tampil");
            if (navMenu.classList.contains("tampil")) {
                tombolHamburger.textContent = "\u2715";
            } else {
                tombolHamburger.textContent = "\u2630";
            }
        });
    </script>

```

```

        document.addEventListener("click", function(event) {
            if (!navMenu.contains(event.target)
                && !tombolHamburger.contains(event.target)) {
                navMenu.classList.remove("tampil");
                tombolHamburger.textContent = "\u2630";
            }
        });
    </script>
</body>
</html>

```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-11/miniproject-desktop.png

Keterangan: Tampilan desktop—menu horizontal, tombol *hamburger* tersembunyi.

Instruksi mahasiswa: Buka mini-proyek-menu.html di peramban dengan lebar penuh.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan \includegraphics setelah aset
assets/images/minggu-11/miniproject-desktop.png tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-11/miniproject-mobile.png

Keterangan: Tampilan *mobile* (browser < 600 px)—tombol *hamburger* terlihat, menu terbuka ke bawah.

Instruksi mahasiswa: Resize browser ke bawah 600 px, lalu klik tombol ≡.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan \includegraphics setelah aset
assets/images/minggu-11/miniproject-mobile.png tersedia.

11.6 Latihan

Latihan 11.1 Kalkulator Sederhana

Buat file latihan11-1.html yang berisi:

1. Dua *input* angka (<input type="number">).
2. Empat tombol: Tambah, Kurang, Kalikan, Bagi.
3. Area *output* untuk menampilkan hasil.
4. JavaScript yang menghitung sesuai tombol yang diklik.

Output yang diharapkan:

- Jika *input* 10 dan 5, klik “Kalikan” → *output* menampilkan Hasil: 50.
- Jika *input* 10 dan 0, klik “Bagi” → *output* menampilkan Error: Tidak bisa dibagi nol!.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-11/latihan1-kalkulator.png`

Keterangan: Kalkulator berfungsi dengan hasil perhitungan.

Instruksi mahasiswa: Buka `latihan11-1.html` di peramban, masukkan dua angka, dan klik salah satu tombol.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-11/latihan1-kalkulator.png` tersedia.

Latihan 11.2 Daftar Tugas dengan Checkbox

Buat file `latihan11-2.html` yang berisi:

1. *Input* untuk menambah tugas baru + tombol “Tambah”.
2. Setiap tugas memiliki *checkbox* dan tombol “Hapus”.
3. Saat *checkbox* dicentang, teks tugas dicoret (*strikethrough*).
4. Total tugas dan tugas selesai ditampilkan di bawah daftar.

Output yang diharapkan:

- Menambah 3 tugas → Total: 3, Selesai: 0.
- Mencentang 1 tugas → Total: 3, Selesai: 1, teks tugas tercoret.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-11/latihan2-tugas.png`

Keterangan: Daftar tugas dengan satu tugas tercentang dan *counter* yang benar.

Instruksi mahasiswa: Buka `latihan11-2.html` di peramban, tambahkan 3 tugas, centang salah satu.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-11/latihan2-tugas.png` tersedia.

Latihan 11.3 Light/Dark Mode Toggle — Opsional

Buat file `latihan11-3.html` yang berisi:

1. Tombol “Dark Mode” / “Light Mode”.
2. Saat diklik, halaman berganti antara *background* terang dan gelap.
3. Preferensi diingat selama sesi (boleh pakai `localStorage` jika sudah tahu, atau cukup *toggle* CSS

class).

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-11/latihan3-darkmode.png`

Keterangan: Halaman dalam mode gelap dan mode terang.

Instruksi mahasiswa: Buka `latihan11-3.html` di peramban dan klik tombol *toggle*.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-11/latihan3-darkmode.png` tersedia.

11.7 Troubleshooting

Peringatan

Event tidak merespons — pastikan elemen sudah ada di DOM saat script dijalankan (pindahkan `<script>` ke akhir `<body>` atau pakai `DOMContentLoaded`). Periksa *selector* ID/class di DevTools dan ejaan nama *event* (mis. "click", bukan "clicl").

Catatan

Form refresh saat dikirim — Anda lupa menambahkan `event.preventDefault()`. Bungkus *handler* submit dengan:

```
form.addEventListener("submit", function(event) {
  event.preventDefault();
  // proses data...
});
```

Catatan

Elemen baru tidak muncul di DOM — pastikan Anda memanggil `appendChild` atau `prepend` setelah `createElement`. Tanpa panggilan tersebut, elemen tidak akan muncul.

Tips

`classList.toggle` tidak berfungsi — periksa ejaan nama *class* (`classList` bersifat *case-sensitive*). Juga pastikan pengecekan `event.target` pada toggle menu benar menggunakan `contains()`.

Bab 12 Pengenalan Tailwind CSS

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- menjelaskan perbedaan antara CSS tradisional dan pendekatan *utility-first*;
- menyiapkan Tailwind CSS menggunakan *CDN* di proyek HTML;
- menggunakan *utility class* dasar untuk tipografi, *spacing*, warna, dan *border*;
- membuat layout *responsive* menggunakan *Flexbox* dan *Grid* Tailwind.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- mengonversi halaman CSS manual menjadi Tailwind CSS;
- bereksperimen dengan *utility class* di *playground*;
- membangun layout *responsive* dengan *Flexbox* dan *Grid*.

Catatan

Tailwind CSS **bukan bahasa pemrograman**—ini adalah kumpulan **class CSS pendek** yang sudah ditulis untuk Anda. Alih-alih menulis `font-size: 18px; font-weight: bold;`, Anda cukup menulis `class="text-lg font-bold"`.

12.1 Konsep: Apa Itu Tailwind CSS?

12.1.1 CSS Tradisional vs Tailwind CSS

CSS Tradisional:

Listing 12.1: Penulisan CSS tradisional

```
/* Di file terpisah (.css) */
.card {
  background-color: white;
  border-radius: 8px;
  padding: 16px;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

.card-title {
  font-size: 1.25rem;
  font-weight: bold;
  color: #333;
  margin-bottom: 8px;
}
```

Dengan Tailwind:

Listing 12.2: Penulisan dengan Tailwind CSS

```

<!-- Tidak perlu file CSS terpisah -->
<div class="bg-white rounded-lg p-4 shadow-md">
  <h2 class="text-xl font-bold text-gray-800 mb-2">
    Judul Card
  </h2>
</div>

```

12.1.2 Mengapa Tailwind?**Tabel 12.1:** Kelebihan Tailwind CSS

Kelebihan	Penjelasan
Cepat	Tidak perlu bolak-balik ke file CSS
Konsisten	<i>Design system</i> built-in (spacing, warna, ukuran terstandarisasi)
Ringan (<i>production</i>)	Hanya class yang dipakai yang masuk ke CSS final
Populer	Banyak digunakan di industri

Catatan

CSS tradisional tetap berguna untuk memahami dasar *styling* dan untuk *selector* kompleks yang sulit dijangkau *utility class*. Tailwind **bukan pengganti** CSS—ini **pendekatan berbeda** untuk menulis CSS.

12.2 Setup Tailwind di Proyek

Cara termudah adalah menggunakan **CDN**—tanpa instalasi, cocok untuk tujuan belajar.

Langkah Kerja

1. Buat file baru minggu12.html.
2. Buka dengan Live Server. Jika halaman berlatar abu-abu dengan judul besar, Tailwind sudah berhasil dimuat!

Listing 12.3: Setup Tailwind CSS via CDN

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Minggu 12 -- Tailwind CSS</title>
  <!-- Tailwind CSS via CDN -->
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 text-gray-800">
  <h1 class="text-3xl font-bold text-center py-10">
    Halo, Tailwind CSS!
  </h1>

```

```
</body>
</html>
```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-12/tailwind-setup.png`

Keterangan: Halaman dengan latar abu-abu terang dan teks judul besar yang sudah *ter-style* oleh Tailwind.

Instruksi mahasiswa: Buka `minggu12.html` di peramban. Jika judul besar muncul dengan latar abu-abu, setup berhasil.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-12/tailwind-setup.png` tersedia.

Tips

Verifikasi dengan DevTools: buka tab Elements → *inspect* pada elemen `<h1>`. Anda akan melihat *class* Tailwind dikonversi menjadi CSS oleh JavaScript CDN.

Catatan

Catatan CDN: CDN ini cocok untuk pengembangan dan belajar. Untuk proyek *production*, Tailwind biasanya diinstal via *npm* dan *di-build*.

12.3 Eksplorasi Utility Class Dasar

12.3.1 Layout & Spacing

Listing 12.4: Utility class untuk spacing

```
<!-- Padding (p = semua sisi, px = horizontal, py = vertical) -->
<div class="p-4">Padding 16px semua sisi</div>
<div class="px-6 py-2">Padding horizontal 24px, vertical 8px</div>

<!-- Margin (m = semua sisi, mx = horizontal, my = vertical) -->
<div class="m-4">Margin 16px semua sisi</div>
<div class="mx-auto">Margin horizontal auto (untuk center) -->

<!-- Skala spacing: angka x 4 = pixel -->
<!-- p-1 = 4px, p-2 = 8px, p-4 = 16px, p-8 = 32px -->
```

12.3.2 Tipografi

Listing 12.5: Utility class untuk tipografi

```

<h1 class="text-4xl">Judul Besar (36px)</h1>
<h2 class="text-2xl">Judul Sedang (24px)</h2>
<p class="text-base">Teks normal (16px)</p>
<p class="text-sm">Teks kecil (14px)</p>

<p class="font-bold">Tebal</p>
<p class="font-semibold">Agak tebal</p>
<p class="font-light">Tipis</p>

<p class="text-red-500">Merah</p>
<p class="text-blue-600">Biru</p>
<p class="text-center">Tengah</p>

```

12.3.3 Warna & Background

Listing 12.6: Utility class untuk warna

```

<div class="bg-white">Latar putih</div>
<div class="bg-blue-500">Latar biru</div>
<div class="bg-gray-200">Latar abu-abu terang</div>

<!-- Skala warna Tailwind: 50 (paling terang) sampai 950 (paling gelap) -->
<!-- blue-100 sangat terang, blue-900 sangat gelap -->

```

12.3.4 Border & Rounded

Listing 12.7: Utility class untuk border

```

<div class="border border-gray-300">Batas abu-abu tipis</div>
<div class="border-2 border-red-500">Batas merah tebal 2px</div>
<div class="rounded-lg">Sudut membulat besar</div>
<div class="rounded-full">Lingkaran penuh</div>

```

12.3.5 Responsive (Mobile-First)

Listing 12.8: Kelas responsive Tailwind

```

<!-- Dasar = mobile -->
<!-- sm: >= 640px, md: >= 768px, lg: >= 1024px -->
<div class="text-sm md:text-base lg:text-lg">
  Teks kecil di HP, sedang di tablet, besar di desktop
</div>

<div class="flex flex-col md:flex-row">
  <!-- Vertikal di HP, horizontal di tablet ke atas -->
  <div class="w-full md:w-1/2">Kolom 1</div>
  <div class="w-full md:w-1/2">Kolom 2</div>
</div>

```

12.3.6 Coba Sendiri: Playground

Langkah Kerja

1. Tambahkan kode berikut ke minggu12.html (ganti isi <body>).
2. *Refresh* dan lihat hasilnya.

Listing 12.9: Tailwind *playground*—header, kartu, dan label warna

```
<body class="bg-gray-100 text-gray-800 p-6">
  <header class="bg-blue-600 text-white p-4 rounded-lg mb-6">
    <h1 class="text-2xl font-bold">Tailwind Playground</h1>
    <p class="text-blue-100">
      Mencoba utility class dasar
    </p>
  </header>

  <div class="bg-white rounded-lg shadow-md p-6 mb-4">
    <h2 class="text-xl font-semibold text-gray-800 mb-2">
      Kartu Contoh
    </h2>
    <p class="text-gray-600 mb-4">
      Ini adalah kartu dengan padding, shadow,
      dan rounded corners -- semua dari class Tailwind.
    </p>
    <button class="bg-blue-500 hover:bg-blue-700
      text-white font-bold py-2 px-4 rounded">
      Klik Saya
    </button>
  </div>

  <div class="bg-white rounded-lg shadow-md p-6">
    <h2 class="text-xl font-semibold text-gray-800 mb-2">
      Daftar Warna
    </h2>
    <div class="flex flex-wrap gap-2">
      <span class="bg-red-100 text-red-800
        px-3 py-1 rounded-full text-sm">
        Red
      </span>
      <span class="bg-blue-100 text-blue-800
        px-3 py-1 rounded-full text-sm">
        Blue
      </span>
      <span class="bg-green-100 text-green-800
        px-3 py-1 rounded-full text-sm">
        Green
      </span>
      <span class="bg-yellow-100 text-yellow-800
        px-3 py-1 rounded-full text-sm">
        Yellow
```

```

        </span>
    </div>
</div>
</body>

```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-12/tailwind-playground.png

Keterangan: Halaman dengan kartu, tombol, dan label warna yang sudah di-*style* oleh Tailwind.

Instruksi mahasiswa: Buka minggu12.html di peramban dan bandingkan tampilan Anda.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset
assets/images/minggu-12/tailwind-playground.png tersedia.

12.4 Konversi: CSS Manual ke Tailwind

Langkah Kerja

1. Buat file kartu-profil-css.html menggunakan CSS manual berikut.
2. Buat file kartu-profil-tailwind.html menggunakan versi Tailwind.
3. Bandingkan keduanya di tab browser berbeda.

Versi CSS Manual:

Listing 12.10: Kartu profil dengan CSS manual

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Kartu Profil -- CSS Manual</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f0f2f5;
      display: flex; justify-content: center;
      align-items: center; min-height: 100vh;
      margin: 0;
    }
    .card {
      background-color: white;
      border-radius: 12px; padding: 32px;
      box-shadow: 0 4px 6px rgba(0,0,0,0.1);
      text-align: center; max-width: 320px;
    }
  </style>

```

```

.avatar {
  width: 100px; height: 100px;
  border-radius: 50%;
  background-color: #4A90D9;
  margin: 0 auto 16px;
  display: flex; align-items: center;
  justify-content: center;
  font-size: 40px; color: white;
}
.name {
  font-size: 24px; font-weight: bold;
  color: #333; margin-bottom: 4px;
}
.role {
  font-size: 14px; color: #888;
  margin-bottom: 16px;
}
.bio {
  font-size: 14px; color: #666;
  line-height: 1.6; margin-bottom: 20px;
}
.btn {
  background-color: #4A90D9; color: white;
  border: none; padding: 10px 24px;
  border-radius: 8px; font-size: 14px;
  cursor: pointer;
}
.btn:hover { background-color: #357ABD; }
</style>
</head>
<body>
  <div class="card">
    <div class="avatar">B</div>
    <div class="name">Budi Santoso</div>
    <div class="role">Mahasiswa D3 Informatika</div>
    <div class="bio">
      Siswa semester 1 yang tertarik dengan
      web design dan UI/UX.
    </div>
    <button class="btn">Hubungi</button>
  </div>
</body>
</html>

```

Versi Tailwind CSS:

Listing 12.11: Kartu profil dengan Tailwind CSS

```

<!DOCTYPE html>
<html lang="id">

```

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width,initial-scale=1.0">
  <title>Kartu Profil -- Tailwind CSS</title>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 flex justify-center
  items-center min-h-screen">
  <div class="bg-white rounded-xl p-8 shadow-lg
    text-center max-w-xs">
    <div class="w-24 h-24 rounded-full bg-blue-500
      mx-auto mb-4 flex items-center
      justify-center text-4xl text-white
      font-bold">
      B
    </div>
    <div class="text-2xl font-bold text-gray-800
      mb-1">
      Budi Santoso
    </div>
    <div class="text-sm text-gray-400 mb-4">
      Mahasiswa D3 Informatika
    </div>
    <div class="text-sm text-gray-500
      leading-relaxed mb-5">
      Siswa semester 1 yang tertarik dengan
      web design dan UI/UX.
    </div>
    <button class="bg-blue-500 hover:bg-blue-700
      text-white font-semibold py-2 px-6
      rounded-lg transition duration-200">
      Hubungi
    </button>
  </div>
</body>
</html>

```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-12/kartu-perbandingan.png

Keterangan: Kartu profil versi Tailwind—tampilan seharusnya sama atau hampir identik dengan versi CSS manual.

Instruksi mahasiswa: Buka kedua file di tab browser berbeda dan bandingkan tampilannya.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-12/kartu-perbandingan.png` tersedia.

12.4.1 Pemetaan CSS ke Tailwind

Tabel 12.2: Pemetaan properti CSS ke class Tailwind

CSS Manual	Tailwind Equivalent
<code>background-color: #f0f2f5;</code>	<code>bg-gray-100</code>
<code>display: flex;</code>	<code>flex</code>
<code>justify-content: center;</code>	<code>justify-center</code>
<code>align-items: center;</code>	<code>items-center</code>
<code>min-height: 100vh;</code>	<code>min-h-screen</code>
<code>border-radius: 12px;</code>	<code>rounded-xl</code>
<code>padding: 32px;</code>	<code>p-8</code>
<code>box-shadow: ...;</code>	<code>shadow-lg</code>
<code>text-align: center;</code>	<code>text-center</code>
<code>max-width: 320px;</code>	<code>max-w-xs</code>
<code>width: 100px;</code>	<code>w-24</code>
<code>font-size: 24px; font-weight: bold;</code>	<code>text-2xl font-bold</code>
<code>color: #333;</code>	<code>text-gray-800</code>
<code>font-size: 14px;</code>	<code>text-sm</code>
<code>line-height: 1.6;</code>	<code>leading-relaxed</code>
<code>:hover {...}</code>	<code>hover:bg-blue-700</code>

12.5 Layout dengan Tailwind: Flexbox & Grid

12.5.1 Flexbox dengan Tailwind

Listing 12.12: Flexbox: horizontal dan responsive

```

<!-- Flex Horizontal (baris) -->
<h3 class="text-lg font-semibold mt-8 mb-3">
  Flex Horizontal (baris)
</h3>
<div class="flex gap-4 bg-white p-4 rounded-lg shadow">
  <div class="bg-blue-500 text-white p-4 rounded-lg
  flex-1 text-center">1</div>
  <div class="bg-green-500 text-white p-4 rounded-lg
  flex-1 text-center">2</div>
  <div class="bg-red-500 text-white p-4 rounded-lg
  flex-1 text-center">3</div>
</div>

```

```

<!-- Flex Responsive: vertikal di HP, horizontal di desktop -->
<h3 class="text-lg font-semibold mt-8 mb-3">
  Flex Responsive
</h3>
<div class="flex flex-col md:flex-row gap-4
            bg-white p-4 rounded-lg shadow">
  <div class="bg-purple-500 text-white p-6 rounded-lg
            md:flex-1 text-center">
    Kolom 1
  </div>
  <div class="bg-yellow-500 text-gray-800 p-6 rounded-lg
            md:flex-1 text-center">
    Kolom 2
  </div>
  <div class="bg-teal-500 text-white p-6 rounded-lg
            md:flex-1 text-center">
    Kolom 3
  </div>
</div>

```

12.5.2 Grid dengan Tailwind

Listing 12.13: Grid: 3 kolom responsive

```

<!-- Grid: 3 kolom responsive -->
<div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3
            gap-4 bg-white p-4 rounded-lg shadow">
  <div class="bg-indigo-500 text-white p-6 rounded-lg
            text-center">Card 1</div>
  <div class="bg-indigo-500 text-white p-6 rounded-lg
            text-center">Card 2</div>
  <div class="bg-indigo-500 text-white p-6 rounded-lg
            text-center">Card 3</div>
  <div class="bg-indigo-500 text-white p-6 rounded-lg
            text-center">Card 4</div>
  <div class="bg-indigo-500 text-white p-6 rounded-lg
            text-center">Card 5</div>
  <div class="bg-indigo-500 text-white p-6 rounded-lg
            text-center">Card 6</div>
</div>

<!-- Penjelasan grid responsive:
      grid-cols-1    = 1 kolom di HP
      sm:grid-cols-2 = 2 kolom di tablet (>= 640px)
      lg:grid-cols-3 = 3 kolom di desktop (>= 1024px) -->

```

Langkah Kerja

1. *Refresh* halaman dan coba *resize* browser untuk melihat layout *responsive*.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-12/tailwind-flexbox.png`

Keterangan: Area *flexbox*—tiga kotak berwarna dalam satu baris.

Instruksi mahasiswa: Buka `minggu12.html` di peramban dan gulir ke bagian Flexbox.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-12/tailwind-flexbox.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-12/tailwind-grid.png`

Keterangan: Area *grid*—kotak-kotak *card* dalam format 3 kolom.

Instruksi mahasiswa: Gulir ke bagian Grid dan perhatikan tampilan kolom.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-12/tailwind-grid.png` tersedia.

12.6 Latihan

Latihan 12.1 Konversi Halaman

Ambil salah satu halaman HTML yang sudah Anda buat di minggu 5–7 (halaman statis dengan CSS). Buat versi baru menggunakan Tailwind CSS.

1. Salin file HTML lama ke file baru.
2. Hapus semua isi `<style>`.
3. Tambahkan CDN Tailwind di `<head>`.
4. Ganti semua *selector* CSS dengan *class* Tailwind yang sesuai.

Output yang diharapkan:

- Halaman terlihat sama atau hampir sama dengan versi CSS manual.
- Semua *style* menggunakan *class* Tailwind, tidak ada `<style>` tersisa.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-12/latihan1-konversi.png`

Keterangan: Halaman hasil konversi—tampilan sebelah kiri (CSS manual) vs kanan (Tailwind).

Instruksi mahasiswa: Bandingkan halaman asli dengan hasil konversi Tailwind Anda.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-12/latihan1-konversi.png` tersedia.

Latihan 12.2 Card Layout Responsive

Buat file `latihan12-2.html` yang menampilkan **3 kartu produk** dalam layout *responsive*:

1. Setiap kartu memiliki: gambar *placeholder* (bg-gray-300 dengan teks “Gambar Produk”), nama produk, harga, dan tombol “Beli”.
2. Di *mobile*: kartu tampil secara vertikal (1 kolom).
3. Di tablet (≥ 640 px): 2 kolom.
4. Di desktop (≥ 1024 px): 3 kolom.

Gunakan hanya Tailwind CSS—tidak ada CSS manual.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-12/latihan2-card.png`

Keterangan: Kartu produk *responsive*—coba di berbagai ukuran browser.

Instruksi mahasiswa: Buka `latihan12-2.html` di peramban dan *resize* untuk melihat perubahan kolom.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-12/latihan2-card.png` tersedia.

Latihan 12.3 Navbar Responsive — Opsional

Buat file `latihan12-3.html` dengan *navbar* yang:

1. Horizontal di desktop (logo + navigasi berdampingan).
2. Di *mobile*: logo di kiri, tombol *hamburger* di kanan, navigasi muncul saat tombol diklik (gunakan JavaScript dari Minggu 11).
3. Semua *styling* dengan Tailwind CSS.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-12/latihan3-navbar.png`

Keterangan: *Navbar* dalam tampilan desktop dan *mobile*.

Instruksi mahasiswa: Buka `latihan12-3.html` di peramban dan *resize* untuk melihat perubahan layout.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-12/latihan3-navbar.png` tersedia.

12.7 Troubleshooting

Peringatan

Style Tailwind tidak muncul — pastikan `<script src="https://cdn.tailwindcss.com">` ada di `<head>`, periksa koneksi internet, periksa ejaan nama *class* (Tailwind sangat spesifik: `bg-blue-500` $\neq$ `bg-blue-5000`), dan coba *hard refresh* (Ctrl+Shift+R).

Tips

Tidak tahu nama class yang tepat — gunakan [Tailwind Docs](#) sebagai referensi. Cari kategori (mis. “padding”, “background color”) lalu *copy-paste class*. Anda tidak perlu menghafal semua *class*!

Catatan

Konflik dengan CSS manual — jangan campur Tailwind dengan *style* manual yang konflik. Gunakan salah satu: Tailwind SAJA atau CSS manual SAJA.

Catatan

CDN lambat atau tidak termuat — gunakan versi tertentu (<https://cdn.tailwindcss.com/3.4.1>). Jika tidak ada internet, kembali ke CSS manual untuk proyek ini.

Catatan

Kelas responsive tidak bekerja — Tailwind menggunakan **mobile-first**. Urutan yang benar: *default* (mobile) $\rightarrow$ sm $\rightarrow$ md $\rightarrow$ lg $\rightarrow$ xl.

Bab 13 Pengenalan React: Komponen, JSX & Props

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- menjelaskan mengapa React digunakan dan bagaimana posisinya dalam pengembangan web;
- membaca dan memahami kode React sederhana (komponen, JSX, *props*);
- membedakan JSX dari HTML biasa;
- mengidentifikasi struktur komponen, termasuk komponen bersarang dan penggunaan `map()` untuk daftar.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- membaca kode React secara *visual* tanpa menjalankannya;
- mengonstruksi file HTML berisi potongan kode React dengan *syntax highlighting*;
- berlatih menjawab pertanyaan pemahaman tentang kode React.

Catatan

Modul ini **hanya pengenalan**. Kita **TIDAK** akan menginstal React, menjalankan `create-react-app` atau Vite, maupun menggunakan *build tools*. Tujuannya: membaca, memahami, dan “merasakan” kode React agar konsepnya tidak asing saat belajar di semester lanjutan.

13.1 Mengapa React?

React adalah *JavaScript library* untuk membangun **antarmuka (UI) berbasis komponen**—artinya, Anda memecah halaman web menjadi potongan-potongan kecil yang bisa digunakan ulang.

13.1.1 Perbandingan: Tanpa React vs Dengan React

Tanpa React (vanilla JavaScript):

Listing 13.1: Manipulasi DOM manual tanpa React

```
// Anda harus secara manual mencari elemen dan mengubahnya
const daftar = document.getElementById("daftar-mahasiswa");
const item = document.createElement("li");
item.textContent = "Budi";
daftar.appendChild(item);
// Jika data berubah -> Anda harus menulis ulang kode manipulasi DOM
```

Dengan React:

Listing 13.2: Menulis UI secara deklaratif dengan React

```
// Anda cukup mendeskripsikan APA yang ingin ditampilkan
function DaftarMahasiswa({ mahasiswa }) {
```

```

return (
  <ul>
    {mahasiswa.map((nama, index) => (
      <li key={index}>{nama}</li>
    ))}
  </ul>
);
}

// React yang mengurus bagaimana update DOM saat data berubah

```

13.1.2 Mengapa React Populer?

Tabel 13.1: Alasan popularitas React

Alasan	Penjelasan
<i>Komponen</i>	UI dipecah jadi blok kecil, bisa digunakan ulang
Deklaratif	Tulis “apa” yang ingin ditampilkan, bukan “bagaimana” update DOM
Ekosistem besar	Banyak library pendukung (routing, <i>state management</i> , UI kit)
<i>Industry standard</i>	Banyak perusahaan teknologi menggunakan React

13.2 Membaca Kode React (Tanpa Menjalankan)

Tujuan: melihat kode React secara utuh agar terbiasa dengan sintaksnya. Kita **tidak menjalankan** kode ini—kita membaca dan memahaminya.

Langkah Kerja

1. Buat file baru `react-preview.html`.
2. Masukkan kode HTML berikut (berisi empat contoh komponen React).
3. Buka di peramban untuk melihat tampilan kode yang sudah diberi warna *syntax highlighting*.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-13/react-preview-kode.png`

Keterangan: Halaman *preview* kode React dengan *syntax highlighting* yang rapi.

Instruksi mahasiswa: Buka `react-preview.html` di peramban setelah menambahkan kode-kode berikut.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-13/react-preview-kode.png` tersedia.

13.2.1 Kode 1 — Komponen Paling Sederhana

Listing 13.3: Komponen “Halo Dunia” paling sederhana

```
// Dalam React, UI ditulis sebagai FUNCTION
// yang mengembalikan JSX
function Sapa() {
  return (
    <div className="container">
      <h1>Halo, Dunia React!</h1>
      <p>Ini adalah komponen React pertama saya.</p>
    </div>
  );
}
```

Catatan

JSX mirip HTML, tapi ada perbedaan kecil—misal: `class` menjadi `className`. Komponen harus diawali huruf KAPITAL (misal: `Sapa`, bukan `sapa`).

13.2.2 Kode 2 — Komponen dengan Props**Listing 13.4:** Komponen yang menerima data melalui *props*

```
// Props = "parameter" untuk komponen React
function KartuProfil({ nama, jurusan, angkatan }) {
  return (
    <div className="bg-white rounded-lg shadow-md p-6">
      <h2 className="text-xl font-bold">{nama}</h2>
      <p className="text-gray-600">
        Jurusan: {jurusan}
      </p>
      <p className="text-gray-600">
        Angkatan: {angkatan}
      </p>
    </div>
  );
}

// Cara menggunakan (memanggil) komponen:
// <KartuProfil nama="Budi" jurusan="Informatika"
//           angkatan="2025" />
// <KartuProfil nama="Ani" jurusan="Informatika"
//           angkatan="2025" />
```

Tips

Fungsi `KartuProfil` menerima tiga *props* (`nama`, `jurusan`, `angkatan`) dan menggunakannya di dalam JSX. Komponen yang sama bisa digunakan berulang kali dengan data berbeda—itulah kekuatan komponen!

13.2.3 Kode 3 — Komponen dengan Daftar (Map)

Listing 13.5: Menampilkan daftar dengan `map()`

```
function DaftarMahasiswa({ mahasiswa }) {
  return (
    <div className="bg-white rounded-lg shadow p-4">
      <h2 className="text-lg font-bold mb-2">
        Daftar Mahasiswa
      </h2>
      <ul>
        {mahasiswa.map((nama, index) => (
          <li key={index} className="py-1 border-b">
            {index + 1}. {nama}
          </li>
        ))}
      </ul>
    </div>
  );
}

// Cara menggunakan:
// <DaftarMahasiswa
//   mahasiswa={["Budi", "Ani", "Andi", "Sari"]} />
```

Catatan

`map()` dalam JSX mirip `forEach()` di JavaScript biasa—tetapi mengembalikan nilai (*return*). `key` diperlukan oleh React untuk mengidentifikasi setiap elemen dalam daftar.

13.2.4 Kode 4 — Komponen Bersarang (Nested)

Listing 13.6: Komponen Tombol yang digunakan di dalam HalamanUtama

```
// Komponen kecil -- tombol
function Tombol({ teks, warna }) {
  return (
    <button className={
      'px-4 py-2 rounded text-white font-bold ${warna}'
    }>
      {teks}
    </button>
  );
}

// Komponen besar -- menggunakan Tombol
function HalamanUtama() {
  return (
    <div className="p-6">
```

```

    <h1 className="text-2xl font-bold mb-4">
      Selamat Datang
    </h1>
    <p className="mb-4">Pilih aksi berikut:</p>
    { /* Menggunakan komponen Tombol */ }
    <Tombol teks="Mulai" warna="bg-blue-500" />
    <span> </span>
    <Tombol teks="Batal" warna="bg-red-500" />
  </div>
);
}

```

Tips

HalamanUtama menggunakan komponen `Tombol` dua kali dengan data berbeda. Ini seperti menggunakan fungsi dalam fungsi—komponen React bisa dipanggil berkali-kali dengan data berbeda.

13.3 JSX: HTML yang “Bisa Dikode”

JSX (JavaScript XML) adalah sintaks yang mirip HTML tapi ditulis **di dalam JavaScript**. React menggunakan JSX sebagai cara mendeskripsikan tampilan UI.

13.3.1 Perbedaan JSX vs HTML

Tabel 13.2: Perbedaan sintaks JSX dan HTML

Aspek	HTML	JSX
<i>class</i>	<code>class="..."</code>	<code>className="..."</code>
<i>for (label)</i>	<code>for="..."</code>	<code>htmlFor="..."</code>
<i>style</i>	<code>style="color: red;"</code>	<code>style={{ color: 'red' }}</code>
<i>self-closing</i>	<code>
, </code>	<code>
, </code>
Ekspresi	Tidak bisa	Bisa pakai <code>{ }</code>
Komentar	<code><!-- ... --></code>	<code>/* ... */</code>

13.3.2 JSX Adalah Ekspresi JavaScript

Listing 13.7: Ekspresi JavaScript di dalam JSX

```

// Anda bisa menulis JavaScript di dalam JSX menggunakan { }

function Profil({ nama, umur }) {
  return (
    <div>
      <h2>Nama: {nama}</h2>
      <p>Umur: {umur} tahun</p>
      <p>Status: {umur >= 17 ? 'Dewasa' : 'Anak-anak'}</p>
      <p>Tahun Lahir: {2025 - umur}</p>
    </div>
  );
}

```

```

    </div>
  );
}

// {} di JSX = "masukkan hasil ekspresi JavaScript di sini"

```

Catatan

Perbandingan dengan JavaScript biasa: `"<h2>Nama: " + nama + "</h2>"` vs `<h2>Nama: {nama}</h2>`—JSX jauh lebih bersih dan mudah dibaca.

13.4 Komponen: Blok Bangunan UI

Analogi LEGO: setiap komponen React adalah bata LEGO—punya bentuk dan warna tersendiri (*props* & *style*), bisa disusun menjadi struktur lebih besar, dan bisa digunakan berulang kali di tempat berbeda.

Listing 13.8: Tiga pola komponen: tanpa props, dengan props, dan komposisi

```

// 1. Komponen tanpa props (self-contained)
function Header() {
  return (
    <nav className="bg-blue-600 text-white p-4">
      <h1 className="text-xl font-bold">My Website</h1>
    </nav>
  );
}

// 2. Komponen dengan props
function Card({ title, description, buttonText }) {
  return (
    <div className="bg-white rounded-lg shadow p-4">
      <h2 className="text-lg font-bold">{title}</h2>
      <p className="text-gray-600 mb-3">{description}</p>
      <button className="bg-blue-500 text-white
      px-4 py-2 rounded">
        {buttonText}
      </button>
    </div>
  );
}

// 3. Komponen menyusun komponen lain (Komposisi)
function App() {
  return (
    <div>
      <Header />
      <Card title="Tentang Kami"
        description="Kami adalah..."
        buttonText="Selengkapnya" />
    </div>
  );
}

```

```

        <Card title="Layanan"
              description="Kami menyediakan..."
              buttonText="Lihat" />
      </div>
    );
  }

```

Tips

Perubahan di satu definisi komponen berlaku untuk *semua* penggunaannya. Misal: jika Anda mengubah struktur Card, semua kartu otomatis berubah. Tanpa komponen, Anda harus mengubah setiap kartu secara manual.

13.5 Props: Mengirim Data ke Komponen

Props (properties) adalah cara mengirim data **dari luar ke dalam** komponen—seperti parameter fungsi.

Listing 13.9: Props sebagai “parameter” komponen

```

function Greeting({ nama }) {
  return <h1>Halo, {nama}!</h1>;
}

// Menggunakan komponen:
// <Greeting nama="Budi" /> -> render: <h1>Halo, Budi!</h1>
// <Greeting nama="Ani" /> -> render: <h1>Halo, Ani!</h1>

```

13.5.1 Analogi Fungsi JavaScript

Listing 13.10: Perbandingan fungsi JS biasa dengan komponen React

```

// JavaScript biasa
function sapa(nama) {
  return "Halo, " + nama + "!";
}

sapa("Budi"); // "Halo, Budi!"
sapa("Ani"); // "Halo, Ani!"

// React = konsep yang sama, tapi output-nya JSX (UI)
// <Greeting nama="Budi" /> = sapa("Budi") dalam konteks UI

```

13.5.2 Props Bisa Berisi Apa Saja

Listing 13.11: Props dengan berbagai tipe data

```

function ProductCard({ name, price, inStock, rating, image }) {
  return (

```

```

    <div className="bg-white rounded-lg shadow p-4">
      <img src={image} alt={name}
        className="w-full h-48 object-cover rounded" />
      <h2 className="text-lg font-bold mt-2">{name}</h2>
      <p className="text-green-600 font-semibold">
        Rp {price.toLocaleString('id-ID')}
      </p>
      <p className="text-sm text-gray-500">
        {inStock ? 'Tersedia' : 'Habis'}
      </p>
      <p className="text-sm">{rating}/5</p>
    </div>
  );
}

// Menggunakan:
// <ProductCard name="Laptop ASUS" price={8500000}
//   inStock={true} rating={4.5} image="/laptop.jpg" />

```

13.5.3 Aturan Penting Props

Listing 13.12: Aturan *props*: read-only, default, dan children

```

// 1. Props bersifat READ-ONLY -- komponen TIDAK BOLEH
//   mengubah props
function Avatar({ nama }) {
  // nama = "Hacker"; // JANGAN! Props tidak boleh diubah
  // Gunakan variabel baru jika perlu
  const namaBesar = nama.toUpperCase();
  return <div>{namaBesar}</div>;
}

// 2. Default props
function Button({ text = "Klik", color = "blue" }) {
  return <button className={`bg-${color}-500 text-white p-2`}>
    {text}
  </button>;
}

// 3. Children props -- konten di antara tag pembuka dan penutup
function Container({ children }) {
  return <div className="max-w-md mx-auto bg-white
    p-6 rounded-lg shadow">
    {children}
  </div>;
}

// Menggunakan children:

```

```
// <Container>
//   <h1>Judul</h1>
//   <p>Paragraf di dalam container</p>
// </Container>
```

13.6 Demo Lengkap: Membaca & Memahami Kode React

Berikut kode React yang **lebih realistis**—seperti yang mungkin Anda temui di proyek nyata.

Langkah Kerja

1. Tambahkan kode berikut ke `react-preview.html` (setelah kode-kode sebelumnya).
2. Buka di peramban dan baca kode dengan seksama.
3. Coba identifikasi pertanyaan-pertanyaan pemahaman di bawah kode.

Listing 13.13: Demo lengkap: Halaman Profil Mahasiswa dalam React

```
// =====
// File: App.jsx -- Halaman Profil Mahasiswa
// =====

// 1. Komponen kecil: badge status
function StatusBadge({ aktif }) {
  if (aktif) {
    return <span className="bg-green-100 text-green-800
xxxxxxxxxxxxxxxxxxxxxxxxpx-3 py-1 rounded-full text-sm">
      Aktif
    </span>;
  }
  return <span className="bg-red-100 text-red-800
xxxxxxxxxxxxxxxxxxxxxxxxpx-3 py-1 rounded-full text-sm">
    Tidak Aktif
  </span>;
}

// 2. Komponen medium: kartu profil
function KartuProfil({ nama, nim, jurusan, ipk, aktif }) {
  return (
    <div className="bg-white rounded-xl shadow-lg
xxxxxxxxxxxxxxxxxxxxxxxxp-6 max-w-sm">
      {/* Avatar */}
      <div className="w-20 h-20 rounded-full bg-blue-500
xxxxxxxxxxxxxxxxxxxxxxxxflex items-center justify-center text-white
xxxxxxxxxxxxxxxxxxxxxxxxtext-2xl font-bold mx-auto mb-4">
        {nama.charAt(0)}
      </div>

      {/* Info */}
      <h2 className="text-xl font-bold text-center mb-1">
        {nama}
```

```

    </h2>
    <p className="text-gray-500 text-center mb-3">
      NIM: {nim}
    </p>
    <div className="text-center mb-3">
      <StatusBadge aktif={aktif} />
    </div>

    { /* Detail */ }
    <div className="border-t pt-3 mt-3 space-y-1
    text-sm">
      <p><strong>Jurusan:</strong> {jurusan}</p>
      <p><strong>IPK:</strong>
        <span className={
          ipk >= 3.5
            ? "text-green-600"
            : "text-red-600"
        }>
          {ipk}
        </span>
      </p>
    </div>
  </div>
);
}

// 3. Komponen utama: halaman
function HalamanProfil() {
  const daftarMahasiswa = [
    { nama: "Budi Santoso", nim: "2401001",
      jurusan: "Informatika", ipk: 3.8, aktif: true },
    { nama: "Ani Wijaya", nim: "2401002",
      jurusan: "Informatika", ipk: 3.2, aktif: true },
    { nama: "Andi Pratama", nim: "2401003",
      jurusan: "Sistem Informasi", ipk: 2.9, aktif: false },
  ];

  return (
    <div className="min-h-screen bg-gray-100 py-10">
      <h1 className="text-3xl font-bold text-center mb-8">
        Daftar Profil Mahasiswa
      </h1>
      <div className="flex flex-wrap justify-center
      gap-6 px-4">
        {daftarMahasiswa.map((mhs) => (
          <KartuProfil
            key={mhs.nim}
            nama={mhs.nama}

```

```

        nim={mhs.nim}
        jurusan={mhs.jurusan}
        ipk={mhs.ipk}
        aktif={mhs.aktif}
      />
    )}}
  </div>
</div>
);
}

```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-13/react-kode-lengkap.png`

Keterangan: Kode lengkap di peramban dengan *syntax highlighting*.

Instruksi mahasiswa: Buka `react-preview.html` di peramban setelah menambahkan kode di atas.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-13/react-kode-lengkap.png` tersedia.

Checklist pemahaman—centang jika sudah bisa menjawab:

- ☐ Komponen `StatusBadge` menerima *props* apa?
- ☐ Di mana `StatusBadge` digunakan (disebut)?
- ☐ Bagaimana cara menentukan warna IPK (hijau/merah)?
- ☐ Bagaimana `HalamanProfil` membuat daftar kartu?
- ☐ Apa fungsi key pada `KartuProfil`?

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-13/react-checklist.png`

Keterangan: Checklist pemahaman yang sudah diisi.

Instruksi mahasiswa: Tuliskan jawaban Anda untuk setiap pertanyaan di atas.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-13/react-checklist.png` tersedia.

13.7 Latihan

Latihan 13.1 Membaca Kode React

Baca kode React berikut dan jawab pertanyaan di bawahnya:

```
function TodoItem({ text, completed }) {
  return (
    <div className="flex_items-center_gap-2_p-2">
      <input type="checkbox" checked={completed}
        readOnly />
      <span className={
        completed ? "line-through_text-gray-400" : ""
      }>
        {text}
      </span>
    </div>
  );
}

function TodoList() {
  const todos = [
    { text: "Belajar_HTML", completed: true },
    { text: "Belajar_CSS", completed: true },
    { text: "Belajar_JavaScript", completed: false },
    { text: "Belajar_React", completed: false },
  ];

  return (
    <div className="max-w-md_mx-auto_bg-white_p-4
      rounded-lg_shadow">
      <h2 className="text-xl_font-bold_mb-3">
        To-Do List
      </h2>
      {todos.map((todo, index) => (
        <TodoItem
          key={index}
          text={todo.text}
          completed={todo.completed}
        />
      ))}
    </div>
  );
}
```

Pertanyaan:

1. Komponen TodoItem menerima *props* apa saja?
2. Apa yang terjadi pada teks jika `completed === true`?

3. Berapa banyak `TodoItem` yang akan ditampilkan?
4. Berapa banyak item yang sudah selesai (centang)?
5. Jika Anda ingin menambah item baru, bagian mana yang perlu diubah?

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-13/latihan1-jawaban.png`

Keterangan: Tuliskan jawaban Anda (boleh di file teks atau di komentar di dalam file HTML).

Instruksi mahasiswa: Jawab setiap pertanyaan di atas berdasarkan kode yang diberikan.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-13/latihan1-jawaban.png` tersedia.

Latihan 13.2 Menulis Pseudo-Code React

Bayangkan Anda diminta membuat komponen `KartuProduk` untuk toko *online*. **Tanpa menjalankan kode**, tuliskan (dalam bentuk *pseudo-code* atau kode JSX mentah) komponen yang:

1. Menerima *props*: nama, harga, gambar (URL), diskon (persen, bisa 0).
2. Menampilkan gambar, nama produk, harga asli, dan harga setelah diskon (jika diskon > 0).
3. Menggunakan Tailwind CSS untuk *styling* (minimal: `rounded`, `shadow`, `padding`).

Output yang diharapkan (kira-kira tampilan yang Anda bayangkan):

```
+-----+
|  [Gambar Produk]  |
|                   |
|  Laptop ASUS      |
|  Rp 8.500.000     |
|  ~~Rp 10.000.000~~ | <-- hanya jika diskon > 0
|  (hemat 15%)      |
+-----+
```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-13/latihan2-pseudocode.png`

Keterangan: Kode JSX *pseudo-code* yang Anda tulis.

Instruksi mahasiswa: Tuliskan kode JSX untuk komponen `KartuProduk` Anda.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-13/latihan2-pseudocode.png` tersedia.

Latihan 13.3 Peta Pemahaman — Opsional

Buat daftar 5 konsep React yang sudah Anda pahami dan 5 konsep yang masih ingin dipelajari lebih lanjut. Ini akan menjadi acuan saat Anda belajar React di semester lanjutan.

Tabel: Contoh format peta pemahaman

Sudah Dipahami	Ingin Dipelajari
Komponen adalah fungsi	<code>useState</code> hook
<i>Props</i> = parameter komponen	<code>useEffect</code> (<i>side effects</i>)
JSX mirip HTML tapi ada bedanya	React Router (navigasi)
Komponen bisa bersarang	API <i>calls</i> di React
<code>map()</code> untuk membuat daftar	Context API (<i>state</i> global)

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-13/latihan3-peta.png`

Keterangan: Daftar pemahaman Anda.

Instruksi mahasiswa: Tuliskan 5 konsep yang sudah dipahami dan 5 yang ingin dipelajari.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-13/latihan3-peta.png` tersedia.

13.8 Troubleshooting & Catatan

13.8.1 FAQ Umum tentang React

Catatan

Q: Apakah saya harus menguasai React sekarang? Tidak. Modul ini hanya pengenalan. Anda akan belajar React secara mendalam di semester lanjutan. Yang penting sekarang adalah **membaca dan memahami konsep dasarnya**.

Catatan

Q: Kenapa ada banyak simbol aneh di kode React?

- `{}` = ekspresi JavaScript di dalam JSX
- `() => {}` = *arrow function* (sudah dipelajari di Minggu 10)
- `className` = *attribute* di JSX untuk menetapkan class CSS (karena `class` adalah kata kunci JavaScript)

Catatan

Q: React vs Vue vs Angular—mana yang terbaik? Tidak ada yang “terbaik” secara mutlak. React paling banyak digunakan di industri global. Vue lebih populer di beberapa region. Angular untuk aplikasi *enterprise*. Untuk kuliah, React adalah pilihan aman karena ekosistemnya paling besar.

13.8.2 Kosa Kata React**Tabel 13.3:** Istilah penting dalam React

Istilah	Arti
<i>Component</i>	Blok bangunan UI—fungsi yang mengembalikan JSX
<i>JSX</i>	JavaScript XML—sintaks HTML- <i>like</i> di dalam JavaScript
<i>Props</i>	Data yang dikirim ke komponen (<i>read-only</i>)
<i>State</i>	Data internal komponen yang bisa berubah (semester lanjutan)
<i>Virtual DOM</i>	Representasi DOM di memori—React update DOM secara efisien
<i>Hook</i>	Fitur khusus React (<i>useState</i> , <i>useEffect</i>)—semester lanjutan

13.8.3 Ringkasan Perjalanan Minggu 10–13**Tabel 13.4:** Topik dan keterampilan yang diperoleh dari Minggu 10–13

Minggu	Topik	Skill yang Didapat
10	JavaScript dasar	Variabel, fungsi, kondisi, DOM selection
11	JavaScript interaktif	<i>Event handling</i> , manipulasi DOM
12	Tailwind CSS	<i>Utility-first CSS</i> , layout <i>responsive</i>
13	React <i>preview</i>	Membaca kode komponen, JSX, <i>props</i>

Tips

Next step (semester lanjutan): Anda akan membangun aplikasi React yang sebenarnya—menggunakan *state*, *effect*, *routing*, dan API. Fondasi dari Minggu 10–13 akan membuat proses itu jauh lebih mudah!

Bab 14 Workshop Proyek Akhir — Riset UI/UX & Perencanaan Desain

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- melakukan *heuristic evaluation* terhadap halaman website *e-governance* menggunakan 10 *Usability Heuristics* (Nielsen Norman Group);
- mendokumentasikan *pain point* dan hasil *benchmarking* secara terstruktur;
- membuat *wireframe* dan *mockup* desain ulang di Figma berdasarkan temuan riset;
- menyusun rasional desain yang menghubungkan temuan riset dengan keputusan desain.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- menyiapkan struktur *repository* dan *branch* untuk proyek akhir;
- mengidentifikasi halaman website target dan mendokumentasikannya;
- melakukan *heuristic evaluation* dan *benchmarking*;
- membuat *wireframe* dan *mockup* di Figma;
- menulis rasional desain.

14.1 Persiapan Kelompok

Setiap kelompok terdiri atas 3 orang. Berikut peralatan yang dibutuhkan:

- **Browser** (Chrome/Firefox) — akses website target dan *heuristic checklist*.
- **Figma** (akun gratis) — *wireframing* dan *mockup*.
- **VS Code** — persiapan untuk minggu 15.
- **GitHub** — *repository* proyek akhir.
- **Google Docs / Markdown editor** — dokumentasi hasil riset.

14.1.1 Inisialisasi Repository

Langkah Kerja

1. Buka *repository* GitHub kelompok (yang sudah dibuat dari minggu 8–9 atau buat baru).
2. Buat *branch* baru untuk proyek akhir:

Listing 14.1: Membuat branch final-project

```
git checkout -b final-project
```

3. Buat struktur folder di dalam *repo*:

Listing 14.2: Struktur folder proyek akhir

```
final-project/  
  research/
```

```

    heuristic-evaluation.md
    benchmarking.md
  design/
  src/
    index.html
    css/
    js/
    assets/
  README.md

```

4. Push branch ke GitHub:

Listing 14.3: Push awal

```

git add .
git commit -m "feat: inisialisasi struktur folder proyek akhir"
git push -u origin final-project

```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-14/struktur-folder-repo.png

Keterangan: Struktur folder repo di VS Code Explorer.

Instruksi mahasiswa: Buka VS Code → Explorer → periksa bahwa seluruh folder dan file sudah sesuai.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset
assets/images/minggu-14/struktur-folder-repo.png tersedia.

14.2 Identifikasi Website Target

Langkah Kerja

1. Pilih satu halaman dari website *e-governance* / layanan publik, contoh:
 - Portal layanan perizinan daerah.
 - Halaman pendaftaran online instansi pemerintah.
 - Portal informasi desa/kelurahan.
 - Halaman pengaduan masyarakat *online*.
2. Dokumentasikan informasi umum di `research/heuristic-evaluation.md` — nama website, URL, halaman yang diriset, tanggal, dan anggota kelompok beserta peran masing-masing.
3. Ambil *screenshot* halaman asli dari berbagai ukuran layar (desktop dan mobile) menggunakan *Developer Tools* (F12) → *toggle device toolbar*.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-14/halaman-target-desktop-mobile.png`

Keterangan: Tampilan halaman website target — versi desktop dan mobile.

Instruksi mahasiswa: Buka halaman target di browser, ambil *screenshot* versi desktop, lalu gunakan *Device Toolbar* di *DevTools* untuk versi mobile.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-14/halaman-target-desktop-mobile.png` tersedia.

Tips

Jika sulit menentukan website target, pilih website pemerintah daerah (portal layanan disdukcapil, BPJS, atau website desa). Hindari website yang sudah sangat modern.

14.3 Heuristic Evaluation

Gunakan 10 *Usability Heuristics* dari Nielsen Norman Group berikut sebagai panduan evaluasi.

Tabel 14.1: 10 Usability Heuristics — Nielsen Norman Group

#	Heuristik	Pertanyaan Panduan
1	Visibility of System Status	Apakah pengguna tahu apa yang sedang terjadi di halaman?
2	Match Between System and Real World	Apakah bahasa dan istilah sesuai dunia nyata pengguna?
3	User Control and Freedom	Apakah pengguna bisa membatalkan aksi atau navigasi dengan mudah?
4	Consistency and Standards	Apakah tampilan dan interaksi konsisten di seluruh halaman?
5	Error Prevention	Apakah ada perlindungan untuk mencegah kesalahan pengguna?
6	Recognition Rather Than Recall	Apakah pengguna perlu mengingat informasi dari satu bagian ke bagian lain?
7	Flexibility and Efficiency of Use	Apakah ada cara cepat untuk pengguna yang sudah terbiasa?
8	Aesthetic and Minimalist Design	Apakah antarmuka bebas dari elemen yang tidak perlu?
9	Help Users Recognize, Diagnose, and Recover from Errors	Apakah pesan <i>error</i> jelas dan membantu?
10	Help and Documentation	Apakah ada panduan atau bantuan jika pengguna kebingungan?

14.3.1 Langkah Evaluasi

Untuk setiap heuristik, diskusikan secara berkelompok dan catat di `research/heuristic-evaluation.md`. Tuliskan: skor pelanggaran (0–4, di mana 0 = tidak ada masalah dan 4 = masalah kritis), temuan, *screenshot*

bukti, dampak ke pengguna, serta saran perbaikan yang akan diterapkan di desain ulang.

Langkah Kerja

1. Buka *heuristic checklist* di atas satu per satu.
2. Untuk setiap heuristik, identifikasi apakah ada pelanggaran pada halaman target.
3. Catat temuan beserta *screenshot* bukti.
4. Uraikan dampak temuan terhadap pengalaman pengguna.
5. Hubungkan setiap temuan dengan rekomendasi desain yang akan diterapkan.

Tips

Jangan khawatir soal skor yang “benar” — yang penting adalah kemampuan menjelaskan mengapa suatu masalah muncul. Satu heuristik bisa memiliki 0 atau lebih temuan. Jika tidak ada masalah, tulis “Tidak ditemukan pelanggaran signifikan.”

14.3.2 Ringkasan Temuan

Buat tabel ringkasan di bagian bawah file `heuristic-evaluation.md` berisi nomor heuristik, skor, dan jumlah temuan. Identifikasi 3 *pain point* terbesar dari seluruh evaluasi.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-14/heuristic-evaluation-dokumentasi.png`

Keterangan: Dokumentasi heuristic evaluation di Markdown — tampilan di VS Code atau GitHub.

Instruksi mahasiswa: Buka file `research/heuristic-evaluation.md` di VS Code, periksa bahwa seluruh heuristik sudah dievaluasi dan tabel ringkasan sudah dilengkapi.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-14/heuristic-evaluation-dokumentasi.png` tersedia.

14.4 Benchmarking Singkat

Bandingkan halaman target dengan minimal 2–3 website sejenis yang memiliki fitur serupa.

Langkah Kerja

1. Pilih minimal 2 website untuk dibandingkan (misalnya website pemerintah daerah lain atau layanan publik modern).
2. Buat file `research/benchmarking.md` yang memuat:
 - Informasi setiap website: URL, halaman sejenis, kelebihan, kekurangan, dan *screenshot*.
 - Tabel perbandingan fitur antara website target, website pembandingan, dan rencana desain ulang.
 - Daftar pelajaran dari *benchmarking* (fitur yang ingin diadopsi, yang perlu dihindari, dan peluang diferensiasi).

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-14/benchmarking-perbandingan.png`

Keterangan: Perbandingan visual antara website target dan website pembandingan.

Instruksi mahasiswa: Buka file `research/benchmarking.md` dan pastikan tabel perbandingan fitur sudah terisi lengkap.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-14/benchmarking-perbandingan.png` tersedia.

14.5 Perencanaan Desain di Figma

14.5.1 Membuat Frame Wireframe

Langkah Kerja

1. Buka Figma (*desktop app* atau browser), klik “**New design file**”.
2. Buat *frame* — tekan F lalu pilih ukuran: Desktop 1440×900, Mobile 375×812. Beri nama “Wireframe — Desktop” dan “Wireframe — Mobile”.
3. Sketsa tata letak **tanpa warna**, hanya menggunakan:
 - Kotak untuk area konten.
 - Garis atau garis putus untuk teks *placeholder*.
 - Ikon sederhana untuk elemen navigasi.
 - Anotasi singkat di sebelah setiap elemen.

Tips

Gunakan hanya warna abu-abu, hitam, dan putih pada wireframe. Jangan tambahkan warna, gambar, atau font khusus. Fokus pada struktur dan alur pengguna, bukan visual.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-14/wireframe-desktop-mobile.png`

Keterangan: Wireframe desktop dan mobile di Figma — tampilan keseluruhan.

Instruksi mahasiswa: Buka file Figma, periksa bahwa *frame* wireframe sudah dibuat untuk versi desktop dan mobile dengan anotasi yang jelas.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-14/wireframe-desktop-mobile.png` tersedia.

14.5.2 Membuat Mockup

Langkah Kerja

1. Duplikat *frame* wireframe → rename menjadi “Mockup — Desktop” dan “Mockup — Mobile”.
2. Terapkan visual pada *mockup*:
 - Warna merek (maksimal 3–4 warna).
 - Font yang sudah dipilih (*Google Fonts* atau *font system*).
 - Gambar/ikon asli.
 - Spasi dan tipografi yang konsisten.
3. Pastikan konsistensi antar halaman — header/footer identik, tipografi konsisten, spasi antar elemen seragam.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-14/mockup-desktop-overview.png`

Keterangan: Mockup desktop di Figma — tampilan keseluruhan.

Instruksi mahasiswa: Periksa bahwa *mockup* sudah menerapkan warna merek, tipografi, dan spasi secara konsisten.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset
`assets/images/minggu-14/mockup-desktop-overview.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-14/mockup-mobile-overview.png`

Keterangan: Mockup mobile di Figma — tampilan keseluruhan.

Instruksi mahasiswa: Periksa bahwa versi mobile memiliki tata letak yang responsif dan konsisten dengan versi desktop.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset
`assets/images/minggu-14/mockup-mobile-overview.png` tersedia.

14.5.3 Menulis Rasional Desain

Langkah Kerja

1. Buat file `research/rasional-desain.md` atau tulis langsung di `README.md`.
2. Isi tabel yang menghubungkan temuan riset dengan keputusan desain: nomor, temuan riset, keputusan desain, dan penjelasan mengapa keputusan tersebut diambil.
3. Daftar prinsip desain yang diterapkan (misalnya *Consistency*, *Minimalism*, *Error Prevention*).

Catatan

Kaitan antara temuan riset dan keputusan desain merupakan komponen kunci yang akan dinilai pada presentasi minggu 16. Pastikan setiap keputusan desain dapat ditelusuri kembali ke minimal satu temuan riset.

14.6 Commit dan Push

Langkah Kerja

1. Kembali ke VS Code, buka terminal.
2. Pastikan semua file sudah tersimpan di folder yang benar.
3. *Commit* dan *push*:

Listing 14.4: Commit hasil riset

```
git add .
git commit -m "feat: hasil riset heuristic evaluation, benchmarking, dan wireframe/
mockup"
git push
```

4. Buka *pull request* di GitHub (opsional, tetapi disarankan) dengan deskripsi singkat mengenai isi *commit*.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-14/commit-history-github.png`

Keterangan: Commit history di GitHub — terlihat branch final-project.

Instruksi mahasiswa: Buka *repository* di GitHub, navigasi ke tab “Commits” untuk memastikan *commit* berhasil.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-14/commit-history-github.png` tersedia.

14.7 Output yang Diharapkan

Setelah sesi ini, setiap kelompok harus memiliki:

- Dokumen *heuristic evaluation* (`research/heuristic-evaluation.md`).
- Dokumen *benchmarking* (`research/benchmarking.md`).
- *Wireframe* (desktop & mobile) di Figma.
- *Mockup* (desktop & mobile) di Figma.
- Rasional desain (`research/rasional-desain.md` atau `README.md`).
- *Screenshot* halaman asli di folder `assets/` atau `research/`.
- *Commit* di GitHub pada branch `final-project`.

14.8 Troubleshooting

Tabel 14.2: Masalah Umum dan Solusi — Minggu 14

Masalah	Solusi
Website target tidak bisa diakses	Gunakan Wayback Machine (web.archive.org) untuk versi terakhir yang bisa diakses, atau pilih website lain.
Tidak yakin bagaimana mengisi <i>heuristic evaluation</i>	Lihat contoh di Nielsen Norman Group — fokus pada kemampuan menjelaskan, bukan skor sempurna.
Figma tidak bisa dibuka di browser	Gunakan <i>Figma desktop app</i> (download dari figma.com), atau pastikan browser versi terbaru.
Anggota kelompok di lokasi berbeda	Semua file riset ada di GitHub — akses dari mana saja. Desain Figma bisa di- <i>share</i> via tautan.
Tidak bisa membuat <i>branch</i> di Git	Pastikan sudah <i>clone repository</i> ke lokal. Jika masih gagal, cek <code>git status</code> .
Pekerjaan terasa terlalu banyak untuk 1 pertemuan	Fokus pada <i>heuristic evaluation</i> dan <i>wireframe</i> terlebih dahulu. <i>Mockup</i> bisa dilanjutkan di minggu 15.

14.9 Tautan Referensi

- 10 Usability Heuristics (NNGroup): nngroup.com/articles/ten-usability-heuristics
- Figma Getting Started: help.figma.com/hc/en-us/articles/360040319993
- Heuristic Evaluation Template (NNGroup): nngroup.com/articles/how-to-conduct-a-heuristic-evaluation
- Wayback Machine: web.archive.org
- Contoh Website E-Governance: <https://www.go.id>, <https://jakartasatu.jakarta.go.id>

14.10 Persiapan untuk Minggu 15

Sebelum pertemuan minggu depan, pastikan:

1. Semua dokumen riset sudah *ter-commit* dan *ter-push* ke GitHub.
2. Desain Figma sudah selesai (*wireframe* + *mockup*).
3. Anggota kelompok sudah berbagi peran untuk implementasi:
 - Anggota A: header, navigasi, *hero section*.
 - Anggota B: bagian tengah (konten utama, formulir, informasi).
 - Anggota C: footer, bagian bawah, optimasi responsif.
4. File `index.html` sudah mulai disiapkan (kosong atau *skeleton HTML* sudah ada di branch `final-project`).

14.11 Latihan Mandiri

Latihan 14.1 Eksplorasi Heuristik

- a. Pilih salah satu heuristik dari tabel, lalu buka 3 website berbeda. Catat minimal 1 temuan per website yang berkaitan dengan heuristik tersebut.
- b. Bandingkan kemudahan penggunaan antara kedua website — tuliskan alasan Anda secara singkat.

Bab 15 Workshop Proyek Akhir — Implementasi, Deployment & Progress Clinic

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- mengimplementasikan halaman web statis (HTML + CSS + JS) sesuai *mockup* Figma yang sudah dibuat;
- mendeplöy website ke GitHub Pages agar bisa diakses publik;
- menyelesaikan kendala teknis melalui sesi *clinic* dengan pengajar;
- memastikan pembagian kerja berjalan merata dan setiap anggota memahami kode tanggung jawabnya.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- menyinkronkan *repository* dan memulai implementasi halaman utama;
- menulis struktur HTML semantik dan CSS sesuai *mockup*;
- mendeploy website ke GitHub Pages;
- mengikuti sesi *clinic* untuk perbaikan teknis;
- menyiapkan bahan presentasi untuk minggu 16.

15.1 Peralatan yang Dibutuhkan

- **VS Code** — menulis HTML, CSS, JS.
- **Live Server** (ekstensi VS Code) — *preview* halaman secara langsung.
- **Browser** (Chrome/Firefox) — *testing* dan *debugging*.
- **Figma** — referensi desain saat coding.
- **Git & GitHub** — *version control* dan deployment.
- **GitHub Copilot atau agentic coding tool** (opsional) — bantuan penulisan kode.

15.2 Synchronisasi Repository

Langkah Kerja

1. Buka terminal di VS Code, pastikan berada di *branch* yang benar:

Listing 15.1: Cek branch

```
git status
git checkout final-project
```

2. Ambil perubahan terbaru dari GitHub:

Listing 15.2: Pull perubahan terbaru

```
git pull origin final-project
```

3. Buka file `index.html` yang sudah ada (atau buat jika belum).

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-15/terminal-git-status.png`

Keterangan: Terminal di VS Code — menunjukkan git status branch `final-project`.

Instruksi mahasiswa: Buka terminal di VS Code, jalankan `git status` dan pastikan branch sudah sesuai.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-15/terminal-git-status.png` tersedia.

15.3 Implementasi Halaman Utama

Fokuskan waktu pada satu halaman utama (*landing page / homepage*). Multi-halaman bisa ditambahkan jika waktu memungkinkan.

15.3.1 Struktur HTML Dasar

Buka `src/index.html` dan buat struktur dasar berikut:

Listing 15.3: Struktur dasar HTML semantik

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>[Nama Website Redesign]</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <!-- Header & Navigasi -->
  <header>
    <nav>
      <!-- Logo + menu navigasi sesuai mockup -->
    </nav>
  </header>

  <!-- Hero Section -->
  <section class="hero">
    <!-- Judul, deskripsi singkat, tombol CTA -->
  </section>
```

```

<!-- Konten Utama -->
<main>
  <!-- Bagian-bagian konten sesuai mockup -->
</main>

<!-- Footer -->
<footer>
  <!-- Informasi kontak, copyright, tautan -->
</footer>

<script src="js/script.js"></script>
</body>
</html>

```

Tips

Gunakan tag HTML semantik: `<header>`, `<nav>`, `<main>`, `<section>`, `<footer>`. Hindari penggunaan `<div>` berlebihan tanpa makna. Tambahkan `lang="id"` di tag `<html>` untuk aksesibilitas.

15.3.2 Pembagian Kerja Kelompok

Agar tidak bentrok, gunakan *branch* terpisah per anggota atau bagian file terpisah.

Opsi A: Branch terpisah (disarankan):

Listing 15.4: Branch per anggota

```

# Anggota A
git checkout -b feat/header-navigasi

# Anggota B
git checkout -b feat/konten-utama

# Anggota C
git checkout -b feat/footer-responsive

```

Setiap anggota bekerja di *branch* sendiri, lalu *merge* ke *final-project* setelah selesai.

Opsi B: Bagian file terpisah (lebih sederhana):

- Anggota A: kerjakan `index.html` bagian `<header>` dan `<nav>`.
- Anggota B: kerjakan `index.html` bagian `<main>`.
- Anggota C: kerjakan `index.html` bagian `<footer>` + buat `css/style.css`.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-15/struktur-branch-github.png`

Keterangan: Struktur branch di GitHub — terlihat branch per anggota.

Instruksi mahasiswa: Buka *repository* di GitHub, klik dropdown branch untuk memastikan semua branch anggota sudah ada.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-15/struktur-branch-github.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-15/html-semantic-tags.png`

Keterangan: Struktur HTML di VS Code dengan semantic tags.

Instruksi mahasiswa: Buka `index.html` di VS Code, periksa penggunaan tag semantik.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-15/html-semantic-tags.png` tersedia.

15.3.3 Styling dengan CSS

Buka `css/style.css` dan mulai *styling* berdasarkan *mockup* Figma.

Listing 15.5: CSS dasar — reset, header, hero, main, footer, responsive

```
/* Reset & Base Styles */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva,
    Verdana, sans-serif;
  line-height: 1.6;
  color: #333;
}

/* Header & Navigation */
header {
  background-color: #2c3e50;
  color: white;
  padding: 1rem 2rem;
}

nav {
  display: flex;
  justify-content: space-between;
```

```
align-items: center;
max-width: 1200px;
margin: 0 auto;
}

/* Hero Section */
.hero {
  background-color: #3498db;
  color: white;
  text-align: center;
  padding: 4rem 2rem;
}

/* Main Content */
main {
  max-width: 1200px;
  margin: 0 auto;
  padding: 2rem;
}

/* Footer */
footer {
  background-color: #2c3e50;
  color: white;
  text-align: center;
  padding: 2rem;
  margin-top: 4rem;
}

/* Responsive Design */
@media (max-width: 768px) {
  nav {
    flex-direction: column;
    gap: 1rem;
  }
  .hero {
    padding: 2rem 1rem;
  }
}
```

Tips

Selalu referensikan *mockup* Figma saat menulis CSS — jangan “nebak” ukuran. Gunakan *flexbox* untuk *layout* dan *media query* untuk responsif. Jangan lupa `box-sizing: border-box` di *reset* — ini mencegah masalah *layout* umum.

15.3.4 Interaktivitas JavaScript (Opsional)

Jika *mockup* memerlukan elemen interaktif (misal: menu *hamburger*, *accordion* FAQ, tab navigasi):

Listing 15.6: Toggle menu mobile (vanilla JS)

```
// js/script.js

// Contoh: Toggle menu mobile
const menuToggle =
  document.querySelector('.menu-toggle');
const navMenu =
  document.querySelector('.nav-menu');

if (menuToggle) {
  menuToggle.addEventListener('click', () => {
    navMenu.classList.toggle('active');
  });
}
```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-15/website-desktop-browser.png

Keterangan: Website tampil di browser (Live Server) — versi desktop.

Instruksi mahasiswa: Buka index.html melalui Live Server di VS Code, pastikan tampilan sesuai *mockup*.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan \includegraphics setelah aset
assets/images/minggu-15/website-desktop-browser.png tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: assets/images/minggu-15/website-mobile-browser.png

Keterangan: Website tampil di browser — versi mobile (toggle responsive).

Instruksi mahasiswa: Gunakan DevTools → *Toggle Device Toolbar* untuk memeriksa tampilan mobile.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan \includegraphics setelah aset
assets/images/minggu-15/website-mobile-browser.png tersedia.

15.4 Sesi Clinic / Mentoring

Pengajar berkeliling ke setiap kelompok selama sesi ini. Manfaatkan waktu untuk:

- review cepat kode dan struktur *repo*;
- tanya-jawab tentang bagian kode yang belum dipahami;
- perbaiki masalah teknis yang ditemukan.

Catatan

Pastikan setiap kelompok bisa menjawab pertanyaan berikut:

1. Bagaimana cara website ini mengatasi *pain point* #1 dari hasil riset?
2. Siapa yang mengerjakan bagian apa? Apakah semua anggota sudah punya tanggung jawab?
3. Apakah website ini responsif di mobile? Bisa didemo?
4. Apakah ada penggunaan *agentic coding tools*? Jika ya, bagian mana?

15.5 Deployment ke GitHub Pages

15.5.1 Persiapan Repository

Langkah Kerja

1. Pastikan file sudah ter-*commit*:

Listing 15.7: Commit implementasi

```
git add .
git commit -m "feat: implementasi halaman utama sesuai mockup Figma"
git push
```

2. Pastikan *branch* main atau final-project berisi kode terbaru. Jika kode di *branch* lain, *merge* terlebih dahulu:

Listing 15.8: Merge ke main

```
git checkout main
git merge final-project
git push origin main
```

15.5.2 Aktifkan GitHub Pages

Langkah Kerja

1. Buka *repository* di GitHub.
2. Klik tab **Settings** → geser ke bagian kiri → klik **Pages**.
3. Di bagian **Source**:
 - Pilih **“Deploy from a branch”**.
 - Pilih *branch*: main (atau final-project).
 - Pilih folder: / (root) atau /docs (tergantung struktur).
 - Klik **Save**.
4. Tunggu 1–2 menit, lalu *refresh* halaman Settings → Pages.
5. Klik tautan yang muncul di bagian atas (contoh: <https://username.github.io/repo-name/>).

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-15/github-pages-settings.png`

Keterangan: GitHub Settings → Pages — konfigurasi source.

Instruksi mahasiswa: Buka Settings → Pages di *repository* GitHub, pastikan *branch* dan folder sudah benar.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-15/github-pages-settings.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-15/website-deployed-browser.png`

Keterangan: Website yang sudah ter-deploy di browser — URL GitHub Pages terlihat.

Instruksi mahasiswa: Buka tautan GitHub Pages di browser, pastikan halaman utama tampil dengan benar.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-15/website-deployed-browser.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-15/website-deployed-mobile.png`

Keterangan: Website di mobile view setelah deploy.

Instruksi mahasiswa: Gunakan DevTools untuk memeriksa tampilan mobile setelah deployment.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-15/website-deployed-mobile.png` tersedia.

15.5.3 Update Deployment

Setiap kali ada perubahan baru, cukup *push* ke *branch* yang sudah dikonfigurasi:

Listing 15.9: Push untuk update otomatis

```
git add .
git commit -m "fix:␣perbaikan␣layout␣header"
git push
```

GitHub Pages akan otomatis *update* dalam $\pm$ 1–2 menit.

15.6 Troubleshooting Deployment

Tabel 15.1: Masalah Umum Deployment — GitHub Pages

Masalah	Solusi
Website tidak muncul setelah deploy	Tunggu 2–3 menit. Jika masih tidak muncul, cek tab Actions di <i>repo</i> — lihat apakah <i>build</i> berhasil.
Halaman 404 Not Found	Pastikan file utama bernama <code>index.html</code> dan berada di <i>folder root</i> (atau <i>folder</i> yang dipilih di Settings).
CSS/JS tidak termuat	Cek path file di HTML — gunakan <i>relative path</i> (<code>css/style.css</code>), bukan <i>absolute path</i> (<code>/css/style.css</code>).
Gambar tidak muncul	Pastikan file gambar ada di <i>repo</i> dan path-nya benar. Perhatikan huruf besar/kecil (<i>case-sensitive</i> di GitHub Pages).
Deploy dari <i>branch</i> selain <i>main</i>	Di Settings → Pages, ganti <i>branch source</i> ke <i>branch</i> yang benar.
Subfolder tidak terbaca	Pastikan <code>index.html</code> ada di <i>root folder</i> yang dipilih di Settings.

15.7 Troubleshooting Umum

Tabel 15.2: Masalah Umum Implementasi dan Solusi

Masalah	Solusi
CSS tidak ter-apply	Cek <code><link></code> di HTML — pastikan path benar. Cek juga apakah file CSS benar-benar ada di folder yang tepat.
Gambar tidak tampil	Gunakan path relatif: <code>assets/gambar.png</code> . Pastikan nama file tepat (<i>case-sensitive</i>).
Layout pecah di mobile	Pastikan <code><meta name="viewport"></code> ada di <code><head></code> . Cek <i>media query</i> di CSS. Gunakan DevTools → <i>Toggle Device Toolbar</i> .
JavaScript tidak jalan	Buka Console di DevTools (F12 → Console). Cek <i>error message</i> . Pastikan file JS sudah di-link di HTML sebelum <code></body></code> .
Git <i>merge conflict</i>	Buka file yang conflict → cari <code><<<<<<</code> → pilih versi yang benar → hapus marker conflict → <i>commit</i> .
Flexbox tidak bekerja	Pastikan <i>parent element</i> punya <code>display: flex</code> . Cek apakah ada <code>width</code> atau <code>height</code> yang membatasi.

15.8 Persiapan Presentasi

15.8.1 Struktur Presentasi

Buat file `docs/presentasi-outline.md` dengan struktur berikut:

1. **Pembukaan (1 menit)** — Perkenalan kelompok & nama website; URL asli vs URL redesign.
2. **Riset UI/UX (3 menit)** — *Screenshot* halaman asli; top 3 *pain point*; temuan *benchmarking*.
3. **Desain Ulang (3 menit)** — *Screenshot mockup* Figma; 3 keputusan desain utama beserta kaitannya dengan temuan riset.

4. **Implementasi (3 menit)** — *Live demo* website; teknologi yang digunakan; demo responsif.
5. **Pembagian Kerja & Penggunaan AI (2 menit)** — Tabel kontribusi; dokumentasi penggunaan *agentic coding tools*.
6. **Penutup (1 menit)** — Refleksi dan Q&A.

15.8.2 Bagikan Peran Presentasi

- Anggota 1: Pembukaan & Riset.
- Anggota 2: Desain Ulang.
- Anggota 3: Implementasi & Demo.
- Pembagian Kerja & Penutup: diskusikan sendiri.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-15/outline-presentasi.png`

Keterangan: Outline presentasi di Markdown.

Instruksi mahasiswa: Buka `docs/presentasi-outline.md` di VS Code, pastikan seluruh segmen sudah terisi.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-15/outline-presentasi.png` tersedia.

15.9 Final Commit dan Documentation

Langkah Kerja

1. Pastikan `README.md` di *root repo* sudah lengkap — nama website, anggota kelompok (beserta NIM dan tugas), *tech stack*, cara menjalankan, tautan deploy, tautan Figma, dan dokumentasi penggunaan *agentic coding tools* (jika ada).
2. *Commit* dan *push*:

Listing 15.10: Final commit documentation

```
git add .
git commit -m "docs: lengkapi README dan siapkan bahan presentasi"
git push
```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-15/readme-lengkap-github.png`

Keterangan: `README.md` yang sudah lengkap di GitHub.

Instruksi mahasiswa: Buka *repository* di GitHub, pastikan `README.md` menampilkan seluruh informasi yang diperlukan.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-15/readme-lengkap-github.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-15/commit-history-contributors.png`

Keterangan: Commit history — terlihat kontribusi per anggota.

Instruksi mahasiswa: Buka tab Insights → Contributors di *repository* GitHub untuk memastikan kontribusi merata.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-15/commit-history-contributors.png` tersedia.

15.10 Panduan Agentic Coding Tools (Opsional)

Jika kelompok menggunakan GitHub Copilot, Claude Code, atau tools serupa, dokumentasikan penggunaannya.

Catatan

Tips penggunaan yang bertanggung jawab:

1. Selalu *review* kode yang dihasilkan — jangan langsung *copy-paste*.
2. Pahami setiap baris — siapkan diri untuk menjelaskan saat sesi tanya-jawab minggu 16.
3. Gunakan sebagai bantuan, bukan pengganti — anggota harus bisa memodifikasi kode sendiri.
4. Dokumentasikan semua *prompt* di `README.md` atau file terpisah.

15.11 Output yang Diharapkan

Setelah sesi ini, setiap kelompok harus memiliki:

- Halaman HTML/CSS/JS utama (`src/index.html`, `src/css/`, `src/js/`).
- Website ter-deploy (tautan GitHub Pages).
- Responsif di mobile & desktop (sudah diuji di browser).
- `README.md` lengkap di *root repository*.
- *Commit history* per anggota (terlihat di GitHub → Insights → Contributors).
- *Outline* presentasi (`docs/presentasi-outline.md`).
- Semua perubahan ter-*push* ke *branch* `final-project` atau `main`.

15.12 Tautan Referensi

- GitHub Pages Docs: <https://docs.github.com/en/pages>
- MDN: HTML Semantics: https://developer.mozilla.org/en-US/docs/Guide/HTML/Sections_and_outlines_of_an_HTML5_document
- CSS Flexbox Guide: <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
- CSS Media Queries: https://developer.mozilla.org/en-US/docs/Web/CSS/Media_Queries
- Figma to Code: <https://www.figma.com/best-practices/from-design-to-code/>

15.13 Persiapan untuk Minggu 16 (Presentasi)

Sebelum pertemuan minggu depan, pastikan:

1. Website sudah ter-deploy dan bisa diakses semua orang.
2. Semua anggota bisa menjelaskan kode bagian masing-masing.
3. *Outline* presentasi sudah final dan semua anggota tahu siapa bicara kapan.
4. Siapkan demo langsung — buka website di browser, siap untuk *resize* dan navigasi.
5. Latihan presentasi — coba presentasikan dalam waktu ± 10 menit.

15.14 Latihan Mandiri

Latihan 15.1 Eksplorasi Deployment

- a. Buat *branch* terpisah di *repository* pribadi, tambahkan perubahan kecil pada file HTML, lalu *merge* ke *main*. Amati apakah GitHub Pages ter-*update* secara otomatis.
- b. Buka DevTools di browser, resize jendela dari 1440px ke 375px, dan identifikasi setidaknya 2 masalah *layout* pada halaman Anda. Perbaiki menggunakan *media query*.

Bab 16 Presentasi, Refleksi & Finalisasi Proyek Akhir

Capaian Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

- mempresentasikan proses proyek redesign dari riset → desain → implementasi → deployment;
- menjawab pertanyaan tentang kode dan keputusan desain secara individual;
- merefleksikan perkembangan diri selama semester dalam mata kuliah ini;
- menyelesaikan seluruh *deliverable* proyek akhir.

Tujuan Praktik

Pada sesi praktik ini, mahasiswa akan:

- melakukan pengecekan kesiapan teknis sebelum presentasi;
- mempresentasikan hasil proyek redesign di depan kelas;
- menjawab pertanyaan individual dari pengajar;
- mengisi refleksi diri akhir semester;
- melakukan finalisasi *repository*.

16.1 Peralatan yang Dibutuhkan

- **Browser** — *live demo* website.
- **GitHub** — menunjukkan *repository* dan *commit history*.
- **Figma** — menunjukkan file desain.
- **Slide presentasi** — pembukaan dan penutup (Google Slides / Markdown).
- **VS Code** — menunjukkan kode saat sesi tanya-jawab.

16.2 Persiapan Sebelum Presentasi

16.2.1 Cek Kesiapan Teknis

Langkah Kerja

1. Buka website via GitHub Pages di browser — coba navigasi seluruh halaman.
2. *Resize* browser dari desktop ke mobile — pastikan tampilan responsif.
3. Pastikan *repository* GitHub bersih — tidak ada file *temporary* atau file yang tidak perlu.
4. Pastikan README.md sudah lengkap.
5. Pastikan semua anggota bisa mengakses *repository* dari laptop masing-masing.

16.2.2 Siapkan Tab Browser

Buka semua tab berikut di browser sebelum giliran presentasi:

1. Tab 1: Website hasil redesign (GitHub Pages).
2. Tab 2: Repository GitHub (terlihat *commit history* & README).

3. Tab 3: File desain Figma.
4. Tab 4: File *heuristic evaluation* di GitHub (atau VS Code).

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-16/persiapan-tab-browser.png`

Keterangan: Persiapan presentasi — browser dengan 4 tab terbuka.

Instruksi mahasiswa: Buka browser dan pastikan keempat tab sudah siap sebelum giliran presentasi kelompok Anda.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-16/persiapan-tab-browser.png` tersedia.

16.3 Presentasi Kelompok

Gunakan *outline* yang sudah disiapkan di minggu 15. Waktu per kelompok: 8–10 menit presentasi + 5 menit tanya-jawab.

16.3.1 Pembukaan (± 1 Menit)

- Perkenalan anggota kelompok.
- Nama website yang diriset dan URL-nya.
- URL website redesign (GitHub Pages).

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-16/slide-pembukaan.png`

Keterangan: Slide pembukaan — nama kelompok, nama website.

Instruksi mahasiswa: Siapkan slide pembukaan yang memuat nama kelompok dan nama website yang diredesign.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-16/slide-pembukaan.png` tersedia.

16.3.2 Riset UI/UX (± 2–3 Menit)

Tampilkan:

1. *Screenshot* halaman asli website yang diriset.
2. Top 3 *pain point* dari *heuristic evaluation* — jelaskan apa masalahnya dan mengapa itu masalah bagi pengguna.
3. 1–2 temuan *benchmarking* — apa yang dipelajari dari website lain.

Catatan

Contoh penyampaian: “Di halaman asli, tombol ‘Ajukan Permohonan’ tidak terlihat karena warnanya menyatu dengan *background*. Ini melanggar heuristic #1 — Visibility of System Status.”

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-16/slide-riset-pain-point.png`

Keterangan: Tampilan slide riset — pain point utama.

Instruksi mahasiswa: Pastikan slide riset memuat *screenshot* halaman asli dan daftar *pain point* utama.

[**Tempatkan tangkapan layar di sini**]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-16/slide-riset-pain-point.png` tersedia.

16.3.3 Desain Ulang (± 2–3 Menit)

Tampilkan:

1. *Mockup* Figma (desktop & mobile) — tunjukkan langsung di Figma.
2. 3 keputusan desain utama, masing-masing dijelaskan dengan kaitannya terhadap temuan riset.

Tips

Contoh penyampaian: “Keputusan 1: Kami memindahkan tombol CTA ke posisi lebih prominent. Ini karena di riset ditemukan bahwa tombol utama sulit ditemukan (*pain point* #1).”

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-16/mockup-figma-desktop.png`

Keterangan: Mockup Figma — tampilan desktop.

Instruksi mahasiswa: Tunjukkan *mockup* desktop di Figma, jelaskan keputusan desain utama.

[**Tempatkan tangkapan layar di sini**]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-16/mockup-figma-desktop.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-16/mockup-figma-mobile.png`

Keterangan: Mockup Figma — tampilan mobile.

Instruksi mahasiswa: Tunjukkan *mockup* mobile di Figma, jelaskan bagaimana desain beradaptasi.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-16/mockup-figma-mobile.png` tersedia.

16.3.4 Live Demo ($\pm$ 2 Menit)

Langkah Kerja

1. Buka website di browser (GitHub Pages).
2. Tunjukkan fitur utama: navigasi, konten utama, responsif (resize browser dari desktop ke mobile).
3. Tunjukkan *commit history* di GitHub — siapa mengerjakan apa.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-16/live-demo-desktop.png`

Keterangan: Live demo website — tampilan desktop.

Instruksi mahasiswa: Buka website di browser, tunjukkan navigasi dan konten utama.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-16/live-demo-desktop.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-16/live-demo-mobile.png`

Keterangan: Live demo website — tampilan mobile.

Instruksi mahasiswa: Resize browser ke ukuran mobile, tunjukkan responsivitas.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-16/live-demo-mobile.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-16/commit-history-github.png`

Keterangan: Commit history di GitHub.

Instruksi mahasiswa: Buka tab GitHub, tunjukkan *commit history* untuk membuktikan kontribusi per anggota.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-16/commit-history-github.png` tersedia.

16.3.5 Pembagian Kerja dan Penggunaan AI (± 1 Menit)

- Tampilkan tabel pembagian tugas beserta jumlah *commit* per anggota.
- Jelaskan tools apa yang dipakai (jika ada), di bagian mana, dan tunjukkan dokumentasi *prompt*-nya.

16.3.6 Penutup (± 1 Menit)

- Satu hal terpenting yang dipelajari dari proyek ini.
- Satu tantangan terbesar yang dihadapi.

16.4 Tanya-Jawab Individual

Setelah presentasi, pengajar mengajukan pertanyaan kepada setiap anggota secara individual (± 5 menit per kelompok). Berikut contoh pertanyaan yang mungkin ditanyakan berdasarkan bagian yang dikerjakan:

Tabel 16.1: Contoh Pertanyaan Tanya-Jawab Individual

Header/Navigasi	Konten Utama	Footer/Responsive
Bagaimana cara kerja navigasi di website ini? Jelaskan HTML-nya.	Kenapa formulir ini diletakkan di posisi ini?	Bagaimana cara <i>media query</i> bekerja di CSS ini?
Apa heuristik yang paling relevan dengan perubahan di header?	Bagaimana website ini mengatasi <i>pain point</i> #2?	Apa bedanya <i>flex-direction: row</i> dan <i>column</i> di kode ini?
Jika saya ingin mengubah warna header, baris CSS mana yang perlu diubah?	Bisakah kamu menjelaskan alur pengguna mengisi formulir ini?	Di mana batas <i>breakpoint</i> untuk mobile di CSS ini?

Tips

Untuk mahasiswa: Buka VS Code saat sesi tanya-jawab — tunjukkan kode langsung. Jujur jika tidak tahu, tapi tunjukkan usaha untuk mencari tahu. Hubungkan jawaban dengan riset — misal: “Ini kami buat karena di riset ditemukan. . .”

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-16/tanya-jawab-vscode.png`

Keterangan: Sesi tanya-jawab — anggota menjelaskan kode di VS Code.

Instruksi mahasiswa: Buka VS Code di layar, navigasi ke file yang relevan dengan pertanyaan pengajar.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-16/tanya-jawab-vscode.png` tersedia.

16.5 Refleksi Individu

Setelah semua kelompok selesai presentasi, setiap mahasiswa mengisi refleksi diri. Buka file `reflection.md` di *repo* (atau gunakan form *online* yang disediakan pengajar) dan jawab pertanyaan-pertanyaan berikut.

16.5.1 Perkembangan Skill

Buat tabel perbandingan perkembangan skill dari awal semester ke akhir semester untuk kompetensi: HTML, CSS, JavaScript, Figma, Git/GitHub, dan *Heuristic Evaluation*.

16.5.2 Refleksi Proyek Akhir

Jawab pertanyaan-pertanyaan berikut secara jujur dan reflektif:

1. Apa yang paling menantang dari proyek ini?
2. Apa yang paling memuaskan dari hasil kerja kelompok?
3. Jika bisa mengulang, apa yang akan dilakukan berbeda?
4. Bagian mana dari kode yang paling dipahami dan bisa dijelaskan dengan baik?
5. Bagian mana yang masih perlu dipelajari lebih lanjut?

16.5.3 Refleksi Pembelajaran

Jawab pertanyaan-pertanyaan berikut:

1. Satu hal terpenting yang dipelajari di mata kuliah ini.
2. Bagaimana mata kuliah ini mengubah cara pandang terhadap *web development*.
3. Apakah ada topik yang ingin dipelajari lebih lanjut?

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-16/refleksi-terisi.png`

Keterangan: Formulir refleksi yang sudah diisi.

Instruksi mahasiswa: Buka `reflection.md` di VS Code, pastikan seluruh pertanyaan sudah dijawab.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-16/refleksi-terisi.png` tersedia.

Langkah Kerja

1. Isi seluruh pertanyaan refleksi di `reflection.md`.

2. Commit dan push:

Listing 16.1: Commit refleksi

```
git add reflection.md
git commit -m "docs: refleksi akhir semester - [nama]"
git push
```

16.6 Finalisasi Repository

Pastikan *repository* dalam kondisi final dan siap untuk portofolio.

16.6.1 Review Checklist Final

Langkah Kerja

1. Pastikan `index.html` berfungsi dengan benar.
2. Pastikan CSS responsif (desktop & mobile).
3. Pastikan JavaScript berfungsi (jika ada).
4. Pastikan gambar & aset semua tampil.
5. Pastikan `README.md` lengkap.
6. Pastikan folder `research/` berisi *heuristic evaluation & benchmarking*.
7. Pastikan file Figma sudah di-*share* (tautan di `README`).
8. Pastikan website ter-deploy di GitHub Pages.
9. Pastikan *commit history* mencerminkan kontribusi per anggota.
10. Pastikan tidak ada file *temporary* (`.DS_Store`, `Thumbs.db`, `dsb`).

16.6.2 Tambahkan File `.gitignore`

Buat `.gitignore` di *root repository* jika belum ada:

Listing 16.2: Contoh file `.gitignore`

```
# OS files
.DS_Store
Thumbs.db

# Editor files
.vscode/settings.json
*.swp
*.swo

# Node modules (jika menggunakan Tailwind)
node_modules/

# Build output
dist/
```

16.6.3 Final Commit

Listing 16.3: Final commit proyek akhir

```
git add .
git commit -m "chore: finalisasi repository proyek akhir"
git push
```

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-16/repository-final-github.png`

Keterangan: Repository GitHub — tampilan root yang bersih dan lengkap.

Instruksi mahasiswa: Buka *repository* di GitHub, pastikan seluruh struktur sudah sesuai dan tidak ada file yang tidak perlu.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-16/repository-final-github.png` tersedia.

Tangkapan Layar (placeholder)

Jalur aset yang diharapkan: `assets/images/minggu-16/commit-history-akhir.png`

Keterangan: Commit history akhir — terlihat kontribusi merata per anggota.

Instruksi mahasiswa: Buka tab Insights → Contributors, pastikan kontribusi merata di antara anggota.

[Tempatkan tangkapan layar di sini]

Ganti blok ini dengan `\includegraphics` setelah aset `assets/images/minggu-16/commit-history-akhir.png` tersedia.

16.7 Struktur Repository Final

Berikut struktur *repository* yang diharapkan setelah proyek akhir selesai:

Listing 16.4: Struktur folder repository final

```
nama-repo/
  README.md
  .gitignore
  research/
    heuristic-evaluation.md
    benchmarking.md
    rasional-desain.md
  src/
    index.html
    css/
```

```

style.css
js/
  script.js
assets/
  logo.png
design/
  figma-link.txt
docs/
  presentasi-outline.md
reflection.md

```

16.8 Deliverable Akhir Proyek

Pastikan seluruh *deliverable* berikut sudah selesai:

- Dokumen hasil riset UI/UX (*research/heuristic-evaluation.md*).
- Dokumen *benchmarking* (*research/benchmarking.md*).
- File Figma (*wireframe + mockup*) — tautan di README.
- Repository GitHub (*source code + README*).
- Website ter-deploy (GitHub Pages) — tautan di README.
- Pembagian tugas di README (*README.md*).
- Dokumentasi penggunaan *agentic coding* (jika ada).
- Refleksi individu (*reflection.md*).

16.9 Troubleshooting

Tabel 16.2: Masalah Umum dan Solusi — Minggu 16

Masalah	Solusi
Website tidak bisa diakses saat presentasi	Buka file <code>index.html</code> langsung dari VS Code (Live Server) sebagai cadangan. Atau unduh <i>repo</i> dan buka secara lokal.
Figma tidak bisa dibuka	Pastikan akun Figma sudah <i>login</i> . Jika di lab kampus, minta bantuan admin untuk install Figma <i>desktop</i> .
Lupa <i>commit</i> bagian tertentu	Cek <code>git status</code> → jika ada file yang belum <i>ter-commit</i> , lakukan <i>commit</i> sekarang. Jangan lupa <i>push</i> .
Tidak sempat mengisi refleksi	Refleksi bisa diisi dan di- <i>push</i> setelah kelas. Deadline: 1 × 24 jam setelah presentasi.
Koneksi internet mati saat presentasi	Gunakan versi lokal (Live Server) sebagai cadangan. Pastikan sudah <i>download repo</i> sebelumnya.
Anggota tidak bisa hadir	Hubungi pengajar sebelum kelas. Kemungkinan presentasi dijadwalkan ulang atau dilakukan <i>remote</i> .
Website <i>deploy</i> expired/blank	Cek Settings → Pages di GitHub. Pastikan <i>branch source</i> masih aktif. <i>Push</i> ulang jika perlu.
<i>Merge conflict</i> di final <i>commit</i>	Selesaikan conflict: buka file → pilih versi yang benar → hapus marker <<<<<<< → <i>commit</i> .

16.10 Tautan Referensi

- GitHub Pages: <https://docs.github.com/en/pages>
- Nielsen Norman Group: <https://www.nngroup.com/articles/ten-usability-heuristics/>
- Figma: <https://www.figma.com>
- MDN Web Docs: <https://developer.mozilla.org>
- GitHub Skills: <https://skills.github.com>

16.11 Latihan Mandiri

Latihan 16.1 Refleksi dan Portofolio

- a. Buka *repository* proyek akhir Anda di GitHub. Tinjau *commit history* dan identifikasi minggu mana yang memiliki kontribusi terbanyak. Apakah ada pola yang bisa diperbaiki di proyek berikutnya?
- b. Buka website hasil redesign Anda di dua browser berbeda (misal Chrome dan Firefox). Catat minimal 1 perbedaan tampilan dan jelaskan penyebabnya.